Spec and test gaps: MCI mailbox

Repository `targets/caliptra-ss` at commit `69a8afa1eeb8`. Report generated 2026-11-10 22:12 UTC by chipbag.

Each finding below gives the gap, the evidence for it as exact quotes at their file and line, and how it was verified: an agent searched the repository to refute the finding, and the finding was kept only if that failed. Every quote was checked against the file at the cited lines. The report does not recommend a fix.

Summary

#	Category	Verified	Withdrawn	Inconclusive
1	Spec too ambiguous to test	7	13	0
2	Spec contradicts itself	1	4	0
3	Test contradicts its own description	12	4	0
4	Spec and test disagree	0	2	0
5	Spec not tested	51 (173 items)	1	1

Scope

Spec sources (4): `docs/CaliptraSSHardwareSpecification.md`, `docs/CaliptraSSIntegrationSpecification.md`, `src/mci/rtl/mci_reg.rdl`, `src/mci/rtl/mcu_mbox_csr.rdl`

Verification sources (40 files): `src/integration/stimulus/` (4), `src/integration/testbench/sv_tests/mci/` (3), `docs/` (2), `src/integration/test_suites/mcu_mbox_rand_rdwr/` (2), `src/integration/test_suites/smoke_test_mcu_mbox/` (2), `src/integration/test_suites/smoke_test_mcu_mbox_axi_param/` (2), `src/integration/test_suites/smoke_test_mcu_mbox_burst/` (2), `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/` (2), `src/integration/test_suites/smoke_test_mcu_mbox_invalid_axi/` (2), `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/` (2), `src/integration/test_suites/smoke_test_mcu_mbox_target_user/` (2), `src/integration/test_suites/smoke_test_mcu_mbox_usr_lock_out_zero/` (2), `src/integration/test_suites/smoke_test_mcu_mbox_valid_user/` (2), `src/integration/test_suites/smoke_test_mcu_mbox_write_user_lock/` (2), `src/integration/asserts/` (1), `src/integration/stimulus/testsuites/` (1), `src/integration/test_suites/mcu_mbox_lock_return_one_during_zeroize/` (1), `src/integration/test_suites/smoke_test_mbox_csr_zeros_after_release/` (1), `src/integration/test_suites/smoke_test_mcu_mbox_csrs_warm_rst/` (1), `src/integration/test_suites/smoke_test_mcu_mbox_ecc/` (1), `src/integration/test_suites/smoke_test_mcu_mbox_strb_wr_csr/` (1), `src/integration/test_suites/smoke_test_mcu_mbox_zeroize/` (1), `src/mci/coverage/` (1). The full list is in the manifest.

487 requirements after merging duplicates; 363 need a test (the rest hold by construction). 39 tests (39 with source), 46 test plan items. 185 requirements are matched to a test.

Files the verifier used as evidence that are not in the manifest: `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c`, `src/integration/testbench/sv_tests/mci/mci_common_tasks.svh`.

1. Spec too ambiguous to test

1.1 Post-MCI-reset SRAM_OWNER encoding: Root (2) or lock holder (1)?

`f_7dbfd12edb46` · found during verification · severity high

Evidence:

- `docs/CaliptraSSHardwareSpecification.md:1395`: `| MCI reset | Root, as the reset lock holder |` — The event table names Root as the owner after MCI reset, but qualifies it as the "reset lock holder".
- `docs/CaliptraSSHardwareSpecification.md:1444-1445`: `| 1 | Requester/lock holder |` — Encoding 1 is labelled "Requester/lock holder", so a lock holder owner can legitimately read back as 1. This supports the test's expectation.
- `src/integration/test_suites/smoke_test_mcu_mbox_csrs_warm_rst/smoke_test_mcu_mbox_csrs_warm_rst.c:113-114`: `if(mcu_mbox_read_sram_owner(mbox_num) != MCU_MBOX_SRAM_OWNER_REQUESTER) {` — The test expects encoding 1 and its message says "reset lock holder", so its author read "lock holder" as the encoding-1 owner.

Verified: The spec's event table says the owner after MCI reset is "Root, as the reset lock holder". The SRAM_OWNER encoding table separates 1 (Requester/lock holder) from 2 (Root). The test expects 1. A spec change that states the exact SRAM_OWNER value after reset would resolve it.

Searched:

- SRAM_OWNER encoding in the spec and library header in `docs/CaliptraSSHardwareSpecification.md, caliptra_ss_lib.h`: 1 = Requester/lock holder, 2 = Root. The spec does not say which encoding applies after reset.

1.2 The associated MBOX0_VALID_AXI_USER register is valid only once locked by this bit.

`f_4a7ed506cf39` · vague · severity high · section: src/mci/rtl/mci_reg.rdl > mci_reg > MBOX0_AXI_USER_LOCK

Gap: It is not stated what 'valid' means observably for the associated register before it is locked.

Spec says:

- `src/mci/rtl/mci_reg.rdl:831`: `Associated MBOX0_VALID_AXI_USER register is only valid once locked by this bit.`

Not answered by the spec: Before MBOX0_AXI_USER_LOCK[i].LOCK is set, is MBOX0_VALID_AXI_USER[i] ignored by the mailbox AXI_USER filter? Is it ignored completely, so a matching AxUSER is rejected unless it matches another locked entry or the default user? And once LOCK is set, is that entry's value used for matching from the next transaction onward?

Verified: No spec text says directly that an unlocked MBOX0_VALID_AXI_USER entry is ignored by the filter. The hardware spec defines trusted users and the AXI error for untrusted ones. It never ties "valid" to "locked" in observable terms. It also never says when a newly locked entry takes effect. A reader can infer the intended behavior, but the spec does not state it.

Searched:

- VALID_AXI_USER and AXI_USER_LOCK in docs and rdl in `docs/, src/mci/rtl`: Only the vague rdl text, integration strap tables and requirement CSS_Axi_9. None of them define the behavior of an unlocked entry.
- MCU Mailbox Limited Trusted AXI users sections in `HardwareSpecification.md:1295, IntegrationSpecification.md:2291`: They describe the trusted set and the AXI error for untrusted users. They do not say what happens to a matching but unlocked entry, or when a locked entry takes effect.

Looked at and ruled out:

- `docs/CaliptraSSHardwareSpecification.md:1295-1299`: It gives the AXI error for untrusted users. Reading an unlocked entry as untrusted is an inference, not stated text. The next-transaction timing is also unspecified.

1.3 BYTE accesses are not supported by the MCU mailbox SRAM.

`f_07a2b69adf09` · missing detail · severity medium · section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Fully addressable SRAM

Gap: The behavior when a BYTE access is attempted is not specified.

Spec says:

- `docs/CaliptraSSHardwareSpecification.md:1458`: `BYTE accesses are not supported by the SRAM.`

Not answered by the spec: For a sub-DWORD access to the MCU mailbox SRAM (AxSIZE<2 or partial WSTRB), what AXI response is returned, and is any SRAM data modified?

Verified: No other spec text says what happens on a sub-DWORD access to the MCU mailbox SRAM. The only statement is "Accesses must be DWORD aligned and BYTE accesses are not supported by the SRAM." The AXI response and any data modification are unspecified.

Searched:

- byte access / sub-DWORD / partial write in markdown docs in `**/*.md`: Only the cited line, the TL-UL README (unrelated), and an integration spec row about a different SRAM.
- mcu mbox with unaligned/error/SLVERR/strobe/byte in md, rdl and txt files in `repo docs`: Only an unrelated CSS_Mem_6 row.

Looked at and ruled out:

- `docs/CaliptraSSIntegrationSpecification.md:2020`: Concerns a different SRAM (the FW exec region), not the MCU mailbox SRAM.

- `docs/CaliptraSSIntegrationSpecification.md:3239`: Concerns an external staging area, not sub-DWORD accesses.

1.4 Accesses to the MCU mailbox SRAM must be DWORD aligned.

`f_67fe7c20865e` · missing detail · severity medium · section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Fully addressable SRAM

Gap: The behavior on a misaligned access (error response, ignore, etc.) is not specified.

Spec says:

- `docs/CaliptraSSHardwareSpecification.md:1458`: `Accesses must be DWORD aligned`

Not answered by the spec: What happens on a non-DWORD-aligned AXI access to the MCU mailbox SRAM? Specifically: what response code is returned, is a write committed (and to which address), and what data does a read return?

Verified: No spec text says what happens on a misaligned access to the MCU mailbox SRAM. The other alignment statements found cover different blocks: the MCU SRAM, MCI CSRs and USB AXI.

Searched:

- unaligned/misaligned/DWORD aligned in all .md files in `repo docs`: Only the MCU SRAM, MCI CSR (CPU fault), USB, and DMA byte-count statements. None cover the MCU mailbox SRAM.

Looked at and ruled out:

- `docs/CaliptraSSHardwareSpecification.md:1885`: Covers MCU SRAM, not the mailbox SRAM, and only says undefined behavior.
- `docs/CaliptraSSIntegrationSpecification.md:3000`: Covers the USB AXI interfaces only.
- `docs/CaliptraSSIntegrationSpecification.md:1025`: Covers MCU CPU accesses to MCI CSRs, not the mailbox SRAM.

1.5 CMD_STATUS is unused by MCU when MCU holds the lock.

`f_4ed0758b1ae5` · vague · severity medium · section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox MCU Access

Gap: 'Unused' does not say whether MCU writes are ignored, blocked, or whether reads return a defined value.

Spec says:

- `docs/CaliptraSSHardwareSpecification.md:1534`: `` `CMD_STATUS` | Unused ``

Not answered by the spec: When MCU holds the lock, is an MCU write to CMD_STATUS blocked (register unchanged) or accepted (value stored but ignored by the protocol)? What value does a read return in that state?

Verified: No other spec text says what happens to an MCU write to CMD_STATUS, or what a read returns, when MCU holds the lock. The other passages only repeat that CMD_STATUS is "not used" in that case.

Further evidence found by the verifier:

- `docs/CaliptraSSHardwareSpecification.md:1436-1437` — Repeats the same vague wording. It says nothing about write blocking or the read value.

  > `` `CMD_STATUS` is not used when MCU is the Requester, because there is no external Requester to serve. ``

Searched:

- cmd_status, case-insensitive, in md/rdl/rst/txt files in `whole repo`: Hits are the hardware spec (lines 1365-1534), the integration spec sequence diagrams (2362-2416) and mcu_mbox_csr.rdl. None defines write or read behavior when MCU holds the lock.

Looked at and ruled out:

- `src/mci/rtl/mcu_mbox_csr.rdl:140`: The field description says 'Not used when MCU is itself the Requester', which is the same vague wording.

1.6 Each bit corresponds to locking the associated MBO0X_VALID_AXI_USER register.

`f_46eb300913e4` · unclear referent · severity medium · section: src/mci/rtl/mci_reg.rdl > mci_reg > MBOX1_AXI_USER_LOCK

Gap: The register name MBO0X_VALID_AXI_USER appears to be a typo, and 'each bit' does not match the single-bit LOCK field.

Spec says:

- `src/mci/rtl/mci_reg.rdl:847`: `Each bit corresponds to locking the associated MBO0X_VALID_AXI_USER register.`

Not answered by the spec: Does MBOX1_AXI_USER_LOCK[i] lock (make read-only) and validate MBOX1_VALID_AXI_USER[i] for the same index i, with no effect on any MBOX0_VALID_AXI_USER register?

Verified: No other spec text settles the question. The MBOX1 lock description is a copy of the MBOX0 one and still names MBOX0_VALID_AXI_USER. The integration spec only refers to `MBOX*_VALID_AXI_USER` and `MBOX*_AXI_USER_LOCK` generically. Nothing states that index i of one lock register maps to index i of the matching VALID_AXI_USER array.

Searched:

- AXI_USER_LOCK / VALID_AXI_USER in all markdown and rdl files in `repo-wide`: Only mci_reg.rdl and the integration spec (generic wildcard mention). The integration spec points to a hardware spec anchor, but the hardware spec has no matching section or text.
- VALID_AXI_USER and AXI_USER_LOCK in CaliptraSSHardwareSpecification.md in `docs/`: No matches

Looked at and ruled out:

- `docs/CaliptraSSIntegrationSpecification.md:2291-2298`: It says the registers must be configured, but gives no index mapping and does not say whether MBOX0 and MBOX1 are independent.

1.7 Hardware resets TARGET_STATUS to BUSY on each Target handoff.

`f_3f562f5bcad1` · missing detail · severity medium · section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_target_status

Gap: The target handoff event is not defined, and BUSY is not an enum name (the enum uses CMD_BUSY).

Spec says:

- `src/mci/rtl/mcu_mbox_csr.rdl:106`: `Reset to BUSY by hardware on each target handoff.`
- `docs/CaliptraSSHardwareSpecification.md:1434`: `Hardware resets `TARGET_STATUS` to BUSY on each Target handoff.`

Not answered by the spec: Which exact event performs the hardware clear of TARGET_STATUS: Root setting TARGET_USER_VALID, Root writing TARGET_USER, or another transition? Is the cleared value CMD_BUSY (4'd0), and in which cycle relative to that event does the clear occur?

Verified: I found no spec text that says which event triggers the hardware reset of TARGET_STATUS. Other text describes handoffs and terminal writes, but never ties the reset to a specific event or cycle. The ownership table does call the "Root writes TARGET_USER_VALID=1" row a handoff to the Target, but nothing links the reset to that row. The reset-to-BUSY text appears only in the two cited places.

Searched:

- TARGET_STATUS and handoff mentions in .md and .rdl files in `whole repo`: The only reset statements are the two cited lines. The Integration Spec sequence diagram (around line 2403) and the other spec text cover terminal writes and the clears of TARGET_USER and TARGET_USER_VALID, not the reset of TARGET_STATUS.

Looked at and ruled out:

- `docs/CaliptraSSIntegrationSpecification.md:2397-2404`: The sequence diagram shows when TARGET_USER and TARGET_USER_VALID are programmed and cleared. It does not say when TARGET_STATUS is reset to BUSY.

---

2. Spec contradicts itself

2.1 Integration spec MCU-to-target flow orders EXECUTE and TARGET_USER_VALID opposite to the HW spec

`f_4ca1667d89be` · found during verification · severity medium

Gap: The Integration Specification diagram writes TARGET_USER_VALID=1 and then EXECUTE=1. The Hardware Specification ownership table says EXECUTE=1 with TARGET_USER_VALID=0 leaves Root as owner. It also says Root writing TARGET_USER_VALID=1 while EXECUTE=1 hands ownership to the target. The test follows the Hardware Specification.

Evidence:

- `docs/CaliptraSSIntegrationSpecification.md:2327-2328` — The integration diagram has EXECUTE after the grant, which conflicts with the HW spec.

  ```
  MCU->>MBOX: Write TARGET_USER_VALID=1
  MCU->>MBOX: Write EXECUTE=1
  ```

- `docs/CaliptraSSHardwareSpecification.md:1398`: `| Root writes `TARGET_USER_VALID=1` while `EXECUTE=1` | The agent named by `TARGET_USER` |` — HW spec makes the TARGET_USER_VALID write the handoff, with EXECUTE already 1. This is the test's order.
- `docs/CaliptraSSHardwareSpecification.md:1397`: `| `EXECUTE=1` while `TARGET_USER_VALID=0` | Root |` — Setting EXECUTE before the grant is a defined state: Root keeps ownership. The test comment says the same.

Verified: The Integration Specification diagram writes TARGET_USER_VALID=1 and then EXECUTE=1. The Hardware Specification ownership table says EXECUTE=1 with TARGET_USER_VALID=0 leaves Root as owner. It also says Root writing TARGET_USER_VALID=1 while EXECUTE=1 hands ownership to the target. The test follows the Hardware Specification.

Searched:

- TARGET_USER_VALID and EXECUTE ordering in the HW spec in `docs/CaliptraSSHardwareSpecification.md`: Ownership table and surrounding text make the TARGET_USER_VALID=1 write, while EXECUTE=1, the handoff to the target.
- Test flow around the grant in `smoke_test_mcu_mbox_target_user.c lines 170-230`: EXECUTE is set first, then TARGET_USER and TARGET_USER_VALID are written, then the test waits for TARGET_STATUS.

---

3. Test contradicts its own description

3.1 smoke_test_mcu_mbox_pass_data: comment vs. code

`f_1583e4389781` · comment vs. code · severity high

Gap: The comment says the lock request made while MCU holds the lock will be checked on the MCU side to confirm the interrupt is set. The Caliptra side only issues the lock request (s12). None of the MCU-side checks (k12-k16) reads or checks a SOC request interrupt status. Plan item p6.k1 (SOC request interrupt is set) is therefore never checked.

Labels in this finding: s12 = `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/cptra_smoke_test_mcu_mbox_pass_data.c:367`; k12 = `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/smoke_test_mcu_mbox_pass_data.c:115`; k16 = `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/smoke_test_mcu_mbox_pass_data.c:164`; p6.k1 = `docs/Caliptra_Gen2_SS_TestPlan.xlsx [Subsystem Test Plan] row 121`

Test description says:

- `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/cptra_smoke_test_mcu_mbox_pass_data.c:365`: `// Attempt to acquire lock even though MCU has the lock (will be checked on MCU side that interrupt is set)`
- `docs/Caliptra_Gen2_SS_TestPlan.xlsx [Subsystem Test Plan] row 121`: `SOC request interrupt set` — SOC request interrupt is set.

Test does:

- `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/cptra_smoke_test_mcu_mbox_pass_data.c:367`: `caliptra_ss_mcu_mbox_acquire_lock(mbox_num, 1);` — stimulus; Request the mailbox lock while MCU holds it.
- `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/smoke_test_mcu_mbox_pass_data.c:115`: `if(!mcu_mbox_wait_for_user_lock(mbox_num, caliptra_uc_axi_id, 10000)) {` — check; Caliptra must acquire the mailbox lock with the expected AXI user within the wait limit; otherwise a fatal message is printed, exit status 1 is sent and the test hangs.
- `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/smoke_test_mcu_mbox_pass_data.c:121`: `if(!mcu_mbox_wait_for_user_execute(mbox_num, 1, 10000)) {` — check; Caliptra must set execute within the wait limit.
- `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/smoke_test_mcu_mbox_pass_data.c:64`: `if (mbox_resp_data != clptra_expected_data[ii]) {` — check; Each SRAM dword written by Caliptra must equal the expected data.
- `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/smoke_test_mcu_mbox_pass_data.c:130`: `if(lsu_read_32(SOC_MCI_TOP_MCU_MBOX0_CSR_MBOX_USER + MCU_MBOX_NUM_STRIDE * mbox_num) != caliptra_uc_axi_id) {` — check; The MBOX_USER CSR must equal the Caliptra AXI user ID that locked the mailbox.
- `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/smoke_test_mcu_mbox_pass_data.c:164`: `if(!mcu_mbox_wait_for_user_execute(mbox_num, 0, 10000)) {` — check; Caliptra must clear execute after reading the response data within the wait limit.

Verified: The Caliptra-side comment says the MCU side will check that the SOC-request-while-MCU-lock interrupt is set. The MCU-side pass_data test never reads or checks that interrupt. It does not call the library helper that waits for it, and it has no ISR or interrupt references. Other tests (smoke_test_mcu_mbox_target_user) do use that helper, but this one does not.

Searched:

- interrupt/intr/SOC_REQ in the pass_data MCU test in `smoke_test_mcu_mbox_pass_data/*.c`: no interrupt check in the MCU-side file; only a comment and unused variables in the Caliptra file
- soc_req_while_mcu_lock helper usages in `src/integration`: used only in smoke_test_mcu_hitless and smoke_test_mcu_mbox_target_user

---

3.2 mci_mbox0_ownership_flow: plan vs. test

`f_481566381db1` · plan vs. test · severity medium

Gap: The plan item says the MCU acquires the mailbox lock, passes data to a SoC agent, and reads the new data after the DATA_READY response. In the test, the MCU is only the root, which sets trusted users and releases the mailbox. A random SoC user (the requester) acquires the lock, and another SoC user is the target. The roles in the plan are reversed relative to what the test drives.

Test description says:

- `docs/Caliptra_Gen2_SS_TestPlan.xlsx`: `MCU acquires lock, passes data to SOC agent, which responds DATA_READY; MCU reads new data.`
- `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:178`: `// Requester and Target are random trusted SoC users; Root is the MCU.`

Test does:

- `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:179`: `mcu_mbox_pick_soc_users(soc_users, 2);` — stimulus; Pick two random SoC users as requester and target; root is the MCU AXI user.
- `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:180`: `mci_comm_flow_requester = soc_users[0];` — stimulus; Pick two random SoC users as requester and target; root is the MCU AXI user.
- `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:181`: `mci_comm_flow_root = cptra_ss_strap_mcu_lsu_axi_user_i;` — stimulus; Pick two random SoC users as requester and target; root is the MCU AXI user.
- `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:182`: `mci_comm_flow_target = soc_users[1];` — stimulus; Pick two random SoC users as requester and target; root is the MCU AXI user.
- `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:195`: `mcu_mbox_set_trusted_users(mbox_num, soc_users);` — stimulus; Set trusted users, release the mailbox as root, and have the requester acquire the lock.
- `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:196`: `mcu_mbox_release(mbox_num, mci_comm_flow_root);` — stimulus; Set trusted users, release the mailbox as root, and have the requester acquire the lock.
- `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:197`: `mcu_mbox_acquire_lock(mbox_num, mci_comm_flow_requester);` — stimulus; Set trusted users, release the mailbox as root, and have the requester acquire the lock.

Verified: The plan rows say the MCU gets the lock and passes data to a SoC agent. In the test the MCU is only the root, and a random SoC user is the requester that acquires the lock. The roles are reversed. Nothing in the repository ties the plan rows to this test by name, so the link is inferred. I found no other test that realizes the plan rows.

Searched:

- Plan rows about the MCU getting the lock and passing data to a SoC agent in `docs/Caliptra_Gen2_SS_TestPlan.xlsx`: Rows B137 and B138 match the finding's quote. No test name is given in those rows.
- Other tests realizing the plan rows (DATA_READY, mcu_mbox) in `src, docs`: Only mci_mbox_ownership_flow.svh uses DATA_READY in the testbench. mcu_mbox_soc_agent_write_fw_image.svh is the only other mailbox test I checked; it uses COMPLETE.

Looked at and ruled out:

- `src/integration/testbench/sv_tests/mci/mcu_mbox_soc_agent_write_fw_image.svh:16-20`: It is a SoC agent firmware-image write flow that uses COMPLETE. It does not realize the DATA_READY plan row.

3.3 mci_error_non_fatal_warm_rst_check: check cannot fail

`f_6f160b38b849` · check cannot fail · severity medium

Gap: The message calls this an 'SVA ERROR' and says HW_ERROR_NON_FATAL is expected to remain unchanged on warm reset. The assertion's failure action only prints that message, so a violation does not fail the test. The stability requirement is therefore reported but

never enforced.

Test description says:

- `src/integration/asserts/caliptra_ss_top_sva.sv:245`: `SVA ERROR: MCI HW ERROR NON FATAL is expected to remain unchanged on warm reset`

Test does:

- `src/integration/asserts/caliptra_ss_top_sva.sv:244`: `((~`CPTRA_SS_TOP_PATH.cptra_ss_rst_b_i & `CPTRA_SS_TOP_PATH.cptra_ss_pwrgood_i) |-> ($stable(`MCI_REG_TOP_PATH.mci_reg_hwif_out.HW_ERROR_NON_FATAL)[*5]))` — check; does not fail the test; When cptra_ss_rst_b_i is low and power good is high, HW_ERROR_NON_FATAL is stable for 5 cycles. Disabled when power good is low. A violation only prints a message.

Verified: The assertion's else branch is only `$display("SVA ERROR: ...")`. It does not call `$error` or `$fatal`, and it sets no failure flag. Nothing in the repository that I found scans logs for "SVA ERROR" to fail a test. A violation is therefore printed but does not fail the test. I did not check whether the simulator's own assertion-failure count feeds into pass/fail.

Searched:

- uses of the assertion name in `whole repo`: Only the definition at line 241; no test references it.
- a log or checker that fails the test on 'SVA ERROR' in `whole repo, excluding the SVA file itself`: Only other $display-based SVA files; no consumer of the string.

Looked at and ruled out:

- `src/integration/testbench/caliptra_ss_top_tb_services.sv:1`: It contains $error/$fatal-style matches, but none that parse SVA messages.

3.4 mci_error_non_fatal_cold_rst_check: check cannot fail

`f_71de8c2f726c` · check cannot fail · severity medium

Gap: The assertion is presented as a check: the message reads 'SVA ERROR: MCI HW ERROR NON FATAL is expected to reset on cold reset'. A violation only prints that message and does not fail the test. A HW_ERROR_NON_FATAL value that survives a cold reset would therefore go unnoticed unless the log is scanned.

Test description says:

- `src/integration/asserts/caliptra_ss_top_sva.sv:239`: `SVA ERROR: MCI HW ERROR NON FATAL is expected to reset on cold reset`

Test does:

- `src/integration/asserts/caliptra_ss_top_sva.sv:238`: `(~`CPTRA_SS_TOP_PATH.cptra_ss_pwrgood_i |-> (`MCI_REG_TOP_PATH.mci_reg_hwif_out.HW_ERROR_NON_FATAL=='h0))` — check; does not fail the test; When cptra_ss_pwrgood_i is low, HW_ERROR_NON_FATAL equals 0. A violation only prints a message.

Verified: The assertion's else branch only calls $display. No $error or $fatal is used, and nothing outside the .sv files looks for "SVA ERROR" in the logs. A violation therefore prints a message but does not fail the test.

Searched:

- "SVA ERROR" in non-.sv files (scripts, makefiles, logs) in `whole repo`: No matches, so no log scanning for this message.
- Other uses of "SVA ERROR" in .sv files in `whole repo`: All are $display-only, matching the same pattern.

3.5 all_error_non_fatal_check: check cannot fail

`f_7b20eefcf778` · check cannot fail · severity medium

Gap: The assertions k1 and k2 are described as checks, and the message i1 says "SVA ERROR: all_error_non_fatal is asserted unexpectedly". A violation only prints this message and does not fail the test, so a real violation would go unnoticed in the result.

Labels in this finding: k1 = `src/integration/asserts/caliptra_ss_top_sva.sv:250`; k2 = `src/integration/asserts/caliptra_ss_top_sva.sv:251`; i1 = `src/integration/asserts/caliptra_ss_top_sva.sv:252`

Test description says:

- `src/integration/asserts/caliptra_ss_top_sva.sv:252`: `SVA ERROR: all_error_non_fatal is asserted unexpectedly`

Test does:

- `src/integration/asserts/caliptra_ss_top_sva.sv:250` — check; does not fail the test; With mbox0/mbox1 ECC masks set, a mailbox double ECC error and mci_intr, all_error_non_fatal stays deasserted for 5 cycles.

  ```
  ((`MCI_REG_TOP_PATH.mci_reg_hwif_out.internal_hw_error_non_fatal_mask.mask_mbox0_ecc_unc &
  `MCI_REG_TOP_PATH.mci_reg_hwif_out.internal_hw_error_non_fatal_mask.mask_mbox1_ecc_unc) &
  (`MCI_REG_TOP_PATH.mbox0_sram_double_ecc_error | `MCI_REG_TOP_PATH.mbox1_sram_double_ecc_error) &
  `MCI_REG_TOP_PATH.mci_intr |=> ~`MCI_REG_TOP_PATH.all_error_non_fatal[*5])
  ```

- `src/integration/asserts/caliptra_ss_top_sva.sv:251`: `((&`MCI_REG_TOP_PATH.mci_reg_hwif_out.internal_fw_error_non_fatal_mask.mask & |`MCI_REG_TOP_PATH.mci_reg_hwif_out.FW_ERROR_NON_FATAL.error_code) |=> ~`MCI_REG_TOP_PATH.all_error_non_fatal[*5])` — check; does not fail the test; With all FW non-fatal mask bits set and a nonzero FW_ERROR_NON_FATAL error_code, all_error_non_fatal stays deasserted for 5 cycles. A violation only prints a message.

Verified: all_error_non_fatal_check is an assert property whose only action is `else $display(...)`. It has no $error, $fatal or error counter, so a violation prints a message and does not fail the test. I found no log scanner or testbench hook in the repository that turns "SVA ERROR" into a failure. The simulator's own assertion-failure count may still be flagged outside the repository, which I couldn't check.

Searched:

- Other consumers of the 'SVA ERROR' string or assertion-failure checks in `whole repo, excluding the SVA file`: Only other $display-style SVA files (caliptra_mcu_top_sva.sv); no scanner.
- Scripts, makefiles or yml that grep logs for SVA or ERROR, or print TESTCASE PASSED/FAILED in `*.mk, *.py, *.sh, *.yml, *.yaml, *.pl, *Makefile`: No matches.

3.6 all_error_fatal for mcu_sram_ecc_unc: check cannot fail

`f_938b193cfb1a` · check cannot fail · severity medium

Gap: The message says "SVA ERROR: all_error_fatal for mcu_sram_ecc_unc is asserted unexpectedly", and the summary says all_error_fatal must stay low. The assertion's failure action only prints that message, so a violation does not fail the test.

Test description says:

- `src/integration/asserts/caliptra_ss_top_sva.sv:227`: `SVA ERROR: all_error_fatal for mcu_sram_ecc_unc is asserted unexpectedly`

Test does:

- `src/integration/asserts/caliptra_ss_top_sva.sv:226`: `((`MCI_REG_TOP_PATH.mci_reg_hwif_out.internal_hw_error_fatal_mask.mask_mcu_sram_ecc_unc) & `MCI_REG_TOP_PATH.mcu_sram_double_ecc_error & `MCI_REG_TOP_PATH.cif_resp_if.error |=> ~`MCI_REG_TOP_PATH.all_error_fatal[*5])` — check; does not fail the test; After the mask, MCU SRAM double ECC error and cif_resp_if error all hold, all_error_fatal stays deasserted for 5 cycles. Disabled while cptra_ss_rst_b_i is low. A violation only prints a message.

Verified: The assertion's else branch is only a $display, so a violation prints a message and nothing more. No other file in the repository references the "SVA ERROR" string, so nothing appears to scan logs for it and fail the test.

Searched:

- "SVA ERROR" outside *_sva.sv files, as a log-scanning check in `whole repo, sv/svh/v/yml/py/sh/tcl/mk files`: no matches

---

3.7 mci_error_non_fatal_check: check cannot fail

`f_b7cfe0f12370` · check cannot fail · severity medium

Gap: The summary says a double ECC error must set the matching HW_ERROR_NON_FATAL ECC uncorrectable bit on the next cycle. The message at i1 is labelled "SVA ERROR". The assertion k1 only prints that message on violation, so a failing property does not fail the test and the check cannot fail it.

Labels in this finding: i1 = `src/integration/asserts/caliptra_ss_top_sva.sv:234`; k1 = `src/integration/asserts/caliptra_ss_top_sva.sv:233`

Test description says:

- `src/integration/asserts/caliptra_ss_top_sva.sv:234`: `SVA ERROR: MCI HW ERROR NON FATAL reg is not set correctly`

Test does:

- `src/integration/asserts/caliptra_ss_top_sva.sv:233` — check; does not fail the test; mbox0 double ECC error implies HW_ERROR_NON_FATAL.mbox0_ecc_unc next cycle, and mbox1 double ECC error implies mbox1_ecc_unc next cycle. A violation only prints a message.

```
(`MCI_REG_TOP_PATH.mbox0_sram_double_ecc_error |=>
`MCI_REG_TOP_PATH.mci_reg_hwif_out.HW_ERROR_NON_FATAL.mbox0_ecc_unc) and
(`MCI_REG_TOP_PATH.mbox1_sram_double_ecc_error |=>
`MCI_REG_TOP_PATH.mci_reg_hwif_out.HW_ERROR_NON_FATAL.mbox1_ecc_unc)
```

Verified: The assertion mci_error_non_fatal_check has only an else $display action. In SystemVerilog, an assert with an else clause runs only that clause on failure, so there is no default error and no $fatal or $error. I found nothing in the repository that scans logs for "SVA ERROR" or otherwise turns this message into a test failure, so the check cannot fail a test. I did not read the regression or pass/fail scripts in full.

Searched:

- Log scanners, $assertoff or fatal-on-assert handling that would turn 'SVA ERROR' into a test failure in `whole repository, excluding *sva.sv files`: Nothing that treats 'SVA ERROR' as a failure. The only hits were unrelated $assertoff calls in USB and fuse_ctrl, and -assert compile flags.
- The assertion body and its action block in `src/integration/asserts/caliptra_ss_top_sva.sv lines 215-254`: Every assertion in this file uses an else $display action. None calls $error or $fatal.

Looked at and ruled out:

- `tools/scripts/Makefile:392-401`: These lines only enable assertion compilation. They do not make a failed assertion fail the test.

3.8 smoke_test_mcu_mbox: plan vs. test

`f_c45dacf55f05` · plan vs. test · severity medium

Gap: The plan says the SOC agent responds with DATA_READY after the MCU acquires the lock and passes data, and the MCU then reads new data. The test only polls for the COMPLETE status (0x2) and then clears execute. It neither waits for DATA_READY nor reads any new data afterward.

Test description says:

- `docs/Caliptra_Gen2_SS_TestPlan.xlsx`: `MCU acquires lock, passes data to SOC agent, which responds DATA_READY; MCU reads new data.`

Test does:

- `src/integration/test_suites/smoke_test_mcu_mbox/cptra_ss_mcu_mbox_test.c:233`: `reg = caliptra_ss_mcu_mbox_wait_status_complete(mbox_num, 200);` — stimulus; Poll the command status register up to 200 times for the completion status.
- `src/integration/test_suites/smoke_test_mcu_mbox/cptra_ss_mcu_mbox_test.c:80`: `if (mbox_data == 0x2) {` — check; The command status must reach 0x2 (complete) within 200 polls; otherwise the test reports a timeout and exits with an error status.
- `src/integration/test_suites/smoke_test_mcu_mbox/cptra_ss_mcu_mbox_test.c:234`: `if (reg == 2) {` — check; The command status must reach 0x2 (complete) within 200 polls; otherwise the test reports a timeout and exits with an error status.

Verified: The plan says the response status is DATA_READY (1). Both sides of the test use CMD_COMPLETE (2): the SOC side polls for status 2, and the MCU firmware writes 0x2. No DATA_READY wait exists. The finding's claim that no data is read afterward is partly wrong, because the test does read MCU data through caliptra_ss_mcu_mbox_get_data. The status mismatch still stands.

Searched:

- DATA_READY usage in the smoke test in `src/integration/test_suites/smoke_test_mcu_mbox`: No DATA_READY wait. Only status 0x2 is polled and written.
- Status enum encoding in `src/mci/rtl/mcu_mbox_csr.rdl`: DATA_READY=1, CMD_COMPLETE=2

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox/cptra_ss_mcu_mbox_test.c:242-247`: get_data does read and check data after completion, so only the DATA_READY part of the finding holds.

3.9 smoke_test_mcu_mbox: check cannot fail

`f_f54d6febfdbe` · check cannot fail · severity medium

Gap: The plan item says the user CSR must show the user holding the lock, and the comment says 'Checking MBOX USER INFO'. The code reads the MBOX USER register and only prints it, with no comparison against an expected user, so nothing is checked and the test cannot fail on it.

Test description says:

- `src/integration/test_suites/smoke_test_mcu_mbox/cptra_ss_mcu_mbox_test.c:112`: `// Checking MBOX USER INFO`
- `docs/Caliptra_Gen2_SS_TestPlan.xlsx [Subsystem Test Plan] row 117`: `User CSR properly displays the user with lock` — User CSR displays the user holding the lock.

Test does:

- `src/integration/test_suites/smoke_test_mcu_mbox/cptra_ss_mcu_mbox_test.c:114`: `soc_ifc_axi_dma_read_ahb_payload(SOC_MCI_TOP_MCU_MBOX0_CSR_MBOX_USER + MCU_MBOX_NUM_STRIDE * mbox_num, 0, read_payload, 4, 0);` — stimulus; Read the MBOX USER register (the value is only printed).

Verified: The "Checking MBOX USER INFO" step reads the MBOX USER register and only prints it. Nothing compares the value to an expected user, so the test cannot fail on it. The MCU-side test only polls until USER is nonzero, which is a weak check, and then prints it.

Further evidence found by the verifier:

- `src/integration/test_suites/smoke_test_mcu_mbox/smoke_test_mcu_mbox.c:115-117`: `while(lsu_read_32(SOC_MCI_TOP_MCU_MBOX0_CSR_MBOX_USER + MCU_MBOX_NUM_STRIDE * mbox_num) == 0);` — The MCU side only waits for USER to be nonzero and does not compare it to an expected user.

Searched:

- MBOX_USER uses in the smoke_test_mcu_mbox directory in `src/integration/test_suites/smoke_test_mcu_mbox`: 3 uses: one read-and-print, one poll until nonzero, and one read into a variable that is printed. No comparison against an expected user.

3.10 smoke_test_mcu_mbox_pass_data: plan vs. test

`f_1f10a1ae6b50` · plan vs. test · severity low

Gap: Plan item p9 says MCU acquires the lock, passes data to the SOC agent, and the SOC agent responds DATA_READY. In the test the roles are reversed. Caliptra acquires the lock, writes the data and sets execute, and MCU only waits for Caliptra's lock (s25) and returns the response with DATA_READY. MCU never acquires the lock.

Labels in this finding: p9 = `docs/Caliptra_Gen2_SS_TestPlan.xlsx`; s25 = `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/smoke_test_mcu_mbox_pass_data.c:115`

Test description says:

- `docs/Caliptra_Gen2_SS_TestPlan.xlsx`: `MCU acquires lock, passes data to SOC agent, which responds DATA_READY; MCU reads new data.`

Test does:

- `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/cptra_smoke_test_mcu_mbox_pass_data.c:391`: `cptra_mcu_mbox_acquire_lock(mbox_num, 1000, true);` — stimulus; Acquire the mailbox lock.
- `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/smoke_test_mcu_mbox_pass_data.c:115`: `mcu_mbox_wait_for_user_lock(mbox_num, caliptra_uc_axi_id, 10000)` — stimulus; Wait for Caliptra to acquire the mailbox lock.
- `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/smoke_test_mcu_mbox_pass_data.c:157`: `mcu_mbox_write_sram_dword(mbox_num, ii, mbox_data[ii]);` — stimulus; Write the response data to mailbox SRAM, write the data length, and set status to DATA_READY.
- `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/smoke_test_mcu_mbox_pass_data.c:160`: `mcu_mbox_write_dlen(mbox_num, sizeof(mbox_data));` — stimulus; Write the response data to mailbox SRAM, write the data length, and set status to DATA_READY.

- `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/smoke_test_mcu_mbox_pass_data.c:162`: `mcu_mbox_update_status(mbox_num, MCU_MBOX_DATA_READY);` — stimulus; Write the response data to mailbox SRAM, write the data length, and set status to DATA_READY.

Verified: Caliptra acquires the lock and MCU only waits for it, then returns the response with DATA_READY. The plan text has MCU acquiring the lock and the SOC agent responding, so the roles are reversed. I did not open the plan spreadsheet; the plan wording comes from the finding.

Searched:

- Read both test sources around the lock, execute and response steps in `smoke_test_mcu_mbox_pass_data directory`: Caliptra is the lock owner and MCU is the responder.
- Search for pass_data in md files in `repo`: The glob failed in zsh, so nothing was searched. I did not look for another test that realizes the plan item.

3.11 smoke_test_mcu_mbox_instantiation: comment vs. code

`f_61330c35c86c` · comment vs. code · severity low

Gap: The intent text contradicts itself about the Caliptra uC's AXI user. i2 and i8 say the uC is configured as a valid AXI user, but i4, i5 and the error messages for k1-k11 say 'invalid user'. The code sets the mailbox valid user to a random axi_user_id and configures the DMA user as caliptra_uc_axi_id, so the description of the scenario is inconsistent. The errors are expected because the mailbox is not instantiated, not because of an invalid user, so the messages describe a different condition than the one under test. This also makes the link to plan item p1 (invalid-user rejection) unclear.

Labels in this finding: i2 = `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/smoke_test_mcu_mbox_instantiation.c:36`; i8 = `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:60`; i4 = `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:40`; i5 = `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:41`; k1 = `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:64`, `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:71`, `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:65`; k11 = `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:208`, `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:217`, `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:63`; p1 = `docs/Caliptra_Gen2_SS_TestPlan.xlsx`

Test description says:

- `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/smoke_test_mcu_mbox_instantiation.c:36`: `// 1. MCU will configure Caliptra uC to be a valid AXI`
- `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:41`: `MCU will configure Caliptra uC to be an invalid AXI`
- `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:60`: `Caliptra: CSR Writes and Read as Valid AXI but no Mbox instantiation`
- `docs/Caliptra_Gen2_SS_TestPlan.xlsx [Subsystem Test Plan] row 104`: `Invalid user AXI accesses are rejected and report AXI error` — Invalid user AXI accesses are rejected and report AXI error

Test does:

- `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/smoke_test_mcu_mbox_instantiation.c:60`: `mcu_cptra_init_d(.cfg_cptra_dma_axi_user=true, .cptra_dma_axi_user=caliptra_uc_axi_id, .cfg_mcu_mbox0_valid_user=mbox0_sel, .mcu_mbox0_valid_user=axi_user_id, .cfg_mcu_mbox1_valid_user=!mbox0_sel, .mcu_mbox1_valid_user=axi_user_id);` — stimulus; Initialize MCU and Caliptra, configuring the Caliptra DMA AXI user and the MCU mailbox valid users.
- `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:64`: `Caliptra: Expected AXI Error writing to CSR EXECUTE as invalid user` — check; AXI write and read of CSR EXECUTE must return an AXI error (status false); otherwise fatal print, error control code and hang.
- `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:71`: `Caliptra: Expected AXI Error reading CSR EXECUTE as invalid user` — check; AXI write and read of CSR EXECUTE must return an AXI error (status false); otherwise fatal print, error control code and hang.
- `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:65`: `SEND_STDOUT_CTRL(0x1);` — check; AXI write and read of CSR EXECUTE must return an AXI error (status false); otherwise fatal print,

error control code and hang.

- `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:208`: `Caliptra: Expected AXI Error writing to Mbox%x SRAM as invalid user` — check; Each of the random AXI writes and reads to the mailbox SRAM must return an AXI error.
- `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:217`: `Caliptra: Expected AXI Error reading Mbox%x SRAM as invalid user` — check; Each of the random AXI writes and reads to the mailbox SRAM must return an AXI error.
- `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:63`: `if(!status) {` — check; Each of the random AXI writes and reads to the mailbox SRAM must return an AXI error.

Verified: The prose is inconsistent. The MCU-side test says the Caliptra uC is a valid AXI user. The Caliptra-side test header and its error messages say invalid user, and one of its log lines says "Valid AXI but no Mbox instantiation". The code sets the DMA user to caliptra_uc_axi_id and the mailbox valid users to axi_user_id. The AXI errors it expects come from the mailbox not being instantiated, not from user rejection.

Searched:

- valid/invalid wording in both test sources in `smoke_test_mcu_mbox_instantiation directory`: The two sources contradict each other. The messages say invalid user even though the test targets a mailbox that is not instantiated.

3.12 smoke_test_mcu_mbox0_write_user_lock: check cannot fail

`f_79e630809b5c` · check cannot fail · severity low

Gap: The Caliptra-side comment says "Checking MBOX USER INFO", but the code only reads the USER register over AXI DMA into read_payload. No extracted check compares the value with the expected user, so this step cannot fail the test.

Test description says:

- `src/integration/test_suites/smoke_test_mcu_mbox_write_user_lock/cptra_smoke_test_mcu_mbox_write_user_lock.c:66`: `// Checking MBOX USER INFO`

Test does:

- `src/integration/test_suites/smoke_test_mcu_mbox_write_user_lock/cptra_smoke_test_mcu_mbox_write_user_lock.c:67`: `soc_ifc_axi_dma_read_ahb_payload(SOC_MCI_TOP_MCU_MBOX0_CSR_MBOX_USER + MCU_MBOX_NUM_STRIDE * mbox_num, 0, read_payload, 4, 0);` — stimulus; Read the mailbox USER register.

Verified: The Caliptra-side code reads the MBOX USER register and only prints it with VPRINTF. Nothing compares it with an expected value, so this step cannot fail the test. The MCU-side file only checks that a write to USER leaves it unchanged. That is a separate check and does not validate the Caliptra-side value.

Searched:

- USER references in the test directory in `smoke_test_mcu_mbox_write_user_lock/`: Caliptra side only reads and prints. MCU side only does a read-only write check.

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_write_user_lock/smoke_test_mcu_mbox_write_user_lock.c:102-107`: This checks that the USER register is read-only. It does not compare the USER value Caliptra read against an expected user.

---

4. Spec and test disagree

No verified findings.

5. Spec not tested

5.1 Partly tested: 4 requirements in CaliptraSSHardwareSpecification.md > MCU Mailbox Limited Trusted AXI users (and 1 more section)

`f_469ba16e55c7` · severity high

1. **Requirement** `cr_7f11ce310155`: There are 5 trusted AXI users, determined at build time via parameters or via MCI lockable registers. Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox

> MCU Mailbox Limited Trusted AXI users

- `docs/CaliptraSSHardwareSpecification.md:1297`: `There are 5 trusted AXI Users determined at build time via parameters or via MCI lockable registers.`

Tested by `smoke_test_mcu_mbox_axi_param`**:** Exercises the Caliptra uC AXI ID set either from a build-time parameter or from the CSR user IDs, and checks AXI errors depending on which is used.
Not tested: Does not verify that there are exactly 5 trusted users, or that each parameter or CSR user slot is accepted as trusted.

Verified: No test checks every one of the 5 slots or rejects a sixth ID.

2. **Requirement** `cr_bb5623ddc32e`: Only the trusted AXI users, the default user 0xFFFF_FFFF, and MCU can access or obtain a lock on the mailbox.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Limited Trusted AXI users

   - `docs/CaliptraSSHardwareSpecification.md:1297`: `These users, a default user 0xFFFF_FFFF, and MCU are the only AXI users that can access or obtain a lock on the mailbox.`

   Tested by `smoke_test_mcu_mbox_invalid_axi`**:** A user outside the allowed set gets AXI errors on all CSR and SRAM accesses.
   Not tested: Does not check lock acquisition by such a user, nor that the default user 0xFFFF_FFFF or MCU are allowed.

   Verified: No test checks lock denial for an untrusted user, or access by the default user.

3. **Requirement** `cr_045d5c38ebd5`: If a second Target user is required, MCU waits for the first Target to publish a terminal TARGET_STATUS.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Target User

   - `docs/CaliptraSSHardwareSpecification.md:1349-1350`: `If a second Target user is required, MCU shall wait for the first Target to publish a terminal `TARGET_STATUS` during normal operation.`

   Tested by `smoke_test_mcu_mbox_target_user`**:** MCU waits for the first Target to publish a COMPLETE terminal TARGET_STATUS before the mailbox flow continues.
   Not tested: No second Target is required or programmed after the first one finishes, so the wait-then-re-grant sequence is not exercised.

   Verified: The target_user test has a single Target only.

   - `src/integration/test_suites/smoke_test_mcu_mbox_target_user/smoke_test_mcu_mbox_target_user.c:211`: `// Wait for the Target to publish a terminal TARGET_STATUS` — The test has a single Target and no second Target is granted.

4. **Requirement** `cr_261014384edd`: Hardware clears and unlocks both registers when the mailbox is released.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Target User

   - `docs/CaliptraSSHardwareSpecification.md:1328-1329`: `Hardware clears and unlocks both registers on a terminal `TARGET_STATUS` write or when the mailbox is released.`

   Tested by `smoke_test_mbox_csr_zeros_after_release`**:** Clearing EXECUTE releases the mailbox directly, and the test then checks that TARGET_USER and TARGET_USER_VALID read 0.
   Not tested: It does not check that the registers are unlocked after release, for example by writing them again.

   Verified: The release test only reads the registers back as 0 and never rewrites them.

   - `src/integration/test_suites/smoke_test_mbox_csr_zeros_after_release/smoke_test_mbox_csr_zeros_after_release.c:103`: `lsu_write_32(SOC_MCI_TOP_MCU_MBOX0_CSR_MBOX_EXECUTE + MCU_MBOX_NUM_STRIDE * mbox_num, 0);` — The test releases the mailbox and only reads the registers back as 0. It never rewrites them to show they are unlocked.

Item 2 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- Item 1: tests for 5 trusted slots and a sixth ID in `smoke_test_mcu_mbox_axi_param and smoke_test_mcu_mbox_invalid_axi`: axi_param reads MBOX_LOCK once. invalid_axi uses 5 random IDs and then an ID outside them. No test exercises each slot or checks for a sixth.
- Item 2: trusted user lock then CSR writes in `smoke_test_mcu_mbox_pass_data`: Caliptra acquires the lock and writes CMD, DLEN and EXECUTE. The MCU reads DLEN back. CMD_STATUS and USER are not covered.

- Item 3: untrusted user acquiring the lock, and the default 0xFFFFFFFF or MCU user in `smoke_test_mcu_mbox_invalid_axi`: MBOX_LOCK is in the list of accesses an invalid user attempts (line 88). No check that the lock stays free, and no positive check for the default user.
- Item 4: second Target user in `smoke_test_mcu_mbox_target_user`: Only a single Target flow.
- Item 5: rewrite after release in `smoke_test_mbox_csr_zeros_after_release`: Only reads the registers back as 0 after release.

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/cptra_smoke_test_mcu_mbox_pass_data.c:103-120`: This is a positive trusted-user flow after acquiring the lock. It does not read back all the CSRs the gap names.

5.2 Partly tested: 1 requirement in CaliptraSSHardwareSpecification.md > MCU Mailbox MCU Access

`f_6ae2ab31ed12` · severity high

1. **Requirement** `cr_2d3206a4c6cc`: TARGET_USER and TARGET_USER_VALID are writable only while TARGET_USER_VALID=0.

   - `docs/CaliptraSSHardwareSpecification.md:1516-1517`

     ```
     Both registers are writable only while
     `TARGET_USER_VALID=0`.
     ```

   Tested by `smoke_test_mcu_mbox_strb_wr_csr`**:** TARGET_USER is written byte by byte while VALID=0 and takes effect. After VALID is set, a write of 0 to TARGET_USER is ignored.
   Not tested: A write to TARGET_USER_VALID while it is 1 is not attempted.

   Verified: No test writes TARGET_USER_VALID while it is 1.

   - `src/integration/test_suites/smoke_test_mcu_mbox_write_user_lock/smoke_test_mcu_mbox_write_user_lock.c:146`: `Wasn't able to write Mbox%x TARGET_USER_VALID as Root` — Only checks a successful write of TARGET_USER_VALID, not a blocked one.

Items 2, 3, 4, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- TARGET_USER_VALID in tests in `src/integration/test_suites`: Only writes while 0, checks for changed-by-target, and clear checks; no blocked write while 1.

5.3 Partly tested: 5 requirements in CaliptraSSHardwareSpecification.md > MCU Mailbox Status Registers (and 2 more sections)

`f_60dc44ecdea3` · severity high

1. **Requirement** `cr_62191b6deb64`: Only the current Target writes TARGET_STATUS (the mailbox target status register).
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Status Registers

   - `docs/CaliptraSSHardwareSpecification.md:1429-1430`

     ```
     Only the current Target
     writes `TARGET_STATUS`
     ```

   - `src/mci/rtl/mcu_mbox_csr.rdl:106`: `Written only by the current target user and read by Root.`

   Tested by `smoke_test_mcu_mbox_target_user`**:** The target's writes are ignored before ownership is granted, and after the grant the target publishes a COMPLETE target status that MCU observes.
   Not tested: Writes by other non-target agents to TARGET_STATUS are not checked.

   Tested by `smoke_test_mcu_mbox_strb_wr_csr`**:** An 8-bit write of TARGET_STATUS from Root is attempted and the test checks the value is unchanged.
   Not tested: Only a Root write is checked. The Target's write succeeding and other agents being blocked are not checked.

   Tested by `smoke_test_mcu_mbox0_write_user_lock`**:** Root attempts to write TARGET_STATUS and the test checks it is unchanged.
   Not tested: Only a Root write is blocked. The current Target's write succeeding and other non-target users being blocked are not checked.

   Verified: No non-target, non-Root write-block check found; the existing tests cover only Root and the target.

2. **Requirement** `cr_ccc86edf401d`: When DLEN = 0 the entire MBOX SRAM is cleared.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox SRAM Clearing

   - `docs/CaliptraSSHardwareSpecification.md:1473`: `**NOTE**: When DLEN = 0 the entire MBOX SRAM is cleared.`

   Tested by `mcu_mbox_rand_rdwr`**:** The final step writes DLEN equal to the full SRAM size and checks that the max address reads 0. This is not DLEN = 0.
   Not tested: Never sets DLEN to 0, so the whole-SRAM clear for DLEN = 0 is not verified.

   Verified: No DLEN=0 whole-SRAM clear test found.

   - `src/integration/test_suites/mcu_mbox_lock_return_one_during_zeroize/mcu_mbox_lock_return_one_during_zeroize.c:39`: `const uint32_t mbox_dlen = 10000;` — This test uses a large non-zero DLEN and checks only that the lock reads 1 during zeroize. It does not cover DLEN=0 or the clearing phases.

3. **Requirement** `cr_fadd412c3094`: Clearing flow order: requester releases lock, SRAM clearing starts, SRAM clearing ends, then mailbox is unlocked.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox SRAM Clearing

   - `docs/CaliptraSSHardwareSpecification.md:1462`: `The flow for clearing the SRAM is:`
   - `docs/CaliptraSSHardwareSpecification.md:1464-1467`: `1. Requester releases lock 2. MCU SRAM starts clearing 3. MCU SRAM clearing ends 4. Mailbox is unlocked`

   Tested by `smoke_test_mcu_mbox_zeroize`**:** Releases the lock and then re-acquires it, and checks that the SRAM is zero at that point. This indirectly shows that the unlock follows the clearing.
   Not tested: Does not observe the intermediate phases of the sequence. Only the end state after unlock is checked.

   Tested by `smoke_test_mcu_mbox_usr_lock_out_zero`**:** Shows that the lock cannot be acquired during zeroization after the release. This is consistent with the order: release, then clearing, then unlock.
   Not tested: Does not check that clearing starts and ends in order or that the unlock happens only after the clearing ends.

   Verified: Only the end state and 'lock reads 1 during zeroize' are checked. No assertion or test observes clearing start and end.

4. **Requirement** `cr_20362a1989f1`: On a single bit ECC error while reading the mailbox, the data is corrected.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Errors

   - `docs/CaliptraSSHardwareSpecification.md:1504`: `Single Bit ECC Error`
   - `docs/CaliptraSSHardwareSpecification.md:1504`: `Data corrected`
   - `docs/CaliptraSSHardwareSpecification.md:1504`: `Single bit ECC error while reading Mailbox.`

   Tested by `smoke_test_mcu_mbox_ecc`**:** Injects SB ECC errors and reads back the written addresses, but only the interrupt and status are checked.
   Not tested: No check that the data read back after the single-bit error equals the written (corrected) data.

   Verified: The data read after a single-bit error injection is not compared.

   - `src/integration/test_suites/smoke_test_mcu_mbox_ecc/smoke_test_mcu_mbox_ecc.c:102`: `rand_data = mcu_mbox_read_sram_dword(mbox_num, rand_dword_addr);` — Read-back data is assigned but never compared to the written or expected data. Only the interrupt and status are checked.

5. **Requirement** `cr_feb29b2f4853`: On a double bit ECC error while reading the mailbox, invalid data is returned.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Errors

   - `docs/CaliptraSSHardwareSpecification.md:1505`: `Double Bit ECC Error`
   - `docs/CaliptraSSHardwareSpecification.md:1505`: `Invalid data returned`
   - `docs/CaliptraSSHardwareSpecification.md:1505`: `Double bit ECC error while reading Mailbox.`

   Tested by `smoke_test_mcu_mbox_ecc`**:** Reads back addresses after DB ECC injection and checks the interrupt and status, but not the returned data.
   Not tested: No check of the data returned on the read that hits the double-bit error.

   Verified: The data returned on a double-bit error read is not checked.

Searched:

- TARGET_STATUS writes by non-target agents in `src/integration/test_suites (12 files reference it)`: The cited tests only cover Root and the target. The invalid_axi test only lists the register among CSRs it exercises with invalid AXI accesses. No non-target SoC user or Requester write-block check was found.
- DLEN=0 followed by an SRAM clear check in `src/integration/test_suites`: No test writes DLEN=0 and then reads back the whole SRAM.
- Clearing-phase ordering checks (start, end, unlock) via assertions in `src/integration/asserts/caliptra_ss_top_sva.sv and RTL files that mention zeroize/clearing`: No mcu_mbox clearing assertions in the SVA file. Only functional lock-reads-1 tests exist.
- Single-bit and double-bit ECC read-data checks in `src/integration/test_suites/smoke_test_mcu_mbox_ecc`: No data comparison. Only interrupt, status and HW_ERROR_NON_FATAL are checked.
- Corrected/invalid data checks for ECC in other tests in `src/integration/test_suites`: Nothing found beyond the ECC smoke test.

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mbox_csr_zeros_after_release/smoke_test_mbox_csr_zeros_after_release.c:107-111`: It checks the DLEN register reads 0 after release. It does not write DLEN=0 or check that the SRAM is cleared.

5.4 Partly tested: 4 requirements in CaliptraSSHardwareSpecification.md > MCU Mailbox Target User (and 1 more section)

`f_e38d580bc493` · severity high

1. **Requirement** `cr_29fbbfde9d79`: Hardware clears and unlocks both registers on a terminal TARGET_STATUS write.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Target User

   - `docs/CaliptraSSHardwareSpecification.md:1328-1329`: `Hardware clears and unlocks both registers on a terminal `TARGET_STATUS` write`

   Tested by `smoke_test_mcu_mbox_target_user`**:** After the Target publishes a COMPLETE terminal status, the test checks that TARGET_USER reads 0 and TARGET_USER_VALID reads 0.
   Not tested: It does not check that the registers are unlocked after the terminal write, for example by writing them again.

   Verified: I found no test that rewrites the registers after a terminal TARGET_STATUS. I did not read the other test directories.

   - `src/integration/test_suites/smoke_test_mcu_mbox_write_user_lock/smoke_test_mcu_mbox_write_user_lock.c:144-149` — Root sets valid; no later attempt to clear TARGET_USER_VALID or rewrite TARGET_USER afterwards.

     ```
     mcu_mbox_write_target_user_valid(mbox_num, 1);
         if (mcu_mbox_read_target_user_valid(mbox_num) != 1) {
     ```

2. **Requirement** `cr_4c05e98bd6a2`: Writing TARGET_USER_VALID=1 locks both target configuration registers against all software writes.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Target User

   - `docs/CaliptraSSHardwareSpecification.md:1326-1327`: `Writing `TARGET_USER_VALID=1` locks both registers against all software writes.`

   Tested by `smoke_test_mcu_mbox_strb_wr_csr`**:** After TARGET_USER_VALID is set, a write of 0 to TARGET_USER is attempted and the test checks that it is ignored.
   Not tested: Writes to TARGET_USER_VALID after the lock are not attempted, and only a single TARGET_USER write is tried.

   Verified: I found no test that makes repeated writes to both registers after the lock. I did not read the other test directories.

3. **Requirement** `cr_8e68a65f2d45`: After the grant, software cannot retarget the mailbox or clear TARGET_USER_VALID.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Target User

   - `docs/CaliptraSSHardwareSpecification.md:1327-1328`: `Software cannot retarget the mailbox or clear valid after the grant.`

   Tested by `smoke_test_mcu_mbox_strb_wr_csr`**:** After valid is set, a write to TARGET_USER is attempted and the test checks that the value is unchanged, so retargeting is blocked.
   Not tested: No attempt to clear TARGET_USER_VALID after the grant is made or checked.

   Verified: I found no test where Root writes 0 to TARGET_USER_VALID after the grant. I did not read the other test directories.

4. **Requirement** `cr_afbf3aa32a43`: Root (typically MCU) is the only agent that may write/program TARGET_USER and TARGET_USER_VALID.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Target User

   - `docs/CaliptraSSHardwareSpecification.md:1318-1319`: `Root, typically MCU, is the only agent that programs` `TARGET_USER` and `TARGET_USER_VALID`.
   - `docs/CaliptraSSHardwareSpecification.md:1515-1516`

     > Root is the only agent that may write `TARGET_USER` and
     > `TARGET_USER_VALID`.

   Tested by `smoke_test_mcu_mbox_target_user`**:** The Target attempts writes to CSRs before it is made valid, and TARGET_USER is still 0 before the grant. This shows the Target cannot program TARGET_USER.
   Not tested: The test does not directly check that a non-Root agent cannot write TARGET_USER_VALID. Other non-Root agents are not tried.

   Tested by `smoke_test_mcu_mbox_valid_user`**:** A non-Root user attempts to write TARGET_USER and TARGET_USER_VALID. The test checks that TARGET_USER does not read back 0xdeadbeef and that TARGET_USER_VALID stays 0.
   Not tested: Only one kind of non-Root agent (the requester) is tried. The Target user, SoC agents and other trusted users are not tried, and the writes are not tried while valid is 0 versus 1.

   Tested by `smoke_test_mcu_mbox_instantiation`**:** An untrusted (not-valid) user writes and reads TARGET_USER and TARGET_USER_VALID and each access must return an AXI error.
   Not tested: Only an untrusted user is tried, and only on an uninstantiated mailbox. Trusted non-Root agents such as the requester or Target are not tried.

   Verified: I did not find tests that try writes from the Target or from other trusted non-Root agents with valid at both 0 and 1. I did not read the other test directories.

Item 5 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- TARGET_USER_VALID usage in C tests in `src/integration/test_suites`: 11 files; I read only write_user_lock in detail. It does not test the lock, relock or clear-after-grant behaviour.
- rewrite after terminal TARGET_STATUS (cr_29fbbfde9d79) in `write_user_lock test`: No rewrite of TARGET_USER or TARGET_USER_VALID after a terminal status.
- writes after lock (cr_4c05e98bd6a2) in `write_user_lock test`: Only the initial valid=1 write; no later writes to either register.
- clear after grant (cr_8e68a65f2d45) in `write_user_lock test`: No write of 0 to TARGET_USER_VALID after the grant.
- non-Root writers (cr_afbf3aa32a43) in `write_user_lock test`: Only Root writes the target registers; no Target or other-agent write attempts.

5.5 Partly tested: 5 requirements in CaliptraSSIntegrationSpecification.md > MCU MBOX SRAM Sizing Requirements (and 3 more sections)

`f_a55fc0460027` · severity high

1. **Requirement** `cr_318abb731c64`: The MCU can be configured to implement up to 2 independent mailboxes.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > MCI Integration Requirements > MCU MBOX SRAM Sizing Requirements

   - `docs/CaliptraSSIntegrationSpecification.md:2263`: `The MCU can be configured to implement up to 2 independent mailboxes`

   Tested by `mci_mbox_dlen_wrap`**:** The test runs its SRAM staging and wipe procedure on both MCU mailboxes, deriving each mailbox's SRAM size from HW_CONFIG0. This shows that two mailboxes are present and working.
   Not tested: It does not check independence between the two mailboxes or that a mailbox configured with size 0 is not instantiated. It also does not check that there are no more than 2.

   Verified: An instantiation test exists, but independence and the limit of 2 mailboxes are unchecked.

   - `src/integration/test_suites/smoke_test_mcu_mbox_instantiation/cptra_smoke_test_mcu_mbox_instantiation.c:60`: `VPRINTF(LOW, "Caliptra: CSR Writes and Read as Valid AXI but no Mbox instantiation\n");` — This covers the not-instantiated mailbox case, which is part of item 1. It does not cover independence or the limit of 2 mailboxes.

2. **Requirement** `cr_554ae7995021`: If build-time straps are not used, trusted MBOX AXI users must be configured via the MBOX*_AXI_USER_LOCK register before SOC agents can use the MBOX.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > Programming interface > MCU Mailbox > MCU Mailbox Limited Trusted AXI users

   - `docs/CaliptraSSIntegrationSpecification.md:2293`: `If build-time integration straps are not used for configuring the trusted MBOX AXI users, then trusted users will need to configured with the following lockable MCI registers before MBOX can be used by SOC agents:`
   - `docs/CaliptraSSIntegrationSpecification.md:2296`: `` `MBOX*_AXI_USER_LOCK` ``

   Tested by `mcu_mbox_soc_agent_write_fw_image`**:** The SoC agent polls until the MBOX AXI user lock reads 1, then writes the FW image, DLEN, CMD and EXECUTE into the mailbox. The AXI write responses must be OKAY and the command status must reach complete. This shows the mailbox is usable once the lock is set.
   Not tested: Does not check that SoC agent accesses are rejected before the users are configured and locked. It also does not exercise the case where build-time straps are absent.

   Verified: No test references AXI_USER_LOCK, and none checks rejection before locking.

3. **Requirement** `cr_daf7f972a12c`: If build-time straps are not used, trusted MBOX AXI users must be configured via the MBOX*_VALID_AXI_USER register before SOC agents can use the MBOX.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > Programming interface > MCU Mailbox > MCU Mailbox Limited Trusted AXI users

   - `docs/CaliptraSSIntegrationSpecification.md:2293`: `If build-time integration straps are not used for configuring the trusted MBOX AXI users, then trusted users will need to configured with the following lockable MCI registers before MBOX can be used by SOC agents:`
   - `docs/CaliptraSSIntegrationSpecification.md:2295`: `` `MBOX*_VALID_AXI_USER` ``

   Tested by `mcu_mbox_soc_agent_write_fw_image`**:** The SoC agent polls until the MBOX valid AXI user register equals its own AXI user. It then uses the mailbox with accepted AXI writes, and the command status must reach complete. This shows the mailbox is usable after the valid user is configured.
   Not tested: Does not check that accesses are rejected before the valid user is programmed. It does not exercise the no-strap case, and the poll on the valid-user register has no check of its own.

   Verified: No test references VALID_AXI_USER, and none checks rejection before the register is programmed.

4. **Requirement** `cr_3f38fb82f957`: When MBOX_DLEN is 0 and MBOX_EXECUTION is cleared, the clearing logic erases the entire MBOX SRAM.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > Programming interface > MCU Mailbox > MCU Mailbox Doorbell Command DLEN

   - `docs/CaliptraSSIntegrationSpecification.md:2306`: `When MBOX_DLEN = 0 and MBOX_EXECUTION is cleared, the clearing logic erases the entire MBOX SRAM.`

   Tested by `smoke_test_mcu_mbox_zeroize`**:** The test sets and clears execute after writing data, then re-acquires the lock and checks that the SRAM is zero up to the max DLEN. This exercises the clear-on-execute-clear path, but only with a nonzero DLEN.
   Not tested: The DLEN = 0 case is never exercised. The test does not check that the entire MBOX SRAM is erased when DLEN is 0 and execute is cleared, because it only checks zeroing up to mbox_dlen.

   Verified: The only DLEN=0 execute-clear test checks the lock value, not that the SRAM is zero.

   - `src/integration/test_suites/mcu_mbox_lock_return_one_during_zeroize/mcu_mbox_lock_return_one_during_zeroize.c:69`: `VPRINTF(LOW, "MCU: Mbox%x write DLEN: 0x0\n", mbox_num, mbox_dlen);` — This is the only test that clears EXECUTE with DLEN=0. It checks only that lock reads 1 during zeroize, not that the SRAM is zero.

5. **Requirement** `cr_de12a01e738b`: After EXECUTE=1 is written, the mailbox owner becomes SOC.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > Programming interface > MCU Mailbox > MCU to SOC Target Flow

   - `docs/CaliptraSSIntegrationSpecification.md:2329`: `Note over MBOX,SOC: owner=SOC`

   Tested by `smoke_test_mcu_mbox_target_user`**:** After the grant the target's accesses to the mailbox succeed, which shows target ownership indirectly. Nothing checks the SRAM owner field or HW_STATUS owner directly.
   Not tested: Direct check that the owner reads SOC/Target after EXECUTE=1 with a valid target

   Tested by `smoke_test_mcu_mbox0_write_user_lock`**:** After TARGET_USER_VALID is written the SRAM owner is checked to be Target and the Target stays owner. The valid bit is written after the requester's EXECUTE, not as an MCU-staged request followed by

EXECUTE=1.
Not tested: The MCU-requester ordering of TARGET_USER_VALID then EXECUTE=1 causing the owner change

Tested by `mci_comm_flow_transaction`**:** Walks a transaction in which Root grants the Target after EXECUTE=1. At the target-owns-request phase it checks that the expected SRAM owner is the Target and that non-owner accesses are blocked.
Not tested: Requester here is not the MCU, and the owner is checked at a phase level rather than immediately after the EXECUTE write

Verified: I found no direct owner check immediately after EXECUTE=1 in an MCU-requester flow. This rests on a grep, not a detailed read of every target-user test.

Searched:

- DLEN=0 followed by execute clear in the mbox tests in `src/integration/test_suites/*mbox*/*.c`: Only mcu_mbox_lock_return_one_during_zeroize, with no SRAM-zero check.
- VALID_AXI_USER or AXI_USER_LOCK references in `src/integration mbox tests`: No matches.
- Mailbox independence, HW_CONFIG0 and MCU_MBOX1 usage in `src/integration/test_suites/*mbox*/*.c`: The instantiation test covers the no-mailbox case. Nothing checks cross-mailbox independence.
- Owner check right after EXECUTE=1 with TARGET_USER_VALID in `src/integration/test_suites/*mbox*/*.c`: The target_user and write_user_lock tests check the owner, but not in an MCU-requester ordering immediately after EXECUTE=1.
- Rejection before AXI users are locked in `smoke_test_mcu_mbox_invalid_axi and smoke_test_mcu_mbox_valid_user`: The invalid-AXI test uses a user that is invalid by configuration. It does not test the unconfigured state before locking.

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_invalid_axi/cptra_smoke_test_mcu_mbox_invalid_axi.c:45`: It checks that an invalid user gets AXI errors. It does not cover the before-lock or no-strap programming flow.

5.6 Partly tested: 3 requirements in mcu_mbox_csr.rdl > mbox_target_user_valid (and 2 more sections)

`f_66e7dd7d0b36` · severity high

1. **Requirement** `cr_8c87d704146c`: Software writes to TARGET_USER_VALID are accepted only from Root while the field is 0.
   Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_target_user_valid

   - `src/mci/rtl/mcu_mbox_csr.rdl:79`: `Writable only by Root while this field is 0.`

   Tested by `smoke_test_mcu_mbox0_write_user_lock`**:** Root writes TARGET_USER_VALID while it is 0 and it reads back 1, so Root acceptance is verified.
   Not tested: Rejection of non-Root writes and of writes when the field is already 1 is not checked.

   Tested by `smoke_test_mcu_mbox_valid_user`**:** A non-Root user's write of 1 to TARGET_USER_VALID is checked to be rejected (field stays 0).
   Not tested: Root acceptance while the field is 0 is not checked, nor is rejection of Root writes once the field is 1.

   Verified: No check found that Root's re-write of TARGET_USER_VALID is ignored once the field is 1, or that a non-Root write is rejected while it is 0 (the smoke tests only cover part of this).

2. **Requirement** `cr_7aa648529b38`: mcu_mbox_csr.mbox_target_status.status: software writes take effect only when mcu_mbox_csr.valid_target_req is asserted.
   Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_target_status

   - `src/mci/rtl/mcu_mbox_csr.rdl:122`: `swwe=valid_target_req`

   Tested by `smoke_test_mcu_mbox_target_user`**:** The target's SRAM and CSR accesses are attempted while TARGET_USER_VALID is 0 and nothing changes. After the grant, the target's COMPLETE status write takes effect.
   Not tested: A TARGET_STATUS write specifically attempted while valid_target_req is deasserted is not clearly checked as blocked. The target-not-valid CSR check reads back zeros, which could be the reset value.

   Tested by `smoke_test_mcu_mbox_strb_wr_csr`**:** A Root byte write of 0x3 to TARGET_STATUS is attempted and the register is checked to be unchanged.
   Not tested: The write comes from Root, not a target user, so the case of a valid target with valid_target_req asserted is not covered.

   Tested by `smoke_test_mcu_mbox0_write_user_lock`**:** A Root write to TARGET_STATUS is attempted and TARGET_STATUS is checked to be unchanged.
   Not tested: The write comes from Root with no valid target. Writes by a valid target are not checked.

Verified: No test attempts a TARGET_STATUS write by the future target while valid_target_req is low, and none checks that the write is rejected again after the field is cleared.

3. **Requirement** `cr_b84a00bfdcae`: Only the MCU writes the status field.
   Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_cmd_status

   - `src/mci/rtl/mcu_mbox_csr.rdl:130`: `Written only by MCU and polled by the Requester.`

   Tested by `smoke_test_mcu_mbox_valid_user`**:** A non-MCU user (the Caliptra uC requester) attempts to write MCU_MBOX_CMD_FAILURE to CMD_STATUS, and the test checks it still reads 0.
   Not tested: Only one non-MCU agent is exercised; Target and other SoC users are not tried.

   Tested by `smoke_test_mcu_mbox_pass_data`**:** The non-MCU Caliptra requester attempts to write CMD_STATUS and the test checks the written value is not read back.
   Not tested: Only one non-MCU agent is exercised; Target and other SoC users are not tried.

   Verified: No test has the Target or a SoC agent write CMD_STATUS while it is not MCU-owned and then checks the value is unchanged. Only the Caliptra uC requester is exercised.

Items 2, 4 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- Root re-writes of TARGET_USER_VALID, and non-Root writes to it, in tests and the SV testbench in `src/integration`: The only Root write is the grant in the ownership flow. There is no re-write after the field reads 1, and no check that a Root write is accepted again after hardware clears it.
- Terminal TARGET_STATUS values and the resulting TARGET_USER_VALID clear and ownership change in `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh`: All three terminal statuses are run. The test checks sram_owner returns to Root. It does not read TARGET_USER_VALID directly.
- TARGET_STATUS writes attempted while valid_target_req is low in `src/integration`: The ownership flow's non-owner writes are SRAM and EXECUTE only. The smoke tests write TARGET_STATUS only as Root. Nothing tests a future target's write being blocked.
- TARGET_DONE notification for each terminal status in `src/integration/testbench/sv_tests/mci/mci_common_tasks.svh`: mcu_mbox_wait_clear_notif fatals if TARGET_DONE never sets, and the test runs it for every terminal status. The test does not check that non-terminal values leave the notification unset.
- Target or SoC writes to CMD_STATUS while it is not MCU-owned in `src/integration`: None. Only the Caliptra uC requester attempt in the smoke tests, which is already partial.

Looked at and ruled out:

- `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:77-87`: Non-owner agents write only SRAM and EXECUTE here. They do not write TARGET_STATUS, TARGET_USER_VALID or CMD_STATUS, so this does not cover items 1, 3 or 5.

5.7 Test plan items with no test: 1 plan item

`f_7613d2a9d0a5` · severity medium

1. **Plan item:** SOC Data available interrupt on MCU EXECUTE — SOC Data available interrupt is set when the MCU holds the lock and sets EXECUTE. (`docs/Caliptra_Gen2_SS_TestPlan.xlsx`)

   Verified: No test has the MCU hold the lock, set EXECUTE and check the SOC data-available interrupt. The hitless test only covers the SoC-set EXECUTE case.

   - `src/integration/test_suites/smoke_test_mcu_hitless/smoke_test_mcu_hitless.c:151-157`: `// Wait for SOC Data Available Interrupt (sample hitless FW image will be written from BFM sequence mcu_mbox_soc_agent_write_fw_image.svh)` — The data-available interrupt is exercised here, but the SoC BFM sets EXECUTE, not the MCU holding the lock. This does not match the plan item.
   - `src/mci/rtl/mcu_mbox.sv:392-393`: `assign soc_mbox_data_available = hwif_out.mbox_execute.execute.value && (hwif_out.mbox_user.user.value != strap_root_axi_user);` — RTL: the SoC data-available condition is EXECUTE set while the user is not the root (MCU) user.

Item 2 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- DEFAULT_USER / 0xFFFFFFFF user usage in `whole repo, excluding xlsx`: smoke_test_mcu_mbox.c uses 0xFFFFFFFF as the Caliptra user with no valid user configured, and checks lock and data.

- soc data avail interrupt tests in `src/integration/test_suites`: Only smoke_test_mcu_hitless waits on it, and there the SoC side sets EXECUTE. No test has the MCU hold the lock, set EXECUTE and check the SoC data-available interrupt.
- soc_req_while_mcu_lock tests in `src/integration/test_suites`: Used in valid_user and target_user tests, but that is a different interrupt (SoC request while MCU holds the lock).

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_valid_user/smoke_test_mcu_mbox_valid_user.c:233-239`: Checks the soc_req_while_mcu_lock interrupt, not the SOC data-available interrupt set by the MCU's own EXECUTE.

5.8 Test plan items with no test: 4 plan items

`f_a06f18895dc1` · severity medium

1. **Plan item:** MCU MBOX - on reset deassertion MCU automically gets lock — On reset deassertion the MCU automatically acquires the mailbox lock. (`docs/Caliptra_Gen2_SS_TestPlan.xlsx`)

   Verified: No test checks that MCU holds the lock after reset deassertion. The helper only clears it.

   - `src/integration/test_suites/libs/caliptra_ss_lib/caliptra_ss_lib.c:314-323`: `void mcu_mbox_clear_lock_out_of_reset(uint32_t mbox_num) {` — The reset-lock helper only clears DLEN and EXECUTE and prints a message. It does not check that MCU holds the lock after reset.

2. **Plan item:** MCU MBOX - Get lock on mailbox by one user and verify the USER value with an agent with a different USER ID. — One agent acquires the MCU mailbox lock, then the USER value is verified from an agent with a different USER ID. (`docs/Caliptra_Gen2_SS_TestPlan.xlsx`)

   Verified: The USER value is read only by MCU itself. No second agent with a different USER ID checks it.

   - `src/integration/test_suites/smoke_test_mcu_mbox_axi_param/smoke_test_mcu_mbox_axi_param.c:47`: `uint32_t axi_select = xorshift32() % 5;` — The randomization is of the Caliptra uC AXI user selection. It is not MCU LSU AXI USER randomization.

3. **Plan item:** MCU MBOX - SRAM size between 0 and 2 MB in 1KB sizes (0, 81, 512, 2MB) — Exercise the MCU mailbox with SRAM sizes from 0 to 2 MB in 1KB granularity. (`docs/Caliptra_Gen2_SS_TestPlan.xlsx`)

   Verified: The tests use whatever SRAM size is configured. I found no sweep over the listed sizes.

   - `src/integration/test_suites/smoke_test_mcu_mbox_valid_user/smoke_test_mcu_mbox_valid_user.c:126-130`: `uint32_t sram_size_kb = mcu_mbox_get_sram_size_kb(mbox_num);` — The tests read the configured SRAM size. They do not sweep the sizes 0, 81, 512 and 2MB.

4. **Plan item:** MCU MBOX - randomize MCU LSU AXI USER — Randomize the MCU LSU AXI USER value on mailbox accesses. (`docs/Caliptra_Gen2_SS_TestPlan.xlsx`)

   Verified: MCU_LSU_AXI_USER is not randomized on mailbox accesses. The only randomization found is the Caliptra uC AXI selection.

Item 4 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- tests that check the MCU lock after reset deassertion in `test_suites mcu_mbox*`: Only the mcu_mbox_clear_lock_out_of_reset helper, which clears the lock without checking it.
- SRAM size sweep and parameter configurations in `test_suites yml files and C files`: The tests only read the configured size. No sweep over sizes was found.
- randomized MCU LSU AXI USER and cross-agent USER check in `test_suites`: target_user compares MBOX_USER to MCU_LSU_AXI_USER, but the value is not randomized.
- valid user CSR works with LOCK set and the parameter not enabled in `valid_user and axi_param tests`: Close but not confirmed. valid_user has MCU acquire the lock and use CSR valid users. axi_param checks CSR-configured users.

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_valid_user/smoke_test_mcu_mbox_valid_user.c:96-110`: MCU acquires the lock and checks that the USER is MCU. This is not verified from a different-USER agent, and it does not check the lock held at reset.

5.9 Partly tested: 1 requirement in CaliptraSSHardwareSpecification.md > MCU Mailbox Address Map

`f_fbbd20161449` · severity medium

1. **Requirement** `cr_e51e0acebd4b`: MBOX always reserves 2MB of address space regardless of SRAM size.
   - `docs/CaliptraSSHardwareSpecification.md:1548`: `MBOX always reserves 2MB address space`

   Tested by `smoke_test_mcu_mbox_valid_user`**:** Writes to invalid SRAM addresses (when SRAM is smaller than the max) are read back as 0, so addresses beyond the SRAM are reserved and handled.
   Not tested: The test does not check that the reserved space is exactly 2MB, nor does it vary the SRAM size.

   Verified: The gap stands. Random invalid addresses are checked to read 0, but the exact 2MB boundary and the address above 2MB are not tested.

   - `src/integration/test_suites/smoke_test_mcu_mbox_valid_user/smoke_test_mcu_mbox_valid_user.c:124-143`: `// If SRAM is <2MB, write and read to a handful of random locations in invalid addresses` — Random invalid addresses are written, then read back and checked for 0. The boundary addresses (last valid SRAM word, 2MB top, just above 2MB) are never exercised.

Searched:

- 2MB window / boundary checks in mbox tests in `src/integration/test_suites`: Only the random-address invalid-SRAM checks in the smoke_test_mcu_mbox_* tests.

5.10 In a test plan, but no test found: 5 requirements in CaliptraSSHardwareSpecification.md > MCU Mailbox Fully addressable SRAM (and 1 more section)

`f_d5f416071152` · severity medium

1. **Requirement** `cr_52fe20427e16`: If the MCU mailbox SRAM size is configured as 0, the mailbox is not instantiated.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Fully addressable SRAM
   - `docs/CaliptraSSHardwareSpecification.md:1456`: `If set to 0 the mailbox is not instantiated.`
   - `docs/CaliptraSSIntegrationSpecification.md:2263`: `If they are configured sizes of 0, they will not be instantiated.`

   Nearest evidence (not counted as a test): `MCU MBOX - SRAM size between 0 and 2 MB in 1KB sizes (0, 81, 512, 2MB)` (plan item only); `MCU MBOX - MBOX SRAM 0 remove MBOX instantiation and AXI access repond with an error` (plan item only)

   Verified: No test configures the size as 0 or checks that the mailbox is not instantiated.

   - `src/integration/test_suites/smoke_test_mcu_mbox_axi_param/cptra_smoke_test_mcu_mbox_axi_param.c:109-110`: `uint32_t sram_size_kb = cptra_mcu_mbox_get_sram_size_kb(mbox_num);` — Tests only read the configured SRAM size. They do not sweep sizes or build a size-0 configuration.

2. **Requirement** `cr_6e68d4291d35`: BYTE accesses are not supported by the MCU mailbox SRAM.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Fully addressable SRAM
   - `docs/CaliptraSSHardwareSpecification.md:1458`: `BYTE accesses are not supported by the SRAM.`

   Nearest evidence (not counted as a test): `MCU MBOX - STRBW to CSRs (not supported to SRAM)` (plan item only)

   Verified: No test checks that BYTE accesses to the SRAM are rejected.

3. **Requirement** `cr_8ea60cdfbb2b`: The maximum MCU mailbox SRAM size is 2MB.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Fully addressable SRAM
   - `docs/CaliptraSSHardwareSpecification.md:1454`: `**Max Size**: 2MB`

   Nearest evidence (not counted as a test): `MCU MBOX - SRAM size between 0 and 2 MB in 1KB sizes (0, 81, 512, 2MB)` (plan item only)

   Verified: No test builds or exercises a 2MB size configuration.

4. **Requirement** `cr_e0684916d838`: The minimum MCU mailbox SRAM size is 0.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Fully addressable SRAM

   - `docs/CaliptraSSHardwareSpecification.md:1452`: `**Min Size**: 0`

   Nearest evidence (not counted as a test): `MCU MBOX - SRAM size between 0 and 2 MB in 1KB sizes (0, 81, 512, 2MB)` (plan item only)

   Verified: No test builds a size-0 configuration.

5. **Requirement** `cr_12dbdc943d69`: notif_mbox*_cmd_avail is asserted when a SOC agent gets the lock and asserts the EXECUTE register.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Interrupts

   - `docs/CaliptraSSHardwareSpecification.md:1482`: `Mailbox data available from SOC`
   - `docs/CaliptraSSHardwareSpecification.md:1482`: `` `notif_mbox*_cmd_avail` ``
   - `docs/CaliptraSSHardwareSpecification.md:1482`: `Asserted when a SOC agent gets lock and asserts the EXECUTE register, indicating data is available for MCU.`

   Nearest evidence (not counted as a test): `MCU CMD available interrupt on SOC EXECUTE` (plan item only); `mci_comm_flow_transaction` (stimulus only, no check)

   Verified: Only the clear of the interrupt is checked. Assertion on SOC EXECUTE is never checked.

   - `src/integration/test_suites/smoke_test_mcu_mbox_valid_user/smoke_test_mcu_mbox_valid_user.c:224-225` — Only the clear of CMD_AVAIL is checked. Nothing checks that it was asserted when the SOC set EXECUTE.

     ```
     // Clear the CMD_AVAIL interrupt and check that it gets cleared
         mcu_mbox_clear_mbox_cmd_avail_interrupt(mbox_num);
     ```

   - `src/integration/test_suites/libs/caliptra_ss_lib/caliptra_ss_lib.c:367-369` — This is a clear check only. It compares the masked value with 1, which is weak, and there is no assertion check.

     ```
     // Check that Mailbox cmd available from SOC interrupt has been cleared
         if((lsu_read_32(SOC_MCI_TOP_MCI_REG_INTR_BLOCK_RF_NOTIF0_INTERNAL_INTR_R) &
     ```

Searched:

- cmd_avail in the integration tests and test libraries in `src/integration`: Only the clear helper and the ISR enable mask. No check that the interrupt asserts on SOC EXECUTE.
- SRAM size configuration and sweep tests in `src/integration/test_suites/*mcu_mbox*`: Tests only read the size through get_sram_size_kb. No size-0 or 2MB configuration test, and no check that the mailbox is not instantiated.
- BYTE or strobe access to the SRAM in `smoke_test_mcu_mbox_valid_user`: Nothing.

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_valid_user/smoke_test_mcu_mbox_valid_user.c:126-130`: Accesses the range above the configured size, but does not cover size 0, the 2MB maximum, or non-instantiation.

5.11 No test found: 4 requirements in CaliptraSSHardwareSpecification.md > MCU Mailbox Fully addressable SRAM (and 3 more sections)

`f_a278e733f80a` · severity medium

1. **Requirement** `cr_a3dd90e3ad99`: Accesses to the MCU mailbox SRAM must be DWORD aligned.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Fully addressable SRAM

   - `docs/CaliptraSSHardwareSpecification.md:1458`: `Accesses must be DWORD aligned`

   Verified: No test of unaligned SRAM access was found.

2. **Requirement** `cr_0d49e75f5219`: CMD_STATUS is unused by MCU when MCU holds the lock.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox MCU Access

- `docs/CaliptraSSHardwareSpecification.md:1534`: `` `CMD_STATUS` | Unused ``

Verified: No test checks that CMD_STATUS is unused when MCU holds the lock.

- `src/integration/test_suites/smoke_test_mcu_mbox_valid_user/smoke_test_mcu_mbox_valid_user.c:145-147`: `// MBOX: Write CMD_STATUS for testing` — MCU writes CMD_STATUS in its own tests. This does not check that CMD_STATUS is unused when MCU holds the lock.

3. **Requirement** `cr_22b98fe21dde`: MBOX CSR occupies address range 0x0020_0000 to 0x020_003F.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Address Map

   - `docs/CaliptraSSHardwareSpecification.md:1546`: `0x0020_0000`
   - `docs/CaliptraSSHardwareSpecification.md:1546`: `0x020_003F`
   - `docs/CaliptraSSHardwareSpecification.md:1546`: `MBOX CSR`
   - `docs/CaliptraSSHardwareSpecification.md:1546`: `Mailbox Control Status Registers`

   Verified: No test checks the CSR range boundaries.

4. **Requirement** `cr_488fae088122`: MBOX SRAM occupies address range 0x0000_0000 to 0x01F_FFFF.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Address Map

   - `docs/CaliptraSSHardwareSpecification.md:1545`: `0x0000_0000`
   - `docs/CaliptraSSHardwareSpecification.md:1545`: `0x01F_FFFF*`
   - `docs/CaliptraSSHardwareSpecification.md:1545`: `MBOX SRAM`
   - `docs/CaliptraSSHardwareSpecification.md:1545`: `Mailbox SRAM`

   Verified: No test checks the SRAM range boundaries.

Item 2 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- unaligned/misaligned access tests in `src/integration/test_suites, src/mci`: No matches in mbox or mci directories.
- mbox interrupt tests in `test_suites`: Only mcu_intr_handler, which exercises the trig and count registers.
- CMD_STATUS tests in `test_suites/*mbox*`: Tests write and read CMD_STATUS as a functional register. None checks that it is unused when MCU holds the lock.
- address map boundary tests in `test_suites`: Tests use the base macros and hit addresses incidentally. None checks the range boundaries.

5.12 In a test plan, but no test found: 2 requirements in CaliptraSSHardwareSpecification.md > MCU Mailbox Interrupts (and 1 more section)

`f_8fb5fb2b3742` · severity medium

1. **Requirement** `cr_4620abd4f768`: cptra_ss_soc_mcu_mbox*_data_avail is asserted while MCU holds the mailbox lock and EXECUTE is 1.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Interrupts

   - `docs/CaliptraSSHardwareSpecification.md:1493`: `Mailbox data available from MCU`
   - `docs/CaliptraSSHardwareSpecification.md:1493`: `` `cptra_ss_soc_mcu_mbox*_data_avail` ``
   - `docs/CaliptraSSHardwareSpecification.md:1493`: `Asserted while MCU holds the mailbox lock and EXECUTE is 1.`

   Nearest evidence (not counted as a test): `SOC Data available interrupt on MCU EXECUTE` (plan item only); `soc_mcu_mbox1_data_avail_cp` (coverage only); `soc_mcu_mbox0_data_avail_cp` (coverage only)

   Verified: No test or assertion samples the data_avail output signal while MCU holds the lock and EXECUTE is 1. Only the interrupt status is polled, in the hitless test.

   - `src/integration/test_suites/smoke_test_mcu_hitless/smoke_test_mcu_hitless.c:152`: `if(!mcu_mbox_wait_for_soc_data_avail_interrupt(mbox_num, 100000)) {` — This only polls an interrupt status register. It does not sample the cptra_ss_soc_mcu_mbox*_data_avail output, and it does not check the MCU-held-lock-and-EXECUTE condition.

2. **Requirement** `cr_1fd84d60bc97`: MBOX SRAM size is configurable.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Address Map

   - `docs/CaliptraSSHardwareSpecification.md:1548`: `MBOX SRAM size is configurable`

   Nearest evidence (not counted as a test): `MCU MBOX - SRAM size between 0 and 2 MB in 1KB sizes (0, 81, 512, 2MB)` (plan item only)

   Verified: The tests adapt to the single configured SRAM size. None exercises several sizes or checks the configurability.

   - `src/integration/test_suites/mcu_mbox_rand_rdwr/cptra_mcu_mbox_rand_rdwr.c:58`: `uint32_t sram_size_kb = cptra_mcu_mbox_get_sram_size_kb(mbox_num);` — The test reads whatever size is configured and adapts to it. It does not vary the size or check that the size is configurable.

Searched:

- data_avail in tests, sequences and the testbench in `src/integration (c, h, svh)`: Only the SoC data-avail interrupt helper functions and their use in the hitless test. No checker or assertion on the output signal.
- SRAM size usage in tests in `src/integration/test_suites`: Tests read the configured size at run time with get_sram_size_kb. I found no test that sweeps or changes the size.

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_valid_user/smoke_test_mcu_mbox_valid_user.c:126-130`: It probes addresses beyond the configured size, but only for the one build-time size. It does not show that the size is configurable.

5.13 No test found: 3 requirements in CaliptraSSHardwareSpecification.md > MCU Mailbox Status Registers (and 1 more section)

`f_0efe09d4bda3` · severity medium

2. **Requirement** `cr_666421936fb3`: CMD_STATUS and TARGET_STATUS value 0 encodes BUSY.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Status Registers

   - `docs/CaliptraSSHardwareSpecification.md:1419`: `` `CMD_STATUS` and `TARGET_STATUS` use the following encodings: ``
   - `docs/CaliptraSSHardwareSpecification.md:1423`: `| 0 | BUSY |`

   Verified: The BUSY=0 encoding is only an enum; no test checks BUSY behavior.

   - `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:136-139`: `// Root finalizes the response and DLEN; a non-BUSY CMD_STATUS returns the SRAM to the Requester.` — Terminal statuses are driven, but no test writes BUSY (0) or checks the encoding; this does not check items 2-4.

3. **Requirement** `cr_680e463fd93d`: A terminal Target status is not visible as command completion to the original Requester until MCU writes a terminal CMD_STATUS.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Status Registers

   - `docs/CaliptraSSHardwareSpecification.md:1430-1432`

     ```
     A terminal Target status shall not be
     visible as command completion to the original Requester until MCU separately
     writes a terminal `CMD_STATUS`.
     ```

   Verified: Only SRAM ownership and reads are checked. The Requester-visible completion is not checked.

4. **Requirement** `cr_cd3b9f710b55`: CMD_STATUS is not used when MCU is the Requester.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Status Registers

   - `docs/CaliptraSSHardwareSpecification.md:1436-1437`

     ```
     `CMD_STATUS` is not used when MCU is the Requester, because there is no
     external Requester to serve.
     ```

   Verified: No test covers CMD_STATUS being unused when MCU is the Requester.

Items 1, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- BUSY encoding, SRAM_OWNER NONE and CMD_STATUS use in tests in `src/integration`: MCU_MBOX_CMD_BUSY and MCU_MBOX_TARGET_STATUS_CMD_BUSY appear only as enum definitions in caliptra_ss_lib.h. No test writes or checks BUSY. OWNER_NONE is checked in the ownership flow.
- MCU as Requester and CMD_STATUS in `src/integration`: Only mci_mbox_dlen_wrap.svh mentions Root as Requester. It does not check CMD_STATUS being unused.

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_usr_lock_out_zero/cptra_smoke_test_mcu_mbox_usr_lock_out.c:175`: Reads CMD_STATUS expecting 0 as an access-lockout check, not a BUSY-encoding check.
- `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:129-133`: Checks SRAM ownership returns to Root after a terminal TARGET_STATUS. It does not check that the Requester sees no command completion until CMD_STATUS is written.

5.14 No test found: 4 requirements in CaliptraSSHardwareSpecification.md > MCU Mailbox Target User (and 1 more section)

`f_dae84f996704` · severity medium

1. **Requirement** `cr_12643f17b618`: After the clear, MCU can program and validate the next Target user.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Target User

   - `docs/CaliptraSSHardwareSpecification.md:1351-1352`: `after which MCU can program and validate the next Target user.`

   Verified: The flow re-programs TARGET_USER in later transactions only after a lock release, and no check covers the clear-then-reprogram sequence directly.

   - `src/integration/test_suites/smoke_test_mcu_mbox_target_user/smoke_test_mcu_mbox_target_user.c:250`: `// Hardware clears both target configuration registers on terminal status.` — The test checks that TARGET_USER_VALID is cleared, but does not check EXECUTE and does not exercise any MCI reset recovery.

2. **Requirement** `cr_eb54be3afc42`: The Target user finishes all SRAM and DMA accesses before writing a terminal TARGET_STATUS value.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Target User

   - `docs/CaliptraSSHardwareSpecification.md:1345-1346`: `The Target user shall finish all SRAM and DMA accesses before writing a terminal `TARGET_STATUS` value`

   Verified: This is a Target-side behaviour. No check detects a Target finishing its accesses after a terminal write.

   - `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:130-131` — The Target terminal write is checked only through the resulting owner (Root); EXECUTE is never read back.

     ```
     mci_comm_flow_write_data(mci_comm_flow_target);
     mcu_mbox_write(mci_comm_flow_target_status_addr, mci_comm_flow_target, status);
     ```

3. **Requirement** `cr_f484f55cbce1`: EXECUTE remains 1 after the terminal TARGET_STATUS write.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox Target User

   - `docs/CaliptraSSHardwareSpecification.md:1352`: `` `EXECUTE` remains 1 ``

   Verified: No test reads EXECUTE after the terminal TARGET_STATUS write. The checks look only at the SRAM owner and the cleared target registers.

4. **Requirement** `cr_3437364f9e35`: Only an MCI reset recovers the mailbox from a non-responsive Target.
   Section: docs/CaliptraSSHardwareSpecification.md > Manufacturer Control Interface (MCI) > MCI Feature Descriptions > MCU Mailbox > MCU Mailbox SRAM Ownership

   - `docs/CaliptraSSHardwareSpecification.md:1414-1415`

```
Only an MCI reset
recovers it.
```

Verified: I found no test for a non-responsive Target or for MCI-reset-only recovery.

Item 5 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- item cr_12643f17b618: re-programming the next Target user after the clear in `smoke_test_mcu_mbox_target_user.c and mci_mbox_ownership_flow.svh`: The smoke test runs a single transaction. The flow re-programs TARGET_USER in later transactions but only after a lock release. No direct check of clear-then-reprogram.
- item cr_eb54be3afc42: Target finishing SRAM/DMA accesses before the terminal write in `cptra_smoke_test_mcu_mbox_target_user.c and the flow`: No check on the Target's own access ordering.
- item cr_f484f55cbce1: EXECUTE read after the terminal TARGET_STATUS write in `grep for mcu_mbox_read_execute/EXECUTE in smoke_test_mcu_mbox_target_user`: Only a non-owner EXECUTE check in the Caliptra-side test. No EXECUTE read after the terminal status.
- item cr_3437364f9e35: non-responsive Target recovered only by MCI reset in `grep for non-responsive/unresponsive across src`: Hits only in pwrmgr, tlul and USB docs; nothing for the MCU mailbox.
- item cr_51f02e52744d: no SRAM owner when lock is free in `mci_mbox_ownership_flow.svh`: mci_comm_flow_end calls check_phase("released", MCU_MBOX_OWNER_NONE), which asserts the SRAM owner is NONE after EXECUTE=0 with the lock free.

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_target_user/smoke_test_mcu_mbox_target_user.c:207-216`: Covers the terminal status and the TARGET_USER_VALID clear, but not EXECUTE staying 1 or re-programming the next Target user.

5.15 In a test plan, but no test found: 1 requirement in CaliptraSSIntegrationSpecification.md > MCU MBOX SRAM Sizing Requirements (and 1 more section)

`f_6492b853e510` · severity medium

1. **Requirement** `cr_3877f33b1448`: MCU MBOX1 SRAM size is set by the MCU_MBOX1_SIZE_KB parameter.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > MCI Integration Requirements > MCU MBOX SRAM Sizing Requirements

   - `docs/CaliptraSSIntegrationSpecification.md:2261`: `MCU MBOX SRAM size is set via `MCU_MBOX0_SIZE_KB` and `MCU_MBOX1_SIZE_KB` parameters`

   Nearest evidence (not counted as a test): `MCU MBOX - SRAM size between 0 and 2 MB in 1KB sizes (0, 81, 512, 2MB)` (plan item only)

   Verified: No test varies or checks MCU_MBOX1_SIZE_KB. The testbenches only set fixed values (256 and 4).

   - `src/integration/testbench/tb_top_pkg.sv:60`: `localparam MCU_MBOX1_SIZE_KB = 256;` — The integration testbench fixes the mailbox size at a single value; there is no sweep.

Item 2 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- MBOX1_SIZE_KB usage in tests and testbenches in `repo`: Only a fixed value in tb_top_pkg (256) and mci_top_tb (4); no sweep and no size check.
- cmd_avail in the test_suites firmware tests in `src/integration/test_suites`: Only a clear helper and clock-gating comments; no check of the notification.
- mci_comm_flow_transaction and the wait helper in `src/integration/testbench/sv_tests/mci`: The helper polls the NOTIF0 status and calls $fatal if it never sets.

Looked at and ruled out:

- `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:115-117`: Used to withdraw item 2. It checks the notification status bit rather than the notif_mbox*_cmd_avail interrupt pin at the MCU, but the status bit is the same notification.
- `src/mci/tb/mci_top_tb.sv:32-33`: Sets one fixed size of 4 KB, with no sweep and no size check.

5.16 Only weak evidence (no check that can fail): 3 requirements in CaliptraSSIntegrationSpecification.md > MCU to SOC Target Flow (and 1 more section)

`f_7c63d47a9d15` · severity medium

1. **Requirement** `cr_b395303f48f6`: The cptra_ss_soc_mcu_mbox*_data_avail signal asserts when the Target uses the subsystem sideband wire.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > Programming interface > MCU Mailbox > MCU to SOC Target Flow

   - `docs/CaliptraSSIntegrationSpecification.md:2330-2331`: `alt Target uses the subsystem sideband wire MBOX-)SOC: cptra_ss_soc_mcu_mbox*_data_avail asserts`

   Nearest evidence (not counted as a test): `soc_mcu_mbox1_data_avail_cp` (coverage only); `soc_mcu_mbox0_data_avail_cp` (coverage only)

   Verified: The wire is only covered by a coverpoint, with no check that it asserts.

   - `src/mci/coverage/mci_top_cov_if.sv:175-176`: `soc_mcu_mbox0_data_avail_cp: coverpoint soc_mcu_mbox0_data_avail;` — Coverage only; it has no pass/fail check.

2. **Requirement** `cr_918e477394f8`: The cptra_ss_soc_mcu_mbox*_data_avail wire stays deasserted while MCU programs the target configuration.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > Programming interface > MCU Mailbox > SOC Requester to SOC Target Communication Flow (MCU as intermediary)

   - `docs/CaliptraSSIntegrationSpecification.md:2418-2420`: `Because the SOC Requester holds the lock in this flow, the `cptra_ss_soc_mcu_mbox*_data_avail` wire stays deasserted when MCU programs the target configuration.`

   Nearest evidence (not counted as a test): `soc_mcu_mbox1_data_avail_cp` (coverage only); `soc_mcu_mbox0_data_avail_cp` (coverage only)

   Verified: No check that the wire stays low while MCU programs the target configuration.

3. **Requirement** `cr_d880e054e4d3`: MCU writes TARGET_USER, sets TARGET_USER_VALID, and only then sends the sideband notification.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > Programming interface > MCU Mailbox > SOC Requester to SOC Target Communication Flow (MCU as intermediary)

   - `docs/CaliptraSSIntegrationSpecification.md:2421-2422`: `so MCU shall write `TARGET_USER`, set `TARGET_USER_VALID`, and then send the notification.`

   Nearest evidence (not counted as a test): `mci_comm_flow_transaction` (stimulus only, no check)

   Verified: The test only sets TARGET_USER_VALID and adds a delay. Nothing checks the write ordering relative to the notification.

Searched:

- soc_mcu_mbox*_data_avail use in `repo`: Only RTL, top-level wiring, the testbench port and the coverage interface. No assertion or checker.
- data_avail in testbench services and firmware lib in `src/integration`: The services file only forces the internal mcu_mbox*_data_avail signals. The lib.c functions poll the data_avail interrupt status, not the wire.

Looked at and ruled out:

- `src/integration/testbench/caliptra_ss_top_tb_services.sv:431-443`: Forces the internal signals as stimulus. It checks nothing about the sideband wire.
- `src/integration/test_suites/libs/caliptra_ss_lib/caliptra_ss_lib.c:412-433`: Checks the SoC data-avail interrupt, which is a different thing from the sideband wire.

5.17 No test found: 5 requirements in CaliptraSSIntegrationSpecification.md > MCU to SOC Target Flow (and 2 more sections)

`f_69aadf0ed71e` · severity medium

1. **Requirement** `cr_bb57badd8526`: Otherwise MCU notifies the Target directly through a sideband notification.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > Programming interface > MCU Mailbox > MCU to SOC Target Flow

- `docs/CaliptraSSIntegrationSpecification.md:2332-2333`: `else MCU notifies the Target directly MCU->>SOC: Sideband notification, for example a mailbox command`

Verified: No test of a sideband notification to the Target.

- `src/integration/test_suites/smoke_test_mcu_mbox_target_user/smoke_test_mcu_mbox_target_user.c:220`: `if(!is_mcu_mbox_target_done_interrupt_set(mbox_num)) {` — The test checks the target-done interrupt, not the absence of an interrupt on handoff, a sideband notification, or MCU validation.

2. **Requirement** `cr_dc1a3093f007`: While the command is being processed, reading CMD_STATUS returns BUSY.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > Programming interface > MCU Mailbox > SOC Requester to MCU Flow

   - `docs/CaliptraSSIntegrationSpecification.md:2362-2364`: `loop While CMD_STATUS is BUSY SOC->>MBOX: Read CMD_STATUS MBOX-->>SOC: BUSY`

   Verified: BUSY is never read or checked in any test.

   - `src/integration/test_suites/libs/caliptra_ss_lib/caliptra_ss_lib.h:239`: `MCU_MBOX_CMD_BUSY = 0x0,` — BUSY is only an enum definition; no test reads or checks it.

3. **Requirement** `cr_1d6b75b3d49f`: No MCU MBOX interrupt or data-avail wire asserts when ownership passes to the Target.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > Programming interface > MCU Mailbox > SOC Requester to SOC Target Communication Flow (MCU as intermediary)

   - `docs/CaliptraSSIntegrationSpecification.md:2401`: `Note over MBOX,TGT: No MCU MBOX interrupt or data-avail wire asserts here`

   Verified: No check that the interrupt and data-avail wires stay deasserted at handoff.

4. **Requirement** `cr_1e471e967ac0`: MCU sends a sideband notification to the Target.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > Programming interface > MCU Mailbox > SOC Requester to SOC Target Communication Flow (MCU as intermediary)

   - `docs/CaliptraSSIntegrationSpecification.md:2400`: `MCU->>TGT: Sideband notification, for example a Caliptra mailbox command`

   Verified: No sideband notification is checked.

5. **Requirement** `cr_d628ab084b9b`: MCU validates the command, range and target identity.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > Programming interface > MCU Mailbox > SOC Requester to SOC Target Communication Flow (MCU as intermediary)

   - `docs/CaliptraSSIntegrationSpecification.md:2396`: `MCU->>MBOX: Validate command, range, and target identity`

   Verified: No test checks MCU validation of command, range and target identity.

Searched:

- Item cr_bb57badd8526: sideband notification to Target, in mcu_mbox tests in `src/integration/test_suites`: No sideband notification check; the target_user test waits for TARGET_USER_VALID instead
- Item cr_dc1a3093f007: BUSY read from CMD_STATUS in `src/integration/test_suites`: Only enum definitions in caliptra_ss_lib.h
- Item cr_1d6b75b3d49f: no interrupt or data-avail on handoff in `smoke_test_mcu_mbox_target_user`: Only positive checks of the soc_req_while_mcu_lock and target_done interrupts; no negative check
- Item cr_1e471e967ac0: MCU sideband notification to the Target in `src/integration/test_suites`: None found
- Item cr_d628ab084b9b: MCU validation of command, range and target identity in `src/integration/test_suites`: No such check in any test

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_target_user/cptra_smoke_test_mcu_mbox_target_user.c:195-212`: Checks that the Target cannot write the TARGET_USER and TARGET_USER_VALID registers. It does not check the validation flow or sideband notification.

5.18 Partly tested: 2 requirements in CaliptraSSIntegrationSpecification.md > SOC Requester to MCU Flow (and 1 more section)

`f_fa3367a3301d` · severity medium

2. **Requirement** `cr_620b876ceb0a`: SOC Requester reads CMD_STATUS and the response.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > Programming interface > MCU Mailbox > SOC Requester to SOC Target Communication Flow (MCU as intermediary)

   - `docs/CaliptraSSIntegrationSpecification.md:2409`: `REQ->>MBOX: Read CMD_STATUS and response`

   Tested by `mci_comm_flow_transaction`**:** In the requester-owns-response phase the Requester is checked as owner and reads its payload, after the non-BUSY CMD_STATUS.
   Not tested: CMD_STATUS is not read or checked by the Requester.

   Verified: mci_comm_flow_transaction has no CMD_STATUS reads.

3. **Requirement** `cr_83b707b90303`: MCU reads TARGET_STATUS and finalizes the response.
   Section: docs/CaliptraSSIntegrationSpecification.md > MCI > Programming interface > MCU Mailbox > SOC Requester to SOC Target Communication Flow (MCU as intermediary)

   - `docs/CaliptraSSIntegrationSpecification.md:2406`: `MCU->>MBOX: Read TARGET_STATUS and finalize response`

   Tested by `smoke_test_mcu_mbox_target_user`**:** MCU waits for the target's COMPLETE TARGET_STATUS and the target done interrupt, then clears it.
   Not tested: Does not check that MCU reads TARGET_STATUS and finalizes the response (writes the final response data or non-BUSY CMD_STATUS).

   Verified: The target_user test waits for COMPLETE but has no finalize step.

   - `src/integration/test_suites/smoke_test_mcu_mbox_target_user/smoke_test_mcu_mbox_target_user.c:211-212`: `// Wait for the Target to publish a terminal TARGET_STATUS` — The MCU only waits for the target's TARGET_STATUS. No finalize-response step follows.

Item 1 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- CMD_STATUS in mci_comm_flow_transaction in `repo`: no matches
- TARGET_STATUS / CMD_STATUS in the target_user tests in `smoke_test_mcu_mbox_target_user`: the MCU waits for COMPLETE, but there is no finalize step

---

5.19 No test found: 5 requirements in mci_reg.rdl > MBOX0_AXI_USER_LOCK (and 1 more section)

`f_d12fe3cac0ac` · severity medium

1. **Requirement** `cr_0b9bfdf42da6`: mci_reg.MBOX0_AXI_USER_LOCK[].LOCK: software writes take effect only when true is deasserted.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > MBOX0_AXI_USER_LOCK

   - `src/mci/rtl/mci_reg.rdl:834`: `swwel`

   Verified: No test writes LOCK after it is set and checks the write is ignored.

   - `src/integration/test_suites/libs/caliptra_ss_lib/caliptra_ss_lib.c:507`: `lsu_write_32(SOC_MCI_TOP_MCI_REG_MBOX0_AXI_USER_LOCK_0 + MCU_MBOX_AXI_CFG_STRIDE*mbox_num, MCI_REG_MBOX0_AXI_USER_LOCK_0_LOCK_MASK);` — Setup write that sets the lock; no post-lock write attempt or readback check.

2. **Requirement** `cr_24d6af90a638`: The associated MBOX0_VALID_AXI_USER register is valid only once locked by this bit.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > MBOX0_AXI_USER_LOCK

   - `src/mci/rtl/mci_reg.rdl:831`: `Associated MBOX0_VALID_AXI_USER register is only valid once locked by this bit.`

   Verified: Tests always lock slots before use; no test uses an unlocked slot as a negative case.

   - `src/integration/testbench/sv_tests/mci/mci_common_tasks.svh:419-429`: `// Program valid-user slots 0..N-1 of the mailbox with `users`. Each slot is locked, since HW ignores unlocked slots.` — Lock is used only as setup; always lock after write, no write-after-lock check and no unlocked-slot negative test.

3. **Requirement** `cr_43669ff8e969`: The LOCK field is read-only once locked.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > MBOX0_AXI_USER_LOCK

   - `src/mci/rtl/mci_reg.rdl:833`: `Read-Only once locked.`

   Verified: The lock is only set and read back, with no write-after-lock check.

- `src/integration/testbench/sv_tests/mci/mcu_mbox_soc_agent_write_fw_image.svh:104-107`: `if (rd_data == 1) begin` — Only polls that LOCK reads 1; does not test read-only-after-lock or VALID write blocking.

4. **Requirement** `cr_b4c4fa8cc446`: Each LOCK bit locks its associated MBOX0_VALID_AXI_USER register.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > MBOX0_AXI_USER_LOCK

   - `src/mci/rtl/mci_reg.rdl:830`: `Each bit corresponds to locking the associated MB0OX_VALID_AXI_USER register.`

   Verified: The per-bit lock-to-slot association is only exercised as setup, with no check of locking behaviour.

5. **Requirement** `cr_8429ea5b7749`: AXI_USER register becomes read-only once locked by AXI_USER_LOCK.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > MBOX1_VALID_AXI_USER

   - `src/mci/rtl/mci_reg.rdl:840`: `Read-Only once locked by AXI_USER_LOCK.`

   Verified: No test writes VALID_AXI_USER after lock and checks the value is unchanged.

Searched:

- cr_0b9bfdf42da6: writes to LOCK after set (swwel), expecting ignore in `src/integration test_suites, testbench, libs`: Only single setup writes of LOCK; no second write or check
- cr_24d6af90a638: use of unlocked VALID_AXI_USER slot as negative test; grep unlocked/ignored in `src/integration .sv/.svh/.c`: nothing relevant; only comment in mci_common_tasks.svh
- cr_43669ff8e969: readback of LOCK / write-after-lock check in `mcu_mbox_soc_agent_write_fw_image.svh, smoke_test_mcu_hitless.c`: Only polls LOCK==1 and reads it; no write-after-lock check
- cr_b4c4fa8cc446: per-bit lock to slot association tests in `mci_common_tasks.svh, caliptra_ss_lib.c`: Address helpers and setup writes only; no check
- cr_8429ea5b7749: write VALID_AXI_USER after lock and check unchanged (MBOX1 too) in `src/integration, mci.c register tables`: mci.c lists registers only; no post-lock write check

Looked at and ruled out:

- `src/integration/test_suites/libs/mci/mci.c:1915-1933`: Register mask table entries for generic register tests; they do not exercise write-after-lock behaviour.
- `src/integration/test_suites/smoke_test_mcu_hitless/smoke_test_mcu_hitless.c:41-42`: Reads LOCK_4 and VALID_4 for hitless data, no check of lock semantics.

5.20 In a test plan, but no test found: 2 requirements in mci_reg.rdl > MBOX0_VALID_AXI_USER

`f_75de07a7b4a0` · severity medium

1. **Requirement** `cr_9a48d7d6fd74`: mci_reg.MBOX0_VALID_AXI_USER[].AXI_USER: software writes take effect only when true is deasserted.

   - `src/mci/rtl/mci_reg.rdl:824`: `swwel`

   Nearest evidence (not counted as a test): `MCU MBOX - Valid user parameters only enabled with VALID parameter set` (plan item only)

   Verified: No test writes the register while it is locked or unlocked and checks the result. The searches found only configuration writes.

   - `src/integration/test_suites/libs/caliptra_ss_lib/caliptra_ss_lib.c:501-511`: `lsu_write_32(SOC_MCI_TOP_MCI_REG_MBOX0_VALID_AXI_USER_0 + MCU_MBOX_AXI_CFG_STRIDE*mbox_num, axi_user_id[0]);` — The library sets the users and then the locks as configuration. It does not attempt a write after lock or check read-only behaviour.

2. **Requirement** `cr_da0583be4faf`: The AXI_USER register is read-only once it is locked by AXI_USER_LOCK.

   - `src/mci/rtl/mci_reg.rdl:823`: `Read-Only once locked by AXI_USER_LOCK.`

   Nearest evidence (not counted as a test): `MCU MBOX - Valid user parameters only enabled with VALID parameter set` (plan item only)

   Verified: No test checks that the register is read-only after lock. The tests only read it back.

   - `src/integration/test_suites/smoke_test_mcu_hitless/smoke_test_mcu_hitless.c:41-43`: `if (lsu_read_32(SOC_MCI_TOP_MCI_REG_MBOX0_AXI_USER_LOCK_4)) {` — The test only reads the lock and the user value. It makes no write-after-lock attempt and has no failing check.

Searched:

- tests referencing VALID_AXI_USER / AXI_USER_LOCK in `src/integration/test_suites`: Only config setup in caliptra_ss_lib.c, a read in the hitless smoke test, and register tables in mci.c. The mbox axi_param test mentions the user values only in a comment.
- locked-write-ignored check in `src/integration/test_suites and src/integration/testbench`: None found.

Looked at and ruled out:

- `src/integration/test_suites/libs/mci/mci.c:273-277`: These are generic register-table entries. Nothing there writes after lock and checks that the value is unchanged.

5.21 No test found: 5 requirements in mci_reg.rdl > MBOX1_VALID_AXI_USER (and 1 more section)

`f_9ef75aff782b` · severity medium

1. **Requirement** `cr_ab2dcc637223`: mci_reg.MBOX1_VALID_AXI_USER[].AXI_USER: software writes take effect only when true is deasserted.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > MBOX1_VALID_AXI_USER

   - `src/mci/rtl/mci_reg.rdl:841`: `swwel`

   Verified: No test writes MBOX1_VALID_AXI_USER after the lock is set and checks that the value is unchanged.

   - `src/integration/test_suites/libs/caliptra_ss_lib/caliptra_ss_lib.c:501-507`: `lsu_write_32(SOC_MCI_TOP_MCI_REG_MBOX0_VALID_AXI_USER_0 + MCU_MBOX_AXI_CFG_STRIDE*mbox_num, axi_user_id[0]);` — The configure helper only writes the valid users and then sets the lock. There is no post-lock write or readback check.

2. **Requirement** `cr_7ad2d7a15e2b`: mci_reg.MBOX1_AXI_USER_LOCK[].LOCK: software writes take effect only when true is deasserted.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > MBOX1_AXI_USER_LOCK

   - `src/mci/rtl/mci_reg.rdl:851`: `swwel`

   Verified: No test writes the lock register after it is locked and checks the result.

3. **Requirement** `cr_8e9d00377b71`: The associated valid-user register is valid only after it has been locked by this bit.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > MBOX1_AXI_USER_LOCK

   - `src/mci/rtl/mci_reg.rdl:848`: `Associated MBOX0_VALID_AXI_USER register is only valid once locked by this bit.`

   Verified: No test checks that the valid-user register is only effective once locked.

4. **Requirement** `cr_a3139c9fee5f`: AXI access type of the lock register is RW1-S.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > MBOX1_AXI_USER_LOCK

   - `src/mci/rtl/mci_reg.rdl:849`: `AXI Access: RW1-S`

   Verified: The helper writes 1 to set the lock. Nothing checks RW1-S behaviour, such as writing 0 having no effect.

5. **Requirement** `cr_a66dd7732b98`: The lock register is read-only once it has been locked.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > MBOX1_AXI_USER_LOCK

   - `src/mci/rtl/mci_reg.rdl:850`: `Read-Only once locked.`

   Verified: No test checks that the lock register is read-only once locked.

Searched:

- MBOX1_VALID_AXI_USER / AXI_USER_LOCK usage across src in `repo`: Only address helpers, the reset-value and mask tables in mci.c, and the generic configure helper. No lock-then-write check.
- callers of the address helpers in `src`: Only mcu_mbox_soc_agent_write_fw_image.svh, and it uses mbox 0.
- tests checking RW1-S or read-only-after-lock behaviour in `test_suites`: None found.

Looked at and ruled out:

- `src/integration/test_suites/libs/mci/mci.c:1935-1953`: Mask entries for the register-access test. They don't test lock-then-write semantics.

5.22 No test found: 5 requirements in mci_reg.rdl > error_mbox0_ecc_unc_intr_count_incr_r (part 1/2) (and 3 more sections)

`f_623adc8d79e6` · severity medium

1. **Requirement** `cr_948e0cfb6d3a`: mci_reg.intr_block_rf.error_mbox0_ecc_unc_intr_count_incr_r.pulse: the field is a counter.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > error_mbox0_ecc_unc_intr_count_incr_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1555`: `counter`

   Verified: The counter attribute is untested. No check in the repo confirms the field behaves as a counter.

   - `src/mci/rtl/mci_reg.rdl:1548-1549` — The behaviour the items describe.

     ```
     This is implemented as a down-counter (1-bit) that will
                     decrement immediately on being set - resulting in a pulse
     ```

   - `src/mci/coverage/mci_top_cov_if.sv:4810`: `INTR_BLOCK_RF_ERROR_MBOX0_ECC_UNC_INTR_COUNT_INCR_R_cp : coverpoint i_mci_reg_top.i_mci_reg.field_storage.intr_block_rf.error_mbox0_ecc_unc_intr_count_incr_r;` — A coverpoint only. It checks nothing.

2. **Requirement** `cr_96b7a083f521`: The 1-bit down-counter decrements immediately after being set, producing a pulse.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_ecc_cor_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1548-1549`: `This is implemented as a down-counter (1-bit) that will decrement immediately on being set - resulting in a pulse`

   Verified: No test reads or checks the incrementor pulse. This one is for mbox0 ecc_cor.

   - `src/mci/rtl/mci_reg.sv:22281`: `end else if(hwif_in.intr_block_rf.notif0_internal_intr_r.notif_mbox0_ecc_cor_sts.hwset) begin // HW Set` — The incrementor pulse is set only by the hardware hwset input. The software trigger bit (trig) does not drive it.
   - `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:64`: `lsu_write_32(SOC_MCI_TOP_MCI_REG_INTR_BLOCK_RF_ERROR0_INTR_TRIG_R, MCI_REG_INTR_BLOCK_RF_ERROR0_INTR_TRIG_R_ERROR_MBOX0_ECC_UNC_TRIG_MASK );` — Near miss: the stimulus is a software trigger write, not a hardware event on the hwset input.

3. **Requirement** `cr_97a9307feeea`: mci_reg.intr_block_rf.notif_mbox0_target_done_intr_count_incr_r.pulse: the field is a counter.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_target_done_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1555`: `counter`

   Verified: No test checks the counter attribute of the mbox0 target_done incrementor.

4. **Requirement** `cr_991e89c92ef8`: The 1-bit down-counter decrements immediately on being set, producing a pulse.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_ecc_cor_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1548-1549`: `This is implemented as a down-counter (1-bit) that will decrement immediately on being set - resulting in a pulse`

   Verified: No test checks the 1-bit down-counter pulse. This one is for mbox1 ecc_cor.

5. **Requirement** `cr_9b97309f0dd7`: The same input that sets the interrupt event also makes the pulse signal assert for 1 clock cycle.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_target_done_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1544-1546`: `Hardware. The same input signal that causes an interrupt event to be set (sticky) also causes this signal to pulse for 1 clock cycle, resulting in the event counter`

   Verified: The only counter-reading test uses software triggers rather than the hwset event input, so the single-cycle pulse is not checked.

   - `src/mci/rtl/mci_reg_top.sv:833-834`: `always_comb mci_reg_hwif_in.intr_block_rf.error0_internal_intr_r.error_mbox0_ecc_unc_sts.hwset = mbox0_sram_double_ecc_error;` — The pulse is driven by real mailbox ECC error events. The tests found never raise them while checking the counters.

Searched:

- count_incr names in tests, sequences and libs in `whole repo`: Only generated RTL, the UVM register model, coverage, SoC address map headers and the register masks in `libs/mci/mci.c`.

- mbox ECC and target_done INTR_COUNT_R reads in tests in `src/integration/test_suites`: Only `mcu_intr_handler.c`, which sums the counters after software trigger writes.
- how the pulse is driven in `src/mci/rtl/mci_reg.sv and mci_reg_top.sv`: The pulse is set only by the status hwset input from real hardware events, not by the trigger bit.

Looked at and ruled out:

- `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:159-204`: It compares a firmware count against the summed hardware count registers, but its stimulus is software triggers, which do not reach the incrementor pulse. It does not check the pulse itself or the 1-bit down-counter behaviour, so it cannot confirm the items.
- `src/integration/test_suites/libs/mci/mci.c:2088-2098`: It only adds register mask entries for the count_incr registers. It does not check pulse behaviour.

5.23 No test found: 2 requirements in mci_reg.rdl > error_mbox0_ecc_unc_intr_count_incr_r (part 1/2) (and 3 more sections)

`f_cebb4101f089` · severity medium

1. **Requirement** `cr_9f969e4a3656`: mci_reg.intr_block_rf.error_mbox0_ecc_unc_intr_count_incr_r.pulse: software writes take effect only when false is deasserted.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > error_mbox0_ecc_unc_intr_count_incr_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1557`: `swwel = false`

   Verified: No test writes the pulse field to check swwel gating.

2. **Requirement** `cr_abec8d18a27b`: mci_reg.intr_block_rf.notif_cptra_mbox_cmd_avail_intr_count_incr_r.pulse: software writes take effect only when false is deasserted.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_cptra_mbox_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1557`: `swwel = false`

   Verified: No test checks swwel behaviour.

Items 2, 3, 4 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- count_incr / intr_count references outside RTL in `repo`: Only mci_top_cov_if.sv (coverpoints), mcu_intr_handler.c, docs
- firmware check of event counters in `mcu_intr_handler.c`: Counts compared to expected at lines 201 and 256, including the MBOX0/MBOX1 ECC_COR and CPTRA_MBOX_CMD_AVAIL counters.

Looked at and ruled out:

- `src/mci/coverage/mci_top_cov_if.sv:4800-4810`: Coverpoints only; no check.

5.24 No test found: 5 requirements in mci_reg.rdl > error_mbox1_ecc_unc_intr_count_incr_r (part 1/2) (and 3 more sections)

`f_5659aeaf1c1a` · severity medium

1. **Requirement** `cr_e56ce5f8f0a1`: mci_reg.intr_block_rf.error_mbox1_ecc_unc_intr_count_incr_r.pulse: software writes take effect only when false is deasserted.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > error_mbox1_ecc_unc_intr_count_incr_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1557`: `swwel = false`

   Verified: No test checks swwel behavior of the pulse field.

   - `src/integration/test_suites/libs/mci/mci.c:2092-2094` — Only a bit-mask table entry (used for generic register tests); it checks no pulse or decrement behavior. The notif mbox0/mbox1 count_incr registers have no entries in the matches.

     ```
     // ERROR_MBOX0_ECC_UNC_INTR_COUNT_INCR_R
     add_mask_entry(SOC_MCI_TOP_MCI_REG_INTR_BLOCK_RF_ERROR_MBOX0_ECC_UNC_INTR_COUNT_INCR_R,
     ```

   - `src/mci/coverage/mci_top_cov_if.sv:4810`: `INTR_BLOCK_RF_ERROR_MBOX0_ECC_UNC_INTR_COUNT_INCR_R_cp : coverpoint` — Coverage only; no check that can fail.

2. **Requirement** `cr_e5847e39e3ea`: mci_reg.intr_block_rf.error_mbox0_ecc_unc_intr_count_incr_r.pulse: the counter decrements by 1.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > error_mbox0_ecc_unc_intr_count_incr_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1554`: `decrvalue = 1`

   Verified: No test checks the decrement by 1.

3. **Requirement** `cr_e7151b4d15af`: mci_reg.intr_block_rf.notif_mbox0_soc_req_lock_intr_count_incr_r.pulse: the field is a counter.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_soc_req_lock_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1555`: `counter`

   Verified: No test checks the notif mbox0 soc_req_lock count_incr field as a counter.

4. **Requirement** `cr_e729c05476cf`: The incrementor is a 1-bit down-counter that decrements immediately on being set, producing a pulse.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_soc_req_lock_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1548-1549`: `This is implemented as a down-counter (1-bit) that will decrement immediately on being set - resulting in a pulse`

   Verified: No test checks the 1-bit down-counter pulse behavior.

5. **Requirement** `cr_e74c6fb973fe`: mci_reg.intr_block_rf.notif_mbox1_ecc_cor_intr_count_incr_r.pulse: the field is a counter.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_ecc_cor_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1555`: `counter`

   Verified: No test checks the notif mbox1 ecc_cor count_incr field as a counter.

Searched:

- item cr_e56ce5f8f0a1: error_mbox1_ecc_unc count_incr swwel in `whole repo`: only mci.c mask entry and a coverpoint; no write-enable test
- item cr_e5847e39e3ea: error_mbox0_ecc_unc count_incr decrement by 1 in `whole repo`: only mci.c mask entry and a coverpoint; no decrement check
- item cr_e7151b4d15af: notif_mbox0_soc_req_lock count_incr counter in `whole repo`: no test or non-generated file references it; only coverage and generated register files
- item cr_e729c05476cf: 1-bit down-counter pulse in `whole repo`: no test checks the pulse behavior
- item cr_e74c6fb973fe: notif_mbox1_ecc_cor count_incr counter in `whole repo`: no test or non-generated file references it; only coverage and generated register files

---

5.25 No test found: 4 requirements in mci_reg.rdl > error_mbox1_ecc_unc_intr_count_incr_r (part 1/2) (and 3 more sections)

`f_87f1e04df40b` · severity medium

1. **Requirement** `cr_8c05acc52333`: The interrupt event counter saturates and does not overflow.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > error_mbox1_ecc_unc_intr_count_incr_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test saturates a counter.

2. **Requirement** `cr_8f7eafce5cc8`: The event counter does not overflow and saturates at its maximum value.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_ecc_cor_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No saturation test for ecc_cor.

3. **Requirement** `cr_909044a1f8f3`: The pulse field is a 1-bit down-counter that decrements immediately after being set, producing a pulse.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1548-1549`: `This is implemented as a down-counter (1-bit) that will decrement immediately on being set - resulting in a pulse`

   Verified: The pulse field is never checked.

4. **Requirement** `cr_92e293d86fd1`: The event counter increments on observing a rising edge of the hardware interrupt event input.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_soc_req_lock_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1542-1544`: `Trigger the event counter to increment based on observing the rising edge of an interrupt event input from the Hardware.`

   Verified: soc_req_lock is never triggered in the only counter test.

   - `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:201-204`: `if (mci_error_intr_count != mci_error_intr_count_hw) {` — The only check on counters: it compares the aggregate sum with a software tally, using software INTR_TRIG writes and not a hardware input edge. No saturation check, and soc_req_lock is not triggered.

Item 2 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- Tests referencing INTR_COUNT_R / COUNT_INCR_R registers in `src/integration, src/mci`: Only mcu_intr_handler.c reads the counters (summed compare). mci.c only lists the registers as sticky in a reset table. The rest is generated headers and coverage.
- NOTIF_SOC_REQ_LOCK in mcu_intr_handler in `mcu_intr_handler.c`: Not present, so soc_req_lock is never triggered or counted.

Looked at and ruled out:

- `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:206-219`: Sums MBOX1_CMD_AVAIL and ECC_COR counts with others, using software triggers; it does not check saturation, the rising edge of a hardware input, or the pulse field.

5.26 No test found: 3 requirements in mci_reg.rdl > error_mbox1_ecc_unc_intr_count_incr_r (part 1/2) (and 4 more sections)

`f_a40e976ebc6e` · severity medium

2. **Requirement** `cr_1e422b850220`: The interrupt event counter saturates and does not overflow.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_ecc_cor_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test drives a counter to saturation.

3. **Requirement** `cr_22a6d4baab20`: mci_reg.intr_block_rf.notif_mbox1_cmd_avail_intr_count_incr_r.pulse: software writes take effect only when false is deasserted.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1557`: `swwel = false`

   Verified: No test checks swwel behaviour on the pulse field.

4. **Requirement** `cr_242db8343387`: mci_reg.intr_block_rf.notif_mbox1_target_done_intr_count_incr_r.pulse: the counter decrements by 1.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_target_done_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1554`: `decrvalue = 1`

   Verified: No test checks the decrvalue=1 mechanics. The summed check is only indirect.

Items 1, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- intr_count / count_incr / INTR_COUNT_R usage in tests and libs in `src/integration/test_suites, tools`: Only mcu_intr_handler.c reads the counters. Other 'intr_count' matches are unrelated firmware variables.
- saturation or 0xFFFFFFFF in mcu_intr_handler in `src/integration/test_suites/mcu_intr_handler`: Only mtime constants, nothing about interrupt counters.
- regression inclusion in `master test list and yml files`: mcu_intr_handler is listed in the nightly regression.

Looked at and ruled out:

- `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:208-257`: The check is summed over all counters and uses software trigger writes. It does not isolate the pulse width, decrvalue, swwel or saturation, so it does not verify those items.

5.27 No test found: 2 requirements in mci_reg.rdl > notif_cptra_mbox_cmd_avail_intr_count_incr_r (part 1/3) (and 3 more sections)

`f_8e312cf1f5e1` · severity medium

1. **Requirement** `cr_ac50ef86787c`: The event counter increments on the rising edge of the hardware interrupt event input.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_cptra_mbox_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1542-1544`: `Trigger the event counter to increment based on observing the rising edge of an interrupt event input from the Hardware.`

   Verified: No test drives the hardware event input and checks the edge. Only software TRIG writes are used.

   - `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:215`: `lsu_read_32(SOC_MCI_TOP_MCI_REG_INTR_BLOCK_RF_NOTIF_CPTRA_MBOX_CMD_AVAIL_INTR_COUNT_R ) +` — The test reads the count registers, never the *_COUNT_INCR_R registers. Events come from software TRIG writes.

2. **Requirement** `cr_b17476f90b95`: The counter increment is triggered by the rising edge of the hardware interrupt event input.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > error_mbox0_ecc_unc_intr_count_incr_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1542-1544`: `Trigger the event counter to increment based on observing the rising edge of an interrupt event input from the Hardware.`

   Verified: Same as the first item. The hardware-edge trigger is untested, because only software TRIG stimulus exists.

   - `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:201-204`: `if (mci_error_intr_count != mci_error_intr_count_hw) {` — Same aggregate check for the error counters, including the ERROR_MBOX0_ECC_UNC count.

Items 2, 3, 4 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- INTR_COUNT_INCR and count_incr usage in tests in `whole repo`: Only address maps, mci_reg.sv, mci_reg_uvm.sv and the coverage interface. No test uses them.
- INTR_COUNT_R usage in test suites in `src/integration/test_suites`: Only mcu_intr_handler.c checks them, via an aggregate sum against a firmware counter.
- Whether mcu_intr_handler is in a regression in `regression yml files`: It is in the Nightly Directed and Strict regressions.

Looked at and ruled out:

- `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:107-111`: Software TRIG writes do exercise the count increment. The check is an aggregate sum, and the stimulus is a software trigger rather than a hardware input rising edge.

5.28 No test found: 5 requirements in mci_reg.rdl > notif_mbox0_cmd_avail_intr_count_incr_r (part 1/3) (and 3 more sections)

`f_c8297cebd7f5` · severity medium

1. **Requirement** `cr_e8d0b114da23`: The same input that sets the interrupt event also makes the increment signal pulse for 1 clock cycle.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1544-1546`: `Hardware. The same input signal that causes an interrupt event to be set (sticky) also causes this signal to pulse for 1 clock cycle, resulting in the event counter`

   Verified: No test checks the 1-cycle pulse from the same input that sets the event.

   - `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256-259`: `if (mci_notif_intr_count != mci_notif_intr_count_hw) {` — Only counter check found: it compares an aggregate sum of ~40 notification counters, including the mbox0/mbox1 cmd_avail, target_done and ecc_cor counters, against an ISR count. It never reads the incr_r pulse register and never checks a single counter or pulse width.

2. **Requirement** `cr_e904756192a7`: The incrementor signal pulses for 1 clock cycle when the interrupt event input that sets the sticky interrupt event occurs.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_target_done_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1544-1546`: `The same input signal that causes an interrupt event to be set (sticky) also causes this signal to pulse for 1 clock cycle`

Verified: No test checks the pulse width for mbox1 target_done.

3. **Requirement** `cr_ead524d0131a`: The incrementor signal pulses for 1 clock cycle on the same input signal that sets the interrupt event.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1544-1546`: `The same input signal that causes an interrupt event to be set (sticky) also causes this signal to pulse for 1 clock cycle`

   Verified: No test checks the pulse width for mbox1 cmd_avail.

4. **Requirement** `cr_ee570505c93a`: The pulse field mirrors the occurrence of the interrupt event.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_ecc_cor_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1557`: `Pulse mirrors interrupt event occurrence`

   Verified: No test reads the pulse field or checks it mirrors the event, for mbox0 ecc_cor.

5. **Requirement** `cr_f0d7f62c747a`: The event counter increments on the rising edge of the hardware interrupt event input.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1542-1543`: `Trigger the event counter to increment based on observing the rising edge of an interrupt event input from the`

   Verified: The only check is aggregate, and it is trigger-driven. It does not check a per-counter increment on the rising edge of the hardware input.

   - `src/mci/rtl/mci_reg.rdl:2498-2500`: `notif_mbox0_cmd_avail_intr_count_incr_r.pulse -> hwset = notif0_internal_intr_r.notif_mbox0_cmd_avail_sts -> hwset;` — RTL wires the pulse to both the hardware set and the software trigger, so a trigger-driven test would not exercise the hardware-input path described by the items.

Searched:

- Per-counter names (notif_mbox0/1 cmd_avail/target_done/ecc_cor intr_count) in tests, assertions and sequences in `whole repo, excluding generated mci_reg*.sv/h and cov_if`: Only the rdl and tools/scripts/gen_mci_reg_covergroups.py (coverage generation). No test or assertion reads them individually.
- intr_count_incr usage in tests in `src/integration/test_suites`: No pulse-register checks.
- Aggregate notification counter check in `mcu_intr_handler.c lines 208-259`: Sum comparison only, driven by trigger registers; near miss.

Looked at and ruled out:

- `src/mci/coverage/mci_top_cov_if.sv:3960`: Coverage points on counter values only. They check nothing.

5.29 No test found: 4 requirements in mci_reg.rdl > notif_mbox0_cmd_avail_intr_count_incr_r (part 1/3) (and 3 more sections)

`f_d6f1d21d50ca` · severity medium

1. **Requirement** `cr_4706175d6d92`: mci_reg.intr_block_rf.notif_mbox0_cmd_avail_intr_count_incr_r.pulse: the counter decrements by 1.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1554`: `decrvalue = 1`

   Verified: No test writes any incr_r pulse register to decrement a counter.

2. **Requirement** `cr_4868f6c38d1f`: The interrupt event counter does not overflow; it saturates.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_cptra_mbox_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test drives a counter to its maximum value to check saturation.

3. **Requirement** `cr_48e9dbdcfdb2`: The interrupt event counter saturates and does not overflow.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No saturation test found.

4. **Requirement** `cr_4b697972bda7`: The interrupt event counter does not overflow; it saturates.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No saturation test found.

Item 4 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- tests referencing *_INTR_COUNT_INCR_R, decrement, or saturation of interrupt counters in `src/integration/test_suites and the rest of the repo, excluding mci_reg*`: Nothing. The only INCR_R hits are in generated register code and mci_top_cov_if.sv coverpoints, which are not checks.
- INTR_COUNT usage in tests in `src/integration/test_suites`: mcu_intr_handler sums the counters and compares them to the trigger count. mcu_isr clears two counters. mci.c lists the counters in a register reset/sticky table.
- mcu_intr_handler in regression lists in `src/integration/stimulus/*.yml`: Included in both the Nightly Directed and Nightly Directed Strict regressions.

Looked at and ruled out:

- `src/mci/coverage/mci_top_cov_if.sv:4800`: Coverpoint only, with no check.
- `src/integration/test_suites/libs/mci/mci.c:579`: A register reset/sticky table, not a check of increment, decrement or saturation behaviour.

5.30 No test found: 1 requirement in mci_reg.rdl > notif_mbox0_ecc_cor_intr_count_incr_r (part 1/3) (and 2 more sections)

`f_c879f7a38c02` · severity medium

2. **Requirement** `cr_0c3afed8d218`: mci_reg.intr_block_rf.notif_mbox0_cmd_avail_intr_count_incr_r.pulse: software writes take effect only when false is deasserted.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1557`: `swwel = false`

   Verified: No test attempts software writes to the incr pulse field.

Items 1, 3, 4, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- tests referencing the three notif incr registers in `repo-wide`: Only the RTL, the UVM model and the coverage interface mention them; no test accesses *_count_incr_r directly.
- tests exercising the notif counters in `src/integration/test_suites`: mcu_intr_handler, which is listed in the nightly regression.

Looked at and ruled out:

- `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:206-220`: The check is an aggregate sum over all notif counters. It does not check each of the three registers individually, and it does not check the counter, decrvalue or swwel attributes.

5.31 No test found: 5 requirements in mci_reg.rdl > notif_mbox0_ecc_cor_intr_count_incr_r (part 1/3) (and 3 more sections)

`f_4d8a1f7379dd` · severity medium

1. **Requirement** `cr_b74f1ee3c9b5`: mci_reg.intr_block_rf.notif_mbox0_ecc_cor_intr_count_incr_r.pulse: software writes take effect only when false is deasserted.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_ecc_cor_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1557`: `swwel = false`

   Verified: No test found that checks the swwel behavior.

   - `src/integration/test_suites/libs/mci/mci.c:2093-2094` — The pulse field is added as a mask entry, so it is excluded from comparison rather than checked.

```
add_mask_entry(SOC_MCI_TOP_MCI_REG_INTR_BLOCK_RF_ERROR_MBOX0_ECC_UNC_INTR_COUNT_INCR_R,
               MCI_REG_INTR_BLOCK_RF_ERROR_MBOX0_ECC_UNC_INTR_COUNT_INCR_R_PULSE_MASK);
```

2. **Requirement** `cr_b9707c71a06b`: mci_reg.intr_block_rf.notif_mbox1_soc_req_lock_intr_count_incr_r.pulse: the field is a counter.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_soc_req_lock_intr_count_incr_r (part 1/3)
   - `src/mci/rtl/mci_reg.rdl:1555`: `counter`

   Verified: No test found that checks the counter behavior.

3. **Requirement** `cr_bd3ec8885ef1`: mci_reg.intr_block_rf.notif_mbox1_target_done_intr_count_incr_r.pulse: software writes take effect only when false is deasserted.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_target_done_intr_count_incr_r (part 1/3)
   - `src/mci/rtl/mci_reg.rdl:1557`: `swwel = false`

   Verified: No test found that checks the swwel behavior.

4. **Requirement** `cr_bfdfd2d94f4b`: mci_reg.intr_block_rf.notif_mbox1_target_done_intr_count_incr_r.pulse: the field is a counter.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_target_done_intr_count_incr_r (part 1/3)
   - `src/mci/rtl/mci_reg.rdl:1555`: `counter`

   Verified: No test found that checks the counter behavior.

5. **Requirement** `cr_c120cfb2596c`: mci_reg.intr_block_rf.notif_cptra_mbox_cmd_avail_intr_count_incr_r.pulse: the counter decrements by 1.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_cptra_mbox_cmd_avail_intr_count_incr_r (part 1/3)
   - `src/mci/rtl/mci_reg.rdl:1554`: `decrvalue = 1`

   Verified: No test found that checks the decrement value.

Searched:

- count_incr references outside rdl/sv in `whole repo`: Only the coverage interface, generated register files, a coverage generator script and mci.c mask entries.
- mci_top_cov_if.sv count_incr in `src/mci/coverage`: Coverpoints only. They sample the field storage and check nothing.
- notif_mbox0_ecc_cor swwel item: tests/checks for this register in `whole repo`: No test or check beyond the coverage point and mask entry.
- notif_mbox1_soc_req_lock counter item in `whole repo`: No test or check found.
- notif_mbox1_target_done swwel and counter items in `whole repo`: No test or check found.
- notif_cptra_mbox_cmd_avail decrvalue item in `whole repo`: No test or check found.

Looked at and ruled out:

- `src/mci/coverage/mci_top_cov_if.sv:4800`: Coverpoint that samples a value but has no check.

5.32 No test found: 4 requirements in mci_reg.rdl > notif_mbox0_ecc_cor_intr_count_incr_r (part 1/3) (and 3 more sections)

`f_bb99ac035c0a` · severity medium

1. **Requirement** `cr_77c7ae0b431e`: A rising edge of the hardware interrupt event input triggers the event counter to increment.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_ecc_cor_intr_count_incr_r (part 1/3)
   - `src/mci/rtl/mci_reg.rdl:1542-1544`: `Trigger the event counter to increment based on observing the rising edge of an interrupt event input from the Hardware.`

   Verified: No test drives the hardware event input. The only stimulus is the software trigger register.

   - `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:201-204`: `if (mci_error_intr_count != mci_error_intr_count_hw) {` — Aggregate check: firmware count must equal the sum of hardware counts. Software-trigger stimulus only, one event at a time.

2. **Requirement** `cr_81ffd1fa635f`: The incrementor signal pulses for 1 clock cycle on the same input signal that sets the interrupt event.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_ecc_cor_intr_count_incr_r (part 1/3)

- `src/mci/rtl/mci_reg.rdl:1544-1546`: `The same input signal that causes an interrupt event to be set (sticky) also causes this signal to pulse for 1 clock cycle`

Verified: No test checks the incr pulse width. Searches found no such test.

3. **Requirement** `cr_84e8df210e83`: The interrupt event counter does not overflow; it saturates.
Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_target_done_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test drives a counter to saturation.

   - `src/mci/rtl/mci_reg.sv:17016-17017`: `field_combo.intr_block_rf.error_mbox0_ecc_unc_intr_count_r.cnt.incrthreshold = (field_storage.intr_block_rf.error_mbox0_ecc_unc_intr_count_r.cnt.value >= 32'hffffffff);` — RTL implements saturation, but no test checks it.

4. **Requirement** `cr_8a0bed11bd11`: The interrupt event counter saturates and does not overflow.
Section: src/mci/rtl/mci_reg.rdl > mci_reg > error_mbox0_ecc_unc_intr_count_incr_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test drives a counter to saturation.

Item 2 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- Tests that read or check the intr_count and intr_count_incr registers in `whole repo, including src/integration/test_suites`: Only mcu_intr_handler.c compares counts. libs/mci/mci.c only lists the registers (register reset/access table). mci_top_cov_if.sv has coverpoints only.
- Saturation or overflow test (counter at 0xffffffff) in `whole repo`: None.
- Check of the incr pulse width, or of a hardware-input rising edge in `whole repo`: None. The only stimulus is the software trigger register.

Looked at and ruled out:

- `src/mci/coverage/mci_top_cov_if.sv:3970`: Coverage coverpoint only, with no check.

5.33 No test found: 5 requirements in mci_reg.rdl > notif_mbox0_ecc_cor_intr_count_r (part 1/2) (and 3 more sections)

`f_9aa072fb9e0b` · severity medium

1. **Requirement** `cr_22dc2380458a`: mci_reg.intr_block_rf.notif_mbox0_ecc_cor_intr_count_r.cnt: the counter increments by 1.
Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_ecc_cor_intr_count_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1569`: `incrvalue = 1`

   Verified: No test reads notif_mbox0_ecc_cor_intr_count_r or checks that it increments by 1.

   - `src/integration/test_suites/libs/mcu_isr/mcu_isr.c:359-360`: `mci_reg[MCI_REG_INTR_BLOCK_RF_NOTIF_DEBUG_LOCKED_INTR_COUNT_R /sizeof(uint32_t)] = 0;` — The only intr_count register access in the test libraries. It clears two other counters (debug-locked and scan-mode) and checks nothing.

2. **Requirement** `cr_363bc0793fc3`: mci_reg.intr_block_rf.notif_mbox0_target_done_intr_count_r.cnt: the field is a counter.
Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_target_done_intr_count_r

   - `src/mci/rtl/mci_reg.rdl:1571`: `counter`

   Verified: No test checks that notif_mbox0_target_done_intr_count_r behaves as a counter.

3. **Requirement** `cr_3d87302757c0`: mci_reg.intr_block_rf.notif_cptra_mbox_cmd_avail_intr_count_r.cnt: the counter increments by 1.
Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_cptra_mbox_cmd_avail_intr_count_r

   - `src/mci/rtl/mci_reg.rdl:1569`: `incrvalue = 1`

   Verified: No test checks the increment of notif_cptra_mbox_cmd_avail_intr_count_r.

4. **Requirement** `cr_3f0d776dda4a`: The counter does not overflow; it saturates at its maximum value.
Section: src/mci/rtl/mci_reg.rdl > mci_reg > error_mbox1_ecc_unc_intr_count_r

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

Verified: No test drives error_mbox1_ecc_unc_intr_count_r to its maximum to check saturation.

5. **Requirement** `cr_43b0313d0b42`: The counter saturates at its maximum value and does not overflow.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_target_done_intr_count_r

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test checks saturation of notif_mbox0_target_done_intr_count_r.

Searched:

- The four counter names (notif_mbox0_ecc_cor, notif_mbox0_target_done, notif_cptra_mbox_cmd_avail, error_mbox1_ecc_unc) with intr_count, in src outside mci_reg and the RDL in `src`: Only address-map definitions, mci_reg.sv, mci_reg_uvm.sv and mci_top_cov_if.sv. No test uses them.
- INTR_COUNT_R, INTR_COUNT_INCR_R and intr_count in test suites in `src/integration/test_suites`: Test files only match the software variable `intr_count`, which they increment themselves. mcu_isr.c clears two unrelated counters.
- intr_count_r in mcu_intr_handler.c in `src/integration/test_suites/mcu_intr_handler`: No matches.

Looked at and ruled out:

- `src/integration/test_suites/libs/mcu_isr/mcu_isr.c:855-856`: This increments a software variable named intr_count, not the hardware counter register.
- `src/mci/coverage/mci_top_cov_if.sv:1`: This is a coverage interface. It samples the counters but does not check them, so it cannot fail a test.

5.34 No test found: 1 requirement in mci_reg.rdl > notif_mbox0_ecc_cor_intr_count_r (part 1/2) (and 4 more sections)

`f_340278e79aab` · severity medium

1. **Requirement** `cr_71500986cef1`: The interrupt event counter saturates and does not overflow.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_ecc_cor_intr_count_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test drives a counter to saturation; the finding stands for this item.

Items 2, 3, 4, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- uses of notif *_INTR_COUNT_R registers in tests in `src/integration/test_suites`: mcu_intr_handler.c reads and sums them and compares them with the fw count; libs/mci/mci.c lists them in a register table
- saturation or overflow test of the counters in `whole repo`: none; only the RDL, generated RTL and UVM register model mention incrsaturate

Looked at and ruled out:

- `src/integration/test_suites/libs/mci/mci.c:582-592`: Register table only (reset and sticky checks), not an increment check.

5.35 No test found: 5 requirements in mci_reg.rdl > notif_mbox0_soc_req_lock_intr_count_incr_r (part 1/3) (and 3 more sections)

`f_f9ec936f1501` · severity medium

1. **Requirement** `cr_3d4f68eca3da`: The incrementor signal pulses for 1 clock cycle on the same input that sets the interrupt event.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_soc_req_lock_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1544-1546`: `The same input signal that causes an interrupt event to be set (sticky) also causes this signal to pulse for 1 clock cycle`

   Verified: No test checks that the incr signal pulses for 1 clock cycle on the same input that sets the event.

   - `src/integration/test_suites/libs/mci/mci.c:2088`: `// ERROR_INTERNAL_INTR_COUNT_INCR_R` — Only a pulse mask entry for generic register access testing. It does not check pulse width or saturation.

2. **Requirement** `cr_3e59aca31cd9`: The 1-bit down-counter decrements immediately on being set, producing a pulse.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_target_done_intr_count_incr_r (part 1/3)

- `src/mci/rtl/mci_reg.rdl:1548-1549`: `This is implemented as a down-counter (1-bit) that will decrement immediately on being set - resulting in a pulse`

Verified: No test checks the 1-bit down-counter or the immediate decrement that produces the pulse.

3. **Requirement** `cr_3ecbe358fb05`: The interrupt event counter saturates and does not overflow.
Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_soc_req_lock_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test drives the counter to its maximum or checks that it saturates.

   - `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256-257`: `if (mci_notif_intr_count != mci_notif_intr_count_hw) {` — Only the sum of notif counters is compared with the number of TRIG writes. This covers the increment-by-1 item only partly, because events come from TRIG and the check is on the sum.

4. **Requirement** `cr_416033921743`: mci_reg.intr_block_rf.notif_mbox0_cmd_avail_intr_count_incr_r.pulse: the field is a counter.
Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1555`: `counter`

   Verified: The 'counter' property of the incr_r pulse field is not checked by any test.

5. **Requirement** `cr_4400c784ffb0`: The event counter increments by 1 for every interrupt event.
Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_soc_req_lock_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1546-1547`: `resulting in the event counter incrementing by 1 for every interrupt event.`

   Verified: Only partly covered. mcu_intr_handler checks that the summed notif counters equal the number of TRIG-triggered events, but not per register and not for real input events. The item stays unverified.

Searched:

- intr_count_incr / INTR_COUNT_INCR in tests, tcl, py, yml, sv in `whole repo`: Only RTL, UVM reg model, coverage interface, address map defines and the mci.c reg list/mask.
- notif intr count usage in tests in `src/integration/test_suites`: mcu_intr_handler sums COUNT_R values and compares with TRIG writes. It does not read INCR_R, check pulse width, saturation or the down-counter.
- mcu_intr_handler in regression lists in `src/integration/stimulus`: Included in the nightly directed regressions.

Looked at and ruled out:

- `src/mci/coverage/mci_top_cov_if.sv:3960`: Coverpoints on counter values only, with no pass/fail check.

5.36 No test found: 2 requirements in mci_reg.rdl > notif_mbox0_soc_req_lock_intr_count_incr_r (part 1/3) (and 4 more sections)

`f_b253afba5a90` · severity medium

4. **Requirement** `cr_72ef5cfc7830`: mci_reg.intr_block_rf.notif_mbox1_ecc_cor_intr_count_incr_r.pulse: software writes take effect only when false is deasserted.
Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_ecc_cor_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1557`: `swwel = false`

   Verified: No test writes the pulse field, and none checks that swwel=false is honoured.

5. **Requirement** `cr_73c6e4152180`: The same input signal that sets the interrupt event also makes the pulse signal pulse for 1 clock cycle.
Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_ecc_cor_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1544-1546`: `The same input signal that causes an interrupt event to be set (sticky) also causes this signal to pulse for 1 clock cycle`

   Verified: Only software trigger bits are used. The real hardware input that sets the event and the pulse is not driven and checked for mbox0_ecc_cor.

   - `src/mci/rtl/mci_reg.sv:22276`: `next_c = field_storage.intr_block_rf.notif_mbox0_ecc_cor_intr_count_incr_r.pulse.value;` — The pulse is set from the trigger bit or from the hardware set input. The test only uses the trigger path.

Items 1, 2, 3 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- count_incr_r / intr_count references outside the register RTL in `whole repo`: Only the coverage interface, mcu_intr_handler.c and the docs mention them.
- checks in mcu_intr_handler.c in `src/integration/test_suites/mcu_intr_handler`: Sum comparison of counters against the software count; the trigger path only; no per-counter or pulse-width check.

Looked at and ruled out:

- `src/mci/coverage/mci_top_cov_if.sv:3960`: Coverage points only; no checks.

5.37 No test found: 5 requirements in mci_reg.rdl > notif_mbox0_soc_req_lock_intr_count_r (part 1/2) (and 3 more sections)

`f_f8fdf28110d1` · severity medium

1. **Requirement** `cr_aa899d455be8`: mci_reg.intr_block_rf.notif_mbox0_soc_req_lock_intr_count_r.cnt: the field is a counter.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_soc_req_lock_intr_count_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1571`: `counter`

   Verified: No test references this counter's field.

   - `src/mci/coverage/mci_top_cov_if.sv:3970`: `INTR_BLOCK_RF_ERROR_MBOX0_ECC_UNC_INTR_COUNT_R_cp : coverpoint i_mci_reg_top.i_mci_reg.field_storage.intr_block_rf.error_mbox0_ecc_unc_intr_count_r;` — A coverpoint only samples the counter value. It checks nothing.

2. **Requirement** `cr_ae2e67d909b3`: mci_reg.intr_block_rf.error_mbox0_ecc_unc_intr_count_r.cnt: the field is a counter.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > error_mbox0_ecc_unc_intr_count_r

   - `src/mci/rtl/mci_reg.rdl:1571`: `counter`

   Verified: Only a coverpoint samples this counter. No test checks it.

3. **Requirement** `cr_ae5b56a5c9c2`: mci_reg.intr_block_rf.notif_mbox0_soc_req_lock_intr_count_r.cnt: the counter increments by 1.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_soc_req_lock_intr_count_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1569`: `incrvalue = 1`

   Verified: No test checks the increment by 1.

4. **Requirement** `cr_b15aa1219258`: The counter saturates and does not overflow.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_cptra_mbox_cmd_avail_intr_count_r

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test checks saturation.

5. **Requirement** `cr_b2c572ea94c4`: mci_reg.intr_block_rf.notif_mbox1_target_done_intr_count_r.cnt: the counter increments by 1.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_target_done_intr_count_r

   - `src/mci/rtl/mci_reg.rdl:1569`: `incrvalue = 1`

   Verified: No test checks the increment by 1.

Searched:

- intr_count_r references in tests in `whole repo, *.c/*.h/*.sv/*.py/*.yml`: Only generated register files, the coverage interface, the docs and the address-map defines.
- notif_mbox0_soc_req_lock counter (cnt field, increment by 1) in `src/integration`: Only soc_address_map defines. No test code uses it.
- error_mbox0_ecc_unc counter (cnt field) in `src/integration and src/mci/coverage`: Only address-map defines and a coverpoint at mci_top_cov_if.sv:3970. No check.
- notif_cptra_mbox_cmd_avail counter (saturation) in `src/integration`: Only address-map defines. No test uses it.
- notif_mbox1_target_done counter (increment by 1) in `src/integration`: Only address-map defines. No test uses it.

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_wdt/smoke_test_wdt.c:28`: intr_count is an unrelated software variable, not the register counter.

5.38 No test found: 5 requirements in mci_reg.rdl > notif_mbox0_target_done_intr_count_incr_r (part 1/3) (and 4 more sections)

`f_da26e2506ab0` · severity medium

1. **Requirement** `cr_156d1497e96f`: The event counter increments on the rising edge of a hardware interrupt event input.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_target_done_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1542-1544`: `Trigger the event counter to increment based on observing the rising edge of an interrupt event input from the Hardware. The same input signal that causes an interrupt`

   Verified: No test checks the counter increment on a rising edge of an interrupt event input.

   - `src/integration/test_suites/libs/mci/mci.c:2088-2090` — These are register mask entries, used to exclude the field from generic register checks. They are not a check of the pulse behavior.

     ```
     // ERROR_INTERNAL_INTR_COUNT_INCR_R
     add_mask_entry(SOC_MCI_TOP_MCI_REG_INTR_BLOCK_RF_ERROR_INTERNAL_INTR_COUNT_INCR_R,
     ```

2. **Requirement** `cr_15934effe86a`: mci_reg.intr_block_rf.notif_mbox1_cmd_avail_intr_count_incr_r.pulse: the counter decrements by 1.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1554`: `decrvalue = 1`

   Verified: No test checks that the counter decrements by 1.

3. **Requirement** `cr_17b3f2ee3de1`: The pulse field mirrors the occurrence of the interrupt event.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > error_mbox0_ecc_unc_intr_count_incr_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1557`: `Pulse mirrors interrupt event occurrence`

   Verified: No test checks that the pulse mirrors the interrupt event.

4. **Requirement** `cr_19033470337c`: The pulse is a 1-bit down-counter that decrements immediately on being set, producing a pulse.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1548-1549`: `This is implemented as a down-counter (1-bit) that will decrement immediately on being set - resulting in a pulse`

   Verified: No test checks the 1-bit down-counter pulse behavior.

5. **Requirement** `cr_193c2e86ca72`: The same input that sets the interrupt event also makes the pulse signal assert for 1 clock cycle.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_cptra_mbox_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1544-1546`: `Hardware. The same input signal that causes an interrupt event to be set (sticky) also causes this signal to pulse for 1 clock cycle, resulting in the event counter`
   - `src/mci/rtl/mci_reg.rdl:1557`: `Pulse mirrors interrupt event occurrence`

   Verified: No test checks that the same input that sets the event makes the pulse assert for 1 clock cycle.

Searched:

- count_incr / INTR_COUNT_INCR across the repository in `whole repo`: Only mci_reg.rdl, mci_reg.sv, mci_reg_uvm.sv, mci_top_cov_if.sv, a covergroup generator script, address-map headers and libs/mci/mci.c. No test body references it.
- rising-edge increment of the event counter (cr_156d1497e96f) in `C tests, SV tests, assertions`: No test or assertion checks it.
- decrvalue = 1 decrement (cr_15934effe86a) in `C tests, SV tests, assertions`: No test or assertion checks it.
- pulse mirrors the interrupt event (cr_17b3f2ee3de1) in `C tests, SV tests, assertions`: No test or assertion checks it.
- 1-bit down-counter producing a pulse (cr_19033470337c) in `C tests, SV tests, assertions`: No test or assertion checks it.
- 1-clock-cycle pulse from the same input as the interrupt event (cr_193c2e86ca72) in `C tests, SV tests, assertions`: No test or assertion checks it.

Looked at and ruled out:

- `src/mci/coverage/mci_top_cov_if.sv:4800-4810`: Coverpoints only sample the field value. They contain no check that can fail.

5.39 No test found: 5 requirements in mci_reg.rdl > notif_mbox1_cmd_avail_intr_count_incr_r (part 1/3) (and 4 more sections)

`f_18a44fce64f3` · severity medium

1. **Requirement** `cr_cbc0fcd7c65d`: The pulse field mirrors the occurrence of the interrupt event.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_cmd_avail_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1557`: `Pulse mirrors interrupt event occurrence`

   Verified: No test checks that the pulse field mirrors the interrupt event.

   - `src/integration/test_suites/libs/mci/mci.c:2096-2098`: `// ERROR_MBOX1_ECC_UNC_INTR_COUNT_INCR_R` — Only a mask entry for register-compare. It does not exercise or check pulse behaviour.

2. **Requirement** `cr_d349c789c93e`: The event counter increments on observing the rising edge of the hardware interrupt event input.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_soc_req_lock_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1542-1544`: `Trigger the event counter to increment based on observing the rising edge of an interrupt event input from the Hardware.`

   Verified: No test checks that the counter increments on the rising edge of the event input.

3. **Requirement** `cr_db5c35819d83`: The interrupt event counter does not overflow; it saturates.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_soc_req_lock_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test drives a counter to its maximum to check saturation.

4. **Requirement** `cr_dd69e103328e`: The 1-bit down-counter decrements immediately after being set, producing a pulse.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > error_mbox1_ecc_unc_intr_count_incr_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1548-1549`: `This is implemented as a down-counter (1-bit) that will decrement immediately on being set - resulting in a pulse`

   Verified: No test checks that the 1-bit down-counter decrements immediately and produces a pulse.

5. **Requirement** `cr_e5160437bc33`: The interrupt event counter does not overflow; it saturates.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_target_done_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test checks saturation of the interrupt event counter.

Searched:

- count_incr register names across the repo outside the RDL in `whole repo`: Only generated RTL/UVM, the coverage interface, address map headers, the covergroup generator and mci.c.
- intr_count_incr in the C tests in `src/integration/test_suites`: Only mci.c mask entries (99 entries in total).
- saturate or incr in the C tests in `src/integration/test_suites/*.c`: Only mci.c and an unrelated LCC test.

Looked at and ruled out:

- `src/mci/coverage/mci_top_cov_if.sv:1`: Coverage only, with no check that can fail.

5.40 No test found: 1 requirement in mci_reg.rdl > notif_mbox1_cmd_avail_intr_count_r (and 3 more sections)

`f_d5b362d9bfbe` · severity medium

4. **Requirement** `cr_a0511df283f8`: The event counter saturates and does not overflow.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > error_mbox0_ecc_unc_intr_count_r

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test drives a counter to its maximum or checks that it saturates.

   - `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:201-204`: `if (mci_error_intr_count != mci_error_intr_count_hw) {` — Check: the sum of the HW counters, including MBOX0/MBOX1 ECC_UNC, must equal the FW count. A missed or double increment fails the test.

Items 1, 2, 3, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- Tests or libs referencing these specific counters in `whole repo, excluding the address map and RTL`: mcu_intr_handler.c (stimulus and check), libs/mci/mci.c (register table only), coverage files
- Saturation or overflow test for the counters in `src/integration/test_suites`: none

Looked at and ruled out:

- `src/integration/test_suites/libs/mci/mci.c:569-585`: Only a register-property table (sticky or not), no stimulus or check of counting.

5.41 No test found: 5 requirements in mci_reg.rdl > notif_mbox1_soc_req_lock_intr_count_incr_r (part 1/3) (and 3 more sections)

`f_287a6254cf79` · severity medium

1. **Requirement** `cr_4d9003ba78b7`: mci_reg.intr_block_rf.notif_mbox1_soc_req_lock_intr_count_incr_r.pulse: software writes take effect only when false is deasserted.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_soc_req_lock_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1557`: `swwel = false`

   Verified: No test found.

   - `src/mci/rtl/mci_reg.rdl:1554`: `default decrvalue = 1;` — Source of the requirements; pulse field defined with swwel=false.

2. **Requirement** `cr_4ee310cb184b`: mci_reg.intr_block_rf.notif_mbox0_soc_req_lock_intr_count_incr_r.pulse: software writes take effect only when false is deasserted.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_soc_req_lock_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1557`: `swwel = false`

   Verified: No test found.

3. **Requirement** `cr_51c70a57d16c`: mci_reg.intr_block_rf.notif_mbox0_target_done_intr_count_incr_r.pulse: software writes take effect only when false is deasserted.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_target_done_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1557`: `swwel = false`

   Verified: No test found.

4. **Requirement** `cr_56a40256092d`: The same input signal that sets the interrupt event also makes the pulse signal assert for 1 clock cycle.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > error_mbox0_ecc_unc_intr_count_incr_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1544-1546`: `The same input signal that causes an interrupt event to be set (sticky) also causes this signal to pulse for 1 clock cycle`

   Verified: No test found.

5. **Requirement** `cr_6883bed01f38`: mci_reg.intr_block_rf.notif_mbox0_soc_req_lock_intr_count_incr_r.pulse: the counter decrements by 1.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_soc_req_lock_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1554`: `decrvalue = 1`

   Verified: No test found.

Searched:

- count_incr names outside rdl in `repo`: only mci_reg.sv, mci_reg_uvm.sv, mci_top_cov_if.sv
- item cr_4d9003ba78b7: notif_mbox1_soc_req_lock count_incr pulse swwel in `repo excluding mci_reg*`: nothing
- item cr_4ee310cb184b: notif_mbox0_soc_req_lock count_incr in `repo excluding mci_reg*`: only coverage if
- item cr_51c70a57d16c: notif_mbox0_target_done count_incr in `repo excluding mci_reg*`: only coverage if
- item cr_56a40256092d: error_mbox0_ecc_unc count_incr pulse in `repo excluding mci_reg*`: only coverage if

- item cr_6883bed01f38: decrement by 1 of soc_req_lock counter in `repo excluding mci_reg*`: only coverage if

Looked at and ruled out:

- `src/mci/coverage/mci_top_cov_if.sv:1`: Coverage only, no pass/fail check.

5.42 No test found: 1 requirement in mci_reg.rdl > notif_mbox1_soc_req_lock_intr_count_r (part 1/2) (and 3 more sections)

`f_975017b57f3d` · severity medium

2. **Requirement** `cr_4e89639f30e9`: The counter does not overflow; it saturates at its maximum value.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_soc_req_lock_intr_count_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test drives a counter to its maximum value to check saturation.

Items 1, 3, 4, 5 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- tests of intr_count registers and saturation in `src/integration, src/mci coverage`: mci.c only checks reset and sticky behaviour; mcu_intr_handler.c has an aggregate count check; no saturation test (grep 'saturat' found nothing); mci_top_cov_if.sv has only coverpoints.

Looked at and ruled out:

- `src/integration/test_suites/libs/mci/mci.c:568-593`: Reset-value and sticky register checks only; does not exercise counting.

5.43 No test found: 5 requirements in mci_reg.rdl > notif_mbox1_target_done_intr_count_incr_r (part 1/3) (and 3 more sections)

`f_71dcb4de0ce5` · severity medium

1. **Requirement** `cr_c1daf87f3285`: The event counter increments upon a rising edge of the hardware interrupt event input.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_target_done_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1542-1544`: `Trigger the event counter to increment based on observing the rising edge of an interrupt event input from the Hardware.`

   Verified: No test creates a hardware interrupt rising edge and then checks the notif_mbox1_target_done counter.

   - `src/integration/test_suites/libs/mci/mci.c:2096-2098` — This is only a register read/write mask entry. It does not stimulate an interrupt event or check the counter.

     ```
     // ERROR_MBOX1_ECC_UNC_INTR_COUNT_INCR_R
     add_mask_entry(SOC_MCI_TOP_MCI_REG_INTR_BLOCK_RF_ERROR_MBOX1_ECC_UNC_INTR_COUNT_INCR_R,
     ```

2. **Requirement** `cr_c43fbab16d30`: The pulse field mirrors the occurrence of an interrupt event.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_target_done_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1557`: `Pulse mirrors interrupt event occurrence`

   Verified: No test checks that the pulse field mirrors an interrupt event.

3. **Requirement** `cr_c65a366c2f2e`: The event counter increments based on a rising edge of the hardware interrupt event input.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > error_mbox1_ecc_unc_intr_count_incr_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1542-1544`: `Trigger the event counter to increment based on observing the rising edge of an interrupt event input from the Hardware.`

   Verified: The only reference to the error_mbox1_ecc_unc incr register is a register mask entry. It does not check the counter.

4. **Requirement** `cr_c79bd4ac60ff`: The event counter increments based on observing the rising edge of a hardware interrupt event input.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_ecc_cor_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1542-1544`: `Trigger the event counter to increment based on observing the rising edge of an interrupt event input from the Hardware.`

Verified: No test references the notif_mbox1_ecc_cor incr register.

5. **Requirement** `cr_c89b47159e50`: mci_reg.intr_block_rf.notif_mbox0_target_done_intr_count_incr_r.pulse: the counter decrements by 1.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_target_done_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1554`: `decrvalue = 1`

   Verified: No test checks the decrvalue = 1 behaviour of the notif_mbox0_target_done pulse.

Searched:

- intr_count_incr_r and the individual register names, outside the RDL and generated reg files in `whole repo`: Only address defines, the UVM register model, coverage coverpoints and the mci.c mask table.
- INTR_COUNT_INCR in C tests, SV and scripts in `src`: Only generated files, coverage, and the mci.c mask entries. No test references the incr registers.
- intr_count in the mcu_mbox smoke test in `src/integration/test_suites/smoke_test_mcu_mbox`: Only a local variable named intr_count. It is unrelated to the register.

Looked at and ruled out:

- `src/mci/coverage/mci_top_cov_if.sv:4820`: This is a coverpoint, so it samples the register but has no check that can fail.

5.44 No test found: 5 requirements in mci_reg.rdl > notif_mbox1_target_done_intr_count_incr_r (part 1/3) (and 4 more sections)

`f_b51910decc48` · severity medium

1. **Requirement** `cr_fb753857ed6c`: The pulse field mirrors the occurrence of the interrupt event.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_target_done_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1557`: `Pulse mirrors interrupt event occurrence`

   Verified: No test or check on the incr pulse field. Only a coverpoint exists.

   - `src/mci/coverage/mci_top_cov_if.sv:4700`: `INTR_BLOCK_RF_NOTIF_MBOX1_TARGET_DONE_INTR_COUNT_R_cp : coverpoint i_mci_reg_top.i_mci_reg.field_storage.intr_block_rf.notif_mbox1_target_done_intr_count_r;` — Coverage sampling only; it does not check counter behaviour.

2. **Requirement** `cr_fd7b0222641a`: The same input that sets the interrupt event also makes the incrementor signal pulse for 1 clock cycle.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_soc_req_lock_intr_count_incr_r (part 1/3)

   - `src/mci/rtl/mci_reg.rdl:1544-1546`: `The same input signal that causes an interrupt event to be set (sticky) also causes this signal to pulse for 1 clock cycle`

   Verified: No test or check ties the interrupt event input to a 1-cycle incr pulse.

3. **Requirement** `cr_01d650d8691a`: mci_reg.intr_block_rf.notif_mbox1_ecc_cor_intr_count_r.cnt: the field is a counter.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_ecc_cor_intr_count_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1571`: `counter`

   Verified: No test or check on ecc_cor intr_count counting behaviour.

4. **Requirement** `cr_0b89e28cd650`: mci_reg.intr_block_rf.notif_mbox1_target_done_intr_count_r.cnt: the field is a counter.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_target_done_intr_count_r

   - `src/mci/rtl/mci_reg.rdl:1571`: `counter`

   Verified: No test or check on target_done intr_count counting behaviour. Only a coverpoint.

5. **Requirement** `cr_0f4287b8ecea`: mci_reg.intr_block_rf.notif_mbox1_cmd_avail_intr_count_r.cnt: the field is a counter.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_cmd_avail_intr_count_r

   - `src/mci/rtl/mci_reg.rdl:1571`: `counter`

   Verified: No test or check on cmd_avail intr_count counting behaviour.

Searched:

- incr pulse field (target_done_intr_count_incr_r) in tests in `whole repo`: Only generated RTL/UVM and a coverpoint at mci_top_cov_if.sv:5540. No check.
- soc_req_lock incr pulse from the interrupt event input in `whole repo`: Only the RDL, generated RTL/UVM and a coverpoint. No test.
- ecc_cor intr_count counter in `whole repo`: Only the RDL, generated RTL/UVM and a coverpoint at mci_top_cov_if.sv:4750. No test.
- target_done intr_count counter in `whole repo`: Only the RDL, generated RTL/UVM and a coverpoint at mci_top_cov_if.sv:4700. No test.
- cmd_avail intr_count counter in `whole repo`: Only the RDL, generated RTL/UVM and a coverpoint at mci_top_cov_if.sv:4720. No test.

Looked at and ruled out:

- `src/mci/coverage/mci_top_cov_if.sv:5540`: Coverpoint on the incr signal. It only samples values and has no pass/fail check.

5.45 No test found: 5 requirements in mci_reg.rdl > notif_mbox1_target_done_intr_count_r (and 4 more sections)

`f_55e07d9fb565` · severity medium

1. **Requirement** `cr_c263d751c70e`: The interrupt event counter saturates instead of overflowing.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_target_done_intr_count_r

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test drives notif_mbox1_target_done_intr_count_r to saturation and checks it.

   - `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:62`: `mci_error_intr_count++; __asm__ volatile ("wfi") /* "Wait for interrupt" */; mcu_sleep(100);` — The interrupt handler test triggers each interrupt once and counts it. It never gets near the counter's saturation point.
   - `src/integration/test_suites/libs/mci/mci.c:588`: `{ SOC_MCI_TOP_MCI_REG_INTR_BLOCK_RF_NOTIF_MBOX1_ECC_COR_INTR_COUNT_R, "NOTIF_MBOX1_ECC_COR_INTR_COUNT_R", "Notification MBOX1 ECC Correctable Intrpt Count", REG_NOT_STICKY, false },` — This is a register reset and access table entry. It does not exercise saturation.

2. **Requirement** `cr_db7951777429`: The interrupt event counter saturates at its maximum value and does not overflow.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_cmd_avail_intr_count_r

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test drives notif_mbox0_cmd_avail_intr_count_r to saturation and checks it.

3. **Requirement** `cr_ecafe675afa3`: The interrupt event counter saturates and does not overflow.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_cmd_avail_intr_count_r

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test drives notif_mbox1_cmd_avail_intr_count_r to saturation and checks it.

4. **Requirement** `cr_eea6fa2277a6`: The interrupt event counter does not overflow; it saturates at its maximum value.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_soc_req_lock_intr_count_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test drives notif_mbox0_soc_req_lock_intr_count_r to saturation and checks it.

5. **Requirement** `cr_f255266393b7`: The counter saturates and does not overflow.
   Section: src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox1_ecc_cor_intr_count_r (part 1/2)

   - `src/mci/rtl/mci_reg.rdl:1565`: `Will not overflow ('incrsaturate').`

   Verified: No test drives notif_mbox1_ecc_cor_intr_count_r to saturation and checks it.

Searched:

- saturat/overflow in the integration tests in `src/integration (c, sv, svh, h, py, yml files)`: Only I3C FIFO overflow defines in the address map. No test of interrupt counter saturation.
- intr_count and saturation in tests and libraries in `mcu_intr_handler.c, libs/mci/mci.c, and the files that matched across the repository`: Only per-interrupt trigger counting and register reset/access tables. The matches in uart_core.sv, tools/scripts and the coverage interface are not tests of saturation.
- notif_mbox1_target_done_intr_count_r saturation in `whole repository, by register name and saturation terms`: Only the mci.c reset/access table entry (line 583). No saturation stimulus.

- notif_mbox0_cmd_avail_intr_count_r saturation in `whole repository, by register name and saturation terms`: Only the mci.c reset/access table entry (line 584). No saturation stimulus.
- notif_mbox1_cmd_avail_intr_count_r saturation in `whole repository, by register name and saturation terms`: Only the mci.c reset/access table entry (line 585). No saturation stimulus.
- notif_mbox0_soc_req_lock_intr_count_r saturation in `whole repository, by register name and saturation terms`: Only the mci.c reset/access table entry (line 591). No saturation stimulus.
- notif_mbox1_ecc_cor_intr_count_r saturation in `whole repository, by register name and saturation terms`: Only the mci.c reset/access table entry (line 588). No saturation stimulus.

5.46 Partly tested: 1 requirement in mcu_mbox_csr.rdl > mbox_cmd_status

`f_24395b698cd0` · severity medium

1. **Requirement** `cr_e44c4e0735c0`: Status value CMD_FAILURE means the requested command experienced a failure.

   - `src/mci/rtl/mcu_mbox_csr.rdl:143`: `The requested command experienced a failure.`

   Tested by `mci_mbox0_ownership_flow`**:** Runs a transaction ending in CMD_FAILURE, but the only visible check is the error counter.
   Not tested: No visible check that the status value read by the requester equals CMD_FAILURE.

   Verified: No test reads CMD_STATUS after a CMD_FAILURE transaction and checks the value.

   - `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:138`: `mcu_mbox_write(mci_comm_flow_cmd_status_addr, mci_comm_flow_root, status);` — Status is only written. The following phase check covers SRAM owner, SRAM reads and HW_STATUS, with no CMD_STATUS value readback.

Searched:

- CMD_FAILURE across the repo in `whole repo`: Only the ownership flow test, the smoke tests (payload value), the library enum and the RDL/package.
- Status readback in the ownership flow in `mci_mbox_ownership_flow.svh`: No read of CMD_STATUS compared to the expected status value.

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_valid_user/cptra_smoke_test_mcu_mbox_valid_user.c:168-177`: Writes MCU_MBOX_CMD_FAILURE as an attempted CMD_STATUS write by a user and checks the write is rejected. It does not check a real CMD_FAILURE outcome.

5.47 No test found: 1 requirement in mcu_mbox_csr.rdl > mbox_cmd_status (and 1 more section)

`f_4ea84c814898` · severity medium

1. **Requirement** `cr_a89b498a6610`: mcu_mbox_csr.mbox_cmd_status.status: on simultaneous hardware and software updates, hw takes precedence.
   Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_cmd_status

   - `src/mci/rtl/mcu_mbox_csr.rdl:146`: `precedence=hw`

   Verified: No test creates a simultaneous hw clear and sw write to cmd_status.status, and none checks that hw wins.

Item 2 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- sram_owner NONE checks in tests in `src/integration`: mci_mbox_ownership_flow.svh checks NONE in the 'released' phase; it is in the nightly regression lists
- tests exercising simultaneous hw clear and sw write of cmd_status in `src/integration test_suites and sv_tests`: none; only ordinary cmd_status accesses

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_target_user/smoke_test_mcu_mbox_target_user.c:79-81`: Touches HW_STATUS but does not test cmd_status hw/sw collision.

5.48 Partly tested: 3 requirements in mcu_mbox_csr.rdl > mbox_lock (and 2 more sections)

`f_4839917347eb` · severity medium

3. **Requirement** `cr_618052e4c5dd`: Setting TARGET_USER_VALID locks both target configuration registers until hardware clears them.
   Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_target_user

   - `src/mci/rtl/mcu_mbox_csr.rdl:71`: `Setting TARGET_USER_VALID locks both target configuration registers until hardware clears them.`

   Tested by `smoke_test_mcu_mbox_target_user`**:** Both target configuration registers are checked to be cleared by hardware after the terminal status.
   Not tested: No attempted software write while TARGET_USER_VALID is set, so the locking itself is not verified.

   Tested by `smoke_test_mcu_mbox_strb_wr_csr`**:** After valid is set, a write to TARGET_USER is ignored, which shows the lock on that register.
   Not tested: Only TARGET_USER is checked, not TARGET_USER_VALID. Release of the lock when hardware clears the registers is not checked.

   Verified: Only TARGET_USER is checked as locked. TARGET_USER_VALID is not, and release of the lock is not checked.

   - `src/integration/test_suites/smoke_test_mcu_mbox_strb_wr_csr/smoke_test_mcu_mbox_strb_wr_csr.c:145-150`: `lsu_write_8(SOC_MCI_TOP_MCU_MBOX0_CSR_MBOX_TARGET_USER + MCU_MBOX_NUM_STRIDE * mbox_num, 0);` — Only TARGET_USER is checked as locked. There is no write to TARGET_USER_VALID and no check that the lock is released.

4. **Requirement** `cr_77efe78b888c`: The user field value is used only while TARGET_USER_VALID is set.
   Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_target_user

   - `src/mci/rtl/mcu_mbox_csr.rdl:70`: `The value is used only while TARGET_USER_VALID is set.`

   Tested by `smoke_test_mcu_mbox_target_user`**:** Target accesses are blocked while TARGET_USER_VALID is 0 and allowed after the grant with TARGET_USER set.
   Not tested: Does not check that target access is blocked again after TARGET_USER_VALID is cleared.

   Verified: I found no check that target access is blocked again after valid is cleared.

5. **Requirement** `cr_465defb96dea`: Writing 1 locks TARGET_USER and TARGET_USER_VALID until hardware clears both.
   Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_target_user_valid

   - `src/mci/rtl/mcu_mbox_csr.rdl:79`: `Writing 1 locks TARGET_USER and TARGET_USER_VALID until hardware clears both.`

   Tested by `smoke_test_mcu_mbox_strb_wr_csr`**:** After TARGET_USER_VALID is set, a write of 0 to TARGET_USER is ignored, so the TARGET_USER lock is verified.
   Not tested: The lock on TARGET_USER_VALID itself (a write of 0 to it being ignored) is not checked, nor is the unlock when hardware clears both.

   Verified: No test writes 0 to TARGET_USER_VALID while it is set, and none checks the unlock after the hardware clear.

Items 1, 2 of this group were not verified (see Withdrawn and Inconclusive).

Searched:

- TARGET_USER / TARGET_USER_VALID uses in tests, for the target_user items (cr_618052e4c5dd, cr_465defb96dea) in `repo-wide, then the strb_wr_csr and write_user_lock tests`: The strb_wr_csr test only attempts a TARGET_USER write after valid is set. No test writes 0 to TARGET_USER_VALID while it is set, and none checks that writes are accepted after a hardware clear.
- Blocked access after valid clears (cr_77efe78b888c) in `write_user_lock test and repo-wide TARGET_USER_VALID grep`: No check found. I did not open the target_user test itself.

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_write_user_lock/smoke_test_mcu_mbox_write_user_lock.c:129-149`: Only Root writes to the two registers before the grant. It does not attempt writes while valid is set.

5.49 No test found: 4 requirements in mcu_mbox_csr.rdl > mbox_lock (and 3 more sections)

`f_b928b4a5e53b` · severity medium

1. **Requirement** `cr_1090477d2115`: mcu_mbox_csr.mbox_lock.lock: on simultaneous hardware and software updates, hw takes precedence.

Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_lock

- `src/mci/rtl/mcu_mbox_csr.rdl:53`: `precedence=hw`

Verified: No test creates a simultaneous hardware and software update of lock.

2. **Requirement** `cr_a206ba683bb5`: mcu_mbox_csr.mbox_target_user.user: on simultaneous hardware and software updates, hw takes precedence.
Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_target_user

   - `src/mci/rtl/mcu_mbox_csr.rdl:73`: `precedence=hw`

   Verified: No test creates a simultaneous update of TARGET_USER.

3. **Requirement** `cr_7eea266009ff`: mcu_mbox_csr.mbox_target_user_valid.valid: on simultaneous hardware and software updates, hw takes precedence.
Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_target_user_valid

   - `src/mci/rtl/mcu_mbox_csr.rdl:81`: `precedence=hw`

   Verified: TARGET_USER_VALID hardware clear is checked, but not the simultaneous-update precedence.

4. **Requirement** `cr_b97504c83bee`: mcu_mbox_csr.mbox_target_status.status: hardware can clear the field (true).
Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_target_status

   - `src/mci/rtl/mcu_mbox_csr.rdl:122`: `hwclr`

   Verified: No test reads TARGET_STATUS back after a handoff to confirm the hardware clear; the only read is after release, when the register was never nonzero.

   - `src/integration/test_suites/smoke_test_mbox_csr_zeros_after_release/smoke_test_mbox_csr_zeros_after_release.c:132`: `if(lsu_read_32(SOC_MCI_TOP_MCU_MBOX0_CSR_MBOX_TARGET_STATUS + MCU_MBOX_NUM_STRIDE * mbox_num) != 0) {` — Reads TARGET_STATUS after release, but Root never makes it nonzero, so the hwclr is not exercised.

Item 4 of this group was not verified (see Withdrawn and Inconclusive).

Searched:

- CMD_FAILURE / TARGET_STATUS in tests in `src/integration`: The ownership-flow SV test writes CMD_FAILURE as a terminal target status and checks the owner phase. Other tests only touch TARGET_STATUS access rights.
- simultaneous / same-cycle / collision tests for mbox in `src/integration mbox files`: none
- TARGET_STATUS clear checks in `zeros_after_release and target_user tests`: zeros_after_release reads TARGET_STATUS == 0, but the status is never made nonzero there. target_user checks only TARGET_USER_VALID cleared.

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_target_user/smoke_test_mcu_mbox_target_user.c:240-245`: Checks that TARGET_USER_VALID is cleared by hardware on terminal status, not the hw-over-sw precedence on simultaneous update.

5.50 No test found: 5 requirements in mcu_mbox_csr.rdl > mbox_target_status (and 1 more section)

`f_2a4333029411` · severity medium

1. **Requirement** `cr_c466f1401b85`: Hardware resets TARGET_STATUS to BUSY on each Target handoff.
Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_target_status

   - `src/mci/rtl/mcu_mbox_csr.rdl:106`: `Reset to BUSY by hardware on each target handoff.`
   - `docs/CaliptraSSHardwareSpecification.md:1434`: `Hardware resets \`TARGET_STATUS\` to BUSY on each Target handoff.`

   Verified: No test reads TARGET_STATUS as BUSY after a handoff.

   - `src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:131-133`: `mcu_mbox_write(mci_comm_flow_target_status_addr, mci_comm_flow_target, status);` — The test writes a terminal TARGET_STATUS (DATA_READY, CMD_COMPLETE or CMD_FAILURE). It then checks only the TARGET_DONE notification and SRAM ownership, never a read-back of the status or a BUSY value.
   - `src/integration/test_suites/libs/caliptra_ss_lib/caliptra_ss_lib.h:245`: `MCU_MBOX_TARGET_STATUS_CMD_BUSY = 0x0,` — This enum definition is the only BUSY reference outside the USB suites. No test checks that the status reads as BUSY.

2. **Requirement** `cr_d48ef4f510ee`: Value DATA_READY means the requested data is ready in the mailbox.
   Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_target_status

   - `src/mci/rtl/mcu_mbox_csr.rdl:113`: `Requested data is ready in the mailbox.`

   Verified: DATA_READY is only written as a terminal status for ownership checks. The data-ready meaning is not checked for target_status.

3. **Requirement** `cr_ea1bed14203c`: mcu_mbox_csr.mbox_target_status.status: on simultaneous hardware and software updates, hw takes precedence.
   Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_target_status

   - `src/mci/rtl/mcu_mbox_csr.rdl:122`: `precedence=hw`

   Verified: No stimulus makes a hardware clear and a software write land together.

4. **Requirement** `cr_f9cc76302a6a`: Value CMD_BUSY means the target is busy and the command is not yet complete.
   Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_target_status

   - `src/mci/rtl/mcu_mbox_csr.rdl:110`: `Target is busy, command is not yet complete.`

   Verified: No test observes or checks the CMD_BUSY value for target_status.

5. **Requirement** `cr_01230983dc89`: Status value CMD_BUSY means MCU is busy and the command is not yet complete.
   Section: src/mci/rtl/mcu_mbox_csr.rdl > mcu_mbox_csr > mbox_cmd_status

   - `src/mci/rtl/mcu_mbox_csr.rdl:134`: `MCU is busy, command is not yet complete.`

   Verified: No test checks CMD_BUSY on cmd_status. The flows only write non-BUSY values.

Searched:

- Item cr_c466f1401b85: a test that reads TARGET_STATUS as BUSY after a target handoff (grep for TARGET_STATUS, target_status, STATUS_BUSY) in `src/integration (testbench, test_suites)`: Tests only write a terminal TARGET_STATUS and wait for it. None reads BUSY after a handoff.
- Item cr_d48ef4f510ee: a check of target_status DATA_READY semantics (grep for DATA_READY) in `src/integration`: DATA_READY is written as a terminal status in the ownership-flow loop. The smoke tests compare CMD_STATUS to DATA_READY, which is cmd_status rather than target_status. No check of what the value means for target_status.
- Item cr_ea1bed14203c: a test of hw/sw precedence on target_status (grep for hwclr, handoff, simultaneous, precedence, case-insensitive) in `mbox-named files under src/integration`: No matches.
- Item cr_f9cc76302a6a: a test that checks CMD_BUSY on target_status (grep for CMD_BUSY, TARGET_STATUS_BUSY) in `src/integration, excluding the USB suites`: Only the enum definitions in caliptra_ss_lib.h, lines 239 and 245.
- Item cr_01230983dc89: a test that writes or checks CMD_BUSY on mbox_cmd_status (grep for cmd_status, CMD_BUSY) in `src/integration/test_suites and testbench/sv_tests/mci`: The flows write only non-BUSY values to CMD_STATUS. Nothing checks that a BUSY state keeps SRAM ownership or leaves the command incomplete.

Looked at and ruled out:

- `src/integration/test_suites/smoke_test_mcu_mbox_pass_data/cptra_smoke_test_mcu_mbox_pass_data.c:423`: This waits for CMD_STATUS DATA_READY on the command-status register. It does not test the target_status DATA_READY semantics or CMD_BUSY.
- `src/integration/test_suites/smoke_test_mcu_mbox_target_user/smoke_test_mcu_mbox_target_user.c:212`: This waits for TARGET_STATUS COMPLETE only. It does not check BUSY after the handoff.

5.51 Register access: no suite covers every register

`f_e684b2415bb9` · register suite · severity low

Gap: 81 register properties of this kind are not covered by a test that runs over every register: the suites found cover only some of the registers.

Suites that touch these registers: `smoke_test_mcu_mbox_strb_wr_csr`

Verified: I found no generic register test that runs over the MCU mailbox CSR block. mci_reg_rand_test sweeps only the MCI-register-space mailbox valid-user registers (REG_GROUP_MCI_MBOX0/1). Its group list leaves out REG_GROUP_MCU_MBOX0, which holds the mcu_mbox CSRs (CMD, DLEN, EXECUTE, TARGET_STATUS and so on). The cited strobe-write test checks only a subset of registers directly.

Further evidence found by the verifier:

- `src/integration/test_suites/smoke_test_mcu_mbox_strb_wr_csr/smoke_test_mcu_mbox_strb_wr_csr.c:93`: `if(mcu_mbox_read_cmd(mbox_num) != mask) {` — The suite directly checks only a handful of registers (DLEN, CMD, CMD_STATUS, TARGET_STATUS, TARGET_USER). It is not a full per-register sweep.

Searched:

- UVM register sequences (reg_hw_reset, bit_bash, reg_access) in `whole repo`: none
- generic register tests in `src/integration/test_suites`: mci_reg_rand_test, which does not include the REG_GROUP_MCU_MBOX0 group. smoke_test_mcu_mbox_csrs_warm_rst is a separate directed test.

Looked at and ruled out:

- `src/integration/test_suites/mci_reg_rand_test/mci_reg_rand_test.c:46-57`: It touches mailbox registers, but only the MCI-space AXI-user registers. It does not touch the MCU_MBOX CSR block.

Inconclusive

The verifier could neither confirm nor refute these. They need a human decision.

- `f_469ba16e55c7` (partly tested) Trusted users require the MBOX lock to write the mailbox CSRs. — pass_data has a positive lock-then-write flow, but CMD_STATUS and USER are not exercised and the readback is weak.
- `f_6ae2ab31ed12` (partly tested) When an external Requester holds the lock, MCU is the only agent that may write CMD_STATUS. — Not searched in depth.
- `f_6ae2ab31ed12` (partly tested) EXECUTE is RW for MCU while MCU owns the SRAM, whether MCU or an external Requester holds the lock. — Not searched in depth.
- `f_6ae2ab31ed12` (partly tested) MCU write access to mailbox CSRs is determined by its role in the current transaction and follows the status register writer rules. — Not searched in depth.
- `f_6ae2ab31ed12` (partly tested) LOCK and USER access for MCU follows mailbox locking rules in both lock cases. — Not searched in depth.
- `f_e38d580bc493` (partly tested) Ownership conditions are evaluated in the stated precedence order. — I did not examine the ownership-flow tests or the RTL precedence logic, so I could not confirm or refute the precedence-combination gap.
- `f_a06f18895dc1` (test plan items with no test) MCU MBOX - Valid user CSR works when LOCK set and parameter not enabled — valid_user and axi_param touch this area but I could not confirm that either covers the exact condition.
- `f_fa3367a3301d` (partly tested) The SOC requester writes SRAM, DLEN and CMD to stage the command. — I did not search the staging-flow tests for a CMD or DLEN readback.
- `f_87f1e04df40b` (no test found) The event counter increments on the rising edge of the hardware interrupt event input. — The software-trigger count check indirectly exercises the increment for cmd_avail, but not a hardware input edge.
- `f_a40e976ebc6e` (no test found) The incrementor signal pulses for 1 clock cycle from the same input signal that sets the sticky interrupt event. — The 1-cycle pulse is only constrained indirectly by the summed count comparison, and the test drives the trigger rather than the hardware event input.
- `f_8e312cf1f5e1` (no test found) mci_reg.intr_block_rf.notif_mbox1_cmd_avail_intr_count_incr_r.pulse: the field is a counter. — The field being a counter is an implementation property. It is only implied by the count registers matching, with no direct check.
- `f_8e312cf1f5e1` (no test found) The pulse is a 1-bit down-counter that decrements immediately on being set, producing a pulse. — The 1-bit down-counter pulse mechanism is internal. No test checks it directly.
- `f_c879f7a38c02` (no test found) mci_reg.intr_block_rf.notif_mbox0_ecc_cor_intr_count_incr_r.pulse: the field is a counter. — Only implicitly exercised by the count-sum check; there is no direct test.
- `f_c879f7a38c02` (no test found) mci_reg.intr_block_rf.notif_cptra_mbox_cmd_avail_intr_count_incr_r.pulse: the field is a counter. — Only implicitly exercised by the count-sum check; there is no direct test.
- `f_c879f7a38c02` (no test found) mci_reg.intr_block_rf.notif_mbox0_ecc_cor_intr_count_incr_r.pulse: the counter decrements by 1. — The decrement is only implicitly exercised by the count-sum check; there is no direct test.
- `f_bb99ac035c0a` (no test found) The event counter increments by 1 for every interrupt event. — `mcu_intr_handler` checks that firmware and hardware counts match after one trigger per event, so +1 per event is partly checked. It does this through the software trigger path and in aggregate. The trigger-to-increment path in the RTL was only partly read.
- `f_b253afba5a90` (no test found) The 1-bit down-counter decrements immediately after being set, producing a pulse. — The 1-bit down-counter pulse is only covered indirectly by the sum (a stuck pulse would over-count). No direct check.
- `f_4839917347eb` (partly tested) mcu_mbox_csr.mbox_lock.lock: hardware can set the field (true). — I did not search for the hardware set of the lock separately.
- `f_4839917347eb` (partly tested) mcu_mbox_csr.mbox_lock.lock: hardware can clear the field (true). — I did not search for the hardware clear of the lock separately.

- `f_7fcd1bcb0f61` (partly tested) Only the current SRAM owner may write CMD, DLEN and EXECUTE. — I did not search for this item in depth.
- `f_7fcd1bcb0f61` (partly tested) No software owner when MBOX_LOCK is set and the mailbox is releasing or SRAM zeroizing. — I did not search for this item in depth.
- `f_7fcd1bcb0f61` (partly tested) Neither Requester nor Root can reclaim the mailbox from a Target that never writes terminal TARGET_STATUS or EXECUTE=0. — I did not search for this item in depth.
- `f_7fcd1bcb0f61` (partly tested) CMD_STATUS and TARGET_STATUS value 2 encodes COMPLETE. — I did not search for this item in depth. The target-user test uses a symbolic macro, so a literal-value check is not shown.

Withdrawn

Candidates the verifier refuted, with the reason. Listed so the refutations can be checked too.

- `f_228d6e140e36` (spec too ambiguous to test) SRAM ownership is derived from LOCK, EXECUTE, TARGET_USER_VALID, and CMD_STATUS. — The hardware specification has a full ownership decision tree and event table covering LOCK, release/zeroization, EXECUTE, TARGET_USER_VALID and CMD_STATUS. (`docs/CaliptraSSHardwareSpecification.md:1358-1359`, `docs/CaliptraSSHardwareSpecification.md:1382-1387`)
- `f_c33e138a2689` (spec too ambiguous to test) Mailbox SRAM is cleared from address 0x0 to max DLEN after lock is released. — Other spec text answers the question. (`docs/CaliptraSSHardwareSpecification.md:1462`, `docs/CaliptraSSHardwareSpecification.md:1473`)
- `f_13ad3b9fb3f3` (spec too ambiguous to test) Writing 0 to the execute register releases the mailbox. — The hardware specification has a dedicated MCU Mailbox SRAM Clearing section and an ownership table that define what releasing the mailbox does. (`docs/CaliptraSSHardwareSpecification.md:1462-1469`, `docs/CaliptraSSHardwareSpecification.md:1473`)
- `f_93d56e2865ea` (spec too ambiguous to test) Hardware clears the field on a terminal TARGET_STATUS write. — The TARGET_STATUS field description in the same RDL defines terminal as any non-BUSY value, which answers the question. (`src/mci/rtl/mcu_mbox_csr.rdl:106`)
- `f_1ab17f54aa12` (spec too ambiguous to test) On a double bit ECC error while reading the mailbox, invalid data is returned. — Other spec text answers the data question. (`docs/CaliptraSSIntegrationSpecification.md:874`, `docs/CaliptraSSHardwareSpecification.md:1507`)
- `f_d2fd297cb09f` (spec too ambiguous to test) If the MCU mailbox SRAM size is configured as 0, the mailbox is not instantiated. — The integration spec's parameter table says each mailbox is removed on its own when its size is 0. (`docs/CaliptraSSIntegrationSpecification.md:1806`, `docs/CaliptraSSIntegrationSpecification.md:1809`)
- `f_f6b9f6ce7717` (spec too ambiguous to test) Neither Requester nor Root can reclaim the mailbox from a Target that never writes terminal TARGET_STATUS or EXECUTE=0. — The register description in mcu_mbox_csr.rdl defines terminal as any non-BUSY TARGET_STATUS value. (`src/mci/rtl/mcu_mbox_csr.rdl:106`, `docs/CaliptraSSHardwareSpecification.md:1421-1426`)
- `f_5398dcbf6e1f` (spec too ambiguous to test) Each mailbox SRAM is fully addressable up to a maximum size of 2 MB. — The parameter table in the same integration spec gives a Min and Max for each of MCU_MBOX0_SIZE_KB and MCU_MBOX1_SIZE_KB separately (0 to 2048 KB). (`docs/CaliptraSSIntegrationSpecification.md:1806`, `docs/CaliptraSSIntegrationSpecification.md:1809`)
- `f_f8d817cdecbe` (spec too ambiguous to test) Only valid once LOCK is set. — Other spec text answers the finding's questions. (`src/mci/rtl/mci_reg.rdl:830-831`, `docs/CaliptraSSHardwareSpecification.md:1297-1299`)
- `f_1d852834951b` (spec too ambiguous to test) Only valid once LOCK is set. — Other spec text resolves the finding's questions. (`src/mci/rtl/mci_reg.rdl:830-831`, `docs/CaliptraSSIntegrationSpecification.md:2293-2296`)
- `f_b3b903089923` (spec too ambiguous to test) Signal valid_sram_owner_req gates CMD, DLEN and EXECUTE. — The hardware spec answers the question. (`docs/CaliptraSSHardwareSpecification.md:1406-1407`, `docs/CaliptraSSHardwareSpecification.md:1503`)
- `f_64041434ec7a` (spec too ambiguous to test) Writing any terminal value notifies Root. — The hardware spec and integration spec name the notification: the `notif_mbox*_target_done` interrupt, asserted when the Target writes a terminal TARGET_STATUS. (`docs/CaliptraSSHardwareSpecification.md:1483`, `docs/CaliptraSSHardwareSpecification.md:1485-1486`)
- `f_71e6abfb06ec` (spec too ambiguous to test) Hardware clears the user field on a terminal TARGET_STATUS write. — The TARGET_STATUS field description in the same RDL file defines terminal as any non-BUSY value. (`src/mci/rtl/mcu_mbox_csr.rdl:106`)
- `f_9f2b10330865` (spec contradicts itself) Root (typically MCU) is the only agent that may write/program TARGET_USER and TARGET_USER_VALID. — The text and table are consistent. (`docs/CaliptraSSHardwareSpecification.md:1318-1319`, `docs/CaliptraSSHardwareSpecification.md:1511-1513`)
- `f_b8016eaba96b` (spec contradicts itself) The description says the lock is set to root_user (MCU) on reset release, but the derived reset value property of the lo — The two statements are consistent. (`src/mci/rtl/mcu_mbox.sv:363`, `src/mci/rtl/mcu_mbox.sv:382`)
- `f_d6a990a3dea8` (spec contradicts itself) The description says AXI access is RW1-S (write-1-to-set), while the software access property is rw with a write-enable- — The two statements are consistent. (`src/mci/rtl/mci_reg.rdl:832-834`)
- `f_d891c4a7bb4a` (spec contradicts itself) The 2MB maximum and configurability by integration team may conflict with the fixed mementries (524288 x 32 bits = 2MB) — The two statements are consistent. (`src/mci/rtl/mcu_mbox_csr.rdl:39`)

- `f_543bab824f7a` (test contradicts its own description) smoke_test_mcu_mbox_usr_lock_out_zero: message vs. condition — The finding misreads the control flow. (`src/integration/test_suites/smoke_test_mcu_mbox_usr_lock_out_zero/cptra_smoke_test_mcu_mbox_usr_lock_out.c:285-291`)
- `f_61bacfa690b8` (test contradicts its own description) smoke_test_mcu_mbox_csrs_warm_rst: plan vs. test — The plan row (DB ECC: status bit, error interrupt, HW_ERROR_NON_FATAL) is realized by smoke_test_mcu_mbox_ecc, which checks all three. (`src/integration/test_suites/smoke_test_mcu_mbox_ecc/smoke_test_mcu_mbox_ecc.c:183-186`, `src/integration/test_suites/smoke_test_mcu_mbox_ecc/smoke_test_mcu_mbox_ecc.c:166-167`)
- `f_aa6e577cb904` (test contradicts its own description) smoke_test_mcu_mbox_csrs_warm_rst: plan vs. test — Plan row 122 (SB ECC sets status bit and sends interrupt to MCU) is realized by smoke_test_mcu_mbox_ecc, which checks the status bit and the SB ECC interrupt. (`src/integration/test_suites/smoke_test_mcu_mbox_ecc/smoke_test_mcu_mbox_ecc.c:104-107`)
- `f_ee15916e88b5` (test contradicts its own description) smoke_test_mcu_mbox_target_user: comment vs. code — The Caliptra helper really does write CMD when it is allowed, and it checks that the write took effect. (`src/integration/test_suites/smoke_test_mcu_mbox_target_user/cptra_smoke_test_mcu_mbox_target_user.c:153-167`, `src/integration/test_suites/smoke_test_mcu_mbox_target_user/cptra_smoke_test_mcu_mbox_target_user.c:161-164`)
- `f_d5ac44d7056b` (spec and test disagree) After MCI reset, Root owns the SRAM as the reset lock holder. — The spec's "Root, as the reset lock holder" can be read two ways, and the test's REQUESTER (1) expectation fits one of them, so this isn't a clear contradiction. (`docs/CaliptraSSHardwareSpecification.md:1444-1445`) Reclassified; see the finding marked *found during verification*.
- `f_e373c3f65ab5` (spec and test disagree) MCU writes EXECUTE=1 to hand the request to the target. — The test's order (EXECUTE=1, then TARGET_USER_VALID=1) matches the Hardware Specification's ownership table. (`docs/CaliptraSSHardwareSpecification.md:1398`, `docs/CaliptraSSHardwareSpecification.md:1397`) Reclassified; see the finding marked *found during verification*.
- `f_8696c28f3a2c` (spec not tested) Untested: 5 requirement(s) in src/mci/rtl/mci_reg.rdl > mci_reg > notif_mbox0_target_done_intr_count_incr_r (part 1/3) … (3 sections) — The mcu_intr_handler directed test triggers each error and notif interrupt, including the three counters cited here. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:201-204`, `src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256-259`)

Withdrawn items from the groups in section 5 (37):

- Writing any terminal value clears TARGET_USER_VALID. — The ownership flow runs DATA_READY, COMPLETE and CMD_FAILURE as terminal Target writes, then checks that SRAM ownership returns to Root. (`src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:202-204`)
- Writing any terminal value notifies Root. — Every terminal status in the ownership flow waits for TARGET_DONE, which fatals if it is missing. (`src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:202-204`)
- MCU MBOX - Verify DEFAULT_USER (1s for all USER bits) is considered a Valid User even when not explicitly set by VALID USER parameter or CSR — smoke_test_mcu_mbox uses 0xFFFFFFFF as Caliptra's user with no valid user configured. (`src/integration/test_suites/smoke_test_mcu_mbox/smoke_test_mcu_mbox.c:164-165`)
- Each interrupt has enable, status, trigger, and count register fields in the MCI register block with _en, _sts, _trig, and _intr_count_r suffixes. — mcu_intr_handler triggers each MBOX interrupt and checks the hardware count registers against its own count. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:201-204`)
- SRAM_OWNER value 0 means no owner. — The ownership flow checks SRAM_OWNER == NONE after release. (`src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:94-95`)
- Reads of the MCU mailbox SRAM are not destructive. — The owner's repeated reads must return the same data in every phase, which checks that reads are non-destructive. (`src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:83-84`)
- No software owner when MBOX_LOCK is not set. — The ownership flow checks that the SRAM owner is NONE in the released phase, where the lock is free, and stimulus drives EXECUTE=0 to reach it. (`src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:94-95`)
- The MCU mailbox sends the notif_mbox*_cmd_avail interrupt to the MCU when a command is available. — The ownership-flow test sets EXECUTE and then waits on the CMD_AVAIL notification status. (`src/integration/testbench/sv_tests/mci/mci_common_tasks.svh:489-491`)
- The pulse is a 1-bit down-counter that decrements immediately after being set, producing a pulse. — A pulse longer than 1 cycle would make the counts mismatch in mcu_intr_handler. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256-258`)
- mci_reg.intr_block_rf.notif_mbox1_ecc_cor_intr_count_incr_r.pulse: the counter decrements by 1. — decrvalue=1 is implicitly checked by the exact event count in mcu_intr_handler. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256-258`)
- The event counter increments by 1 for every interrupt event. — mcu_intr_handler triggers one event each and checks the counter sums match, including NOTIF_MBOX0_ECC_COR. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256-258`)
- The event counter increments by 1 for every interrupt event. — mcu_intr_handler triggers every notif and error interrupt and fails if the hardware counter sum differs from the firmware per-event count. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256-257`)
- The event counter increments by 1 for every interrupt event. — The mcu_intr_handler test fires one trigger per event, including CPTRA_MBOX_CMD_AVAIL and MBOX1_CMD_AVAIL, and fails if the summed hardware counts differ from the firmware count. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256-259`)

- The event counter increments by 1 for every interrupt event. — mcu_intr_handler triggers the mbox1_ecc_cor notification once and checks that the summed counters match the number of events. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256-259`)
- The event counter increments by 1 for every interrupt event. — mcu_intr_handler triggers each event and the summed counts must equal the firmware count, so it checks increment by 1 per event. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:213-216`)
- mci_reg.intr_block_rf.notif_mbox1_ecc_cor_intr_count_r.cnt: the counter increments by 1. — mbox1_ecc_cor is triggered once, and the summed hardware counters must equal the fw count. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:215-221`)
- mci_reg.intr_block_rf.notif_cptra_mbox_cmd_avail_intr_count_r.cnt: the field is a counter. — The cptra_mbox_cmd_avail counter is triggered (line 111) and read back (line 215), and the sum is compared. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:111`)
- mci_reg.intr_block_rf.notif_mbox0_cmd_avail_intr_count_r.cnt: the field is a counter. — The mbox0_cmd_avail counter is triggered and read back, and the sum is compared. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:215-221`)
- mci_reg.intr_block_rf.notif_mbox1_soc_req_lock_intr_count_r.cnt: the counter increments by 1. — mbox1_soc_req_lock is triggered once (line 117), and the summed counters must equal the fw count. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:117`)
- The event counter increments by 1 for every interrupt event. — mcu_intr_handler.c triggers soc_req_lock events and checks the summed counters equal the software count. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256`)
- The event counter increments by 1 for every interrupt event. — mbox1 target_done is triggered and its counter is included in the summed check. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256`)
- mci_reg.intr_block_rf.notif_mbox1_cmd_avail_intr_count_r.cnt: the counter increments by 1. — mcu_intr_handler triggers MBOX1 CMD_AVAIL once and checks the HW counter sum against the FW count. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:201-204`)
- mci_reg.intr_block_rf.error_mbox0_ecc_unc_intr_count_r.cnt: the counter increments by 1. — MBOX0 ECC_UNC is triggered once and its counter is in the sum that is checked. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:201-204`)
- mci_reg.intr_block_rf.error_mbox1_ecc_unc_intr_count_r.cnt: the field is a counter. — MBOX1 ECC_UNC is triggered and its counter is checked in the sum, which shows the field counts. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:201-204`)
- mci_reg.intr_block_rf.notif_mbox0_target_done_intr_count_r.cnt: the counter increments by 1. — MBOX0 TARGET_DONE is triggered once and its counter is in the checked notif sum. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:211-214`)
- mci_reg.intr_block_rf.notif_mbox1_soc_req_lock_intr_count_r.cnt: the field is a counter. — mcu_intr_handler triggers MBOX1_SOC_REQ_LOCK once and checks that the summed counters match the SW count. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256-259`)
- mci_reg.intr_block_rf.notif_mbox0_cmd_avail_intr_count_r.cnt: the counter increments by 1. — The test triggers MBOX0_CMD_AVAIL once; the summed-count check would fail if the increment were not 1. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256-259`)
- mci_reg.intr_block_rf.notif_mbox0_ecc_cor_intr_count_r.cnt: the field is a counter. — The test triggers MBOX0_ECC_COR once and the summed counters are checked. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256-259`)
- mci_reg.intr_block_rf.error_mbox1_ecc_unc_intr_count_r.cnt: the counter increments by 1. — The test triggers MBOX1_ECC_UNC once; the summed-count check would fail if the increment were not 1. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:201-204`)
- sram_owner value NONE (0) means no software agent owns the SRAM. — mci_mbox_ownership_flow checks sram_owner == NONE after release, and the check errors on mismatch. (`src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:93-95`)
- Value CMD_FAILURE means the requested command experienced a failure. — The ownership-flow test writes CMD_FAILURE as a terminal TARGET_STATUS and checks the resulting owner phase. (`src/integration/testbench/sv_tests/mci/mci_mbox_ownership_flow.svh:202-204`)
- Only MCU writes CMD_STATUS. — The target-user test has the Target attempt a CMD_STATUS write and checks the value is unchanged. (`src/integration/test_suites/smoke_test_mcu_mbox_target_user/cptra_smoke_test_mcu_mbox_target_user.c:171`)
- The event counter increments by 1 for every interrupt event. — notif_mbox0_target_done is triggered once, and its counter is read into the checked notif sum. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256-259`)
- The event counter increments by 1 for every interrupt event. — error_mbox1_ecc_unc is triggered once, and its counter is read into the checked error sum. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:201-204`)
- mci_reg.intr_block_rf.notif_mbox1_soc_req_lock_intr_count_incr_r.pulse: the counter decrements by 1. — The pulse counter decrementing by 1 is a pulse-generation attribute. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:256-259`)
- mci_reg.intr_block_rf.error_mbox1_ecc_unc_intr_count_incr_r.pulse: the field is a counter. — The counter attribute is exercised: the pulse field must act as a counter for the count to reach 1 per trigger. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:201-204`)
- mci_reg.intr_block_rf.error_mbox1_ecc_unc_intr_count_incr_r.pulse: the counter decrements by 1. — The decrvalue=1 pulse behavior is exercised: a wrong value would give a wrong count in the error sum. (`src/integration/test_suites/mcu_intr_handler/mcu_intr_handler.c:201-204`)

Run record

Stage	Model
Spec extraction	claude-sonnet-5-5
Test extraction	claude-sonnet-5-5
Merge and link	claude-sonnet-5-5
Correlate	claude-sonnet-5-5
Analyze	claude-opus-5-5, claude-sonnet-5-5
Verify	claude-sonnet-5-5

This run: $0.00 in 20 s (cached results reused where the inputs were unchanged). All LLM spend recorded for this workspace: $38.80.

Manifest `work/caliptra-ss-mci-mailbox/manifest.yaml` (sha256 `496706fd3104`); findings `work/caliptra-ss-mci-mailbox/findings.yaml`; verifier transcripts in `work/caliptra-ss-mci-mailbox/verify_runs/<finding id>/`.